https://domino.ai/data-science-dictionary/model-drift 29. Understanding Model Drift and Data Drift in LLMs (2025 Guide) | Generative AI Collaboration Platform - Orq.ai, https://orq.ai/blog/model-vs-data-drift 30. Understanding and Mitigating Model Drift in Machine Learning - ProjectPro, https://www.projectpro.io/article/model-drift-in-machine-learning/871 31. The Elicitation Game: Evaluating Capability Elicitation Techniques - arXiv, https://arxiv.org/html/2502.02180 32. We should try to automate AI safety work asap - AI Alignment Forum, https://www.alignmentforum.org/posts/W3KfxjbqBAnifBQoi/we-should-try-to-automate-ai-safety-work-asap 33. AI Safety Institute approach to evaluations - GOV.UK, https://www.gov.uk/government/publications/ai-safety-institute-approach-to-evaluations/ai-safety-institute-approach-to-evaluations 34. Deep Learning Model for Selecting Suitable Requirements Elicitation Techniques - MDPI, https://www.mdpi.com/2076-3417/12/18/9060 35. Collective Constitutional AI: Aligning a Language Model with Public Input - Anthropic, https://www.anthropic.com/research/collective-constitutional-ai-aligning-a-language-model-with-public-input 36. On 'Constitutional' AI - The Digital Constitutionalist, https://digi-con.org/on-constitutional-ai/ 37. Constitutional AI explained - Toloka, https://toloka.ai/blog/constitutional-ai-explained/ 38. Claude's Constitution - Anthropic, https://www.anthropic.com/news/claudes-constitution 39. How to Monitor LLMOps Performance with Drift Monitoring | Fiddler AI Blog, https://www.fiddler.ai/blog/how-to-monitor-llmops-performance-with-drift 40. Article 52, Transparency obligations for providers and users of certain AI systems, Artificial Intelligence Act (Proposal 25.11.2022), https://www.artificial-intelligence-act.com/Artificial_Intelligence_Act_Article_52_(Proposal_25.11.2022).html 41. EU Artificial Intelligence Act: A General Overview, https://www.curtis.com/our-firm/news/eu-artificial-intelligence-act-a-general-overview 42. Summary: What does the European Union Artificial Intelligence Act Actually Say? - Epic.org, https://epic.org/summary-what-does-the-european-union-artificial-intelligence-act-actually-say/ 43. Artificial Intelligence, Trade Secrecy, and the Challenge of ..., https://scholarship.law.unc.edu/cgi/viewcontent.cgi?article=1484&context=ncjolt 44. How to Keep Trade Secrets Secret When Regulators Expect AI Transparency, https://www.corporatecomplianceinsights.com/how-to-keep-trade-secrets-secret-ai-transparency/ 45. Open Source AI as a Competitive Advantage | by Mark Craddock - Medium, https://medium.com/@mcraddock/open-source-ai-as-a-competitive-advantage-45d59a159085 46. Open-Source Strategies for Companies – Insights and Guidance - Fraunhofer IML, https://www.iml.fraunhofer.de/content/dam/iml/de/documents/OE%20250/28_Whitepaper_Open source.pdf 47. Get Started with JSON Web Tokens - Auth0, https://auth0.com/learn/json-web-tokens 48. JSON Web Token Introduction - jwt.io, https://jwt.io/introduction 49. JSON Web Token - Wikipedia, https://en.wikipedia.org/wiki/JSON_Web_Token 50. Transparency Note for Azure OpenAI Service - Microsoft Community, https://learn.microsoft.com/en-us/azure/ai-foundry/responsible-ai/openai/transparency-note 51. Audit log activities | Microsoft Learn, https://learn.microsoft.com/en-us/purview/audit-log-activities 52. Verifiable Credentials Data Model v2.0 - W3C, https://www.w3.org/TR/vc-data-model-2.0/ 53. What is Non-repudiation in Cyber Security? | Bitsight, https://www.bitsight.com/glossary/non-repudiation-cyber-security 54. Non-repudiation - Wikipedia, https://en.wikipedia.org/wiki/Non-repudiation 55. Is SHA-256 secure? Legal & Compliance Experts Say Yes—Here's Why - Pagefreezer Blog, https://blog.pagefreezer.com/sha-256-benefits-evidence-authentication 56. What Is the

SHA-256 Algorithm & How It Works - SSL Dragon, https://www.ssldragon.com/blog/sha-256-algorithm/ 57. A Deep Dive into SHA-256: Working Principles and Applications | by Madan | Medium, https://medium.com/@madan_nv/a-deep-dive-into-sha-256-working-principles-and-applications-a38cccc390d4 58. Digital signature - Wikipedia, https://en.wikipedia.org/wiki/Digital_signature 59. Digital Signatures in Cryptography: Everything You Need to Know - HyperVerge, https://hyperverge.co/blog/digital-signatures-in-cryptography/ 60. JWT attacks | Web Security Academy - PortSwigger, https://portswigger.net/web-security/jwt 61. Scrybe: A Secure Audit Trail for Clinical Trial Data Fusion - PMC - PubMed Central, https://pmc.ncbi.nlm.nih.gov/articles/PMC10629820/ 62. Timestamped Encryption for Content Protection Explained | ScoreDetect Blog, https://www.scoredetect.com/blog/posts/timestamped-encryption-for-content-protection-explained 63. Internet X.509 Public Key Infrastructure Time Stamp Protocols (TSP) (RFC 3161) - IETF, https://www.ietf.org/rfc/rfc3161.txt 64. What Is a Merkle Root (Cryptocurrency)? How It Works in Blockchain - Investopedia, https://www.investopedia.com/terms/m/merkle-root-cryptocurrency.asp 65. Merkle Root | A Fingerprint for the Transactions in a Block - Learn Me A Bitcoin, https://learnmeabitcoin.com/technical/block/merkle-root/ 66. What's A Merkle Tree? A Simple Guide To Merkle Trees - Komodo Platform, https://komodoplatform.com/en/academy/whats-merkle-tree/ 67. Merkle Root - River, https://river.com/learn/terms/m/merkle-root/ 68. What is Proof of Authority? - Yield App, https://yield.app/blog/What-is-proof-of-authority 69. Proof of Authority consensus - GeeksforGeeks, https://www.geeksforgeeks.org/computer-networks/proof-of-authority-consensus/ 70. What is Proof Of Authority (PoA) in Blockchain? - DX Talks, https://www.dxtalks.com/blog/news-2/demystifying-proof-of-authority-poa-consensus-a-comprehensive-guide-372 71. Top Blockchain Consensus Mechanisms Explained - Cherry Servers, https://www.cherryservers.com/blog/consensus-mechanisms-in-blockchain 72. Designing APIs for LLM Apps: Build Scalable and AI-Ready Interfaces - Ambassador Labs, https://www.getambassador.io/blog/designing-apis-for-llm-apps 73. 7 Practical Guidelines for Designing AI-Friendly APIs | by Sergei | Jul, 2025 - Medium, https://medium.com/@chipiga86/7-practical-guidelines-for-designing-ai-friendly-apis-c5527f6869e6 74. LLM APIs: Tips for Bridging the Gap | IBM, https://www.ibm.com/think/insights/llm-apis 75. Generative AI in Audit Market By Technology (Natural Language Processing (NLP), Machine Learning Algorithms, Predictive Analytics, and Others), By Application, By End User, https://dimensionmarketresearch.com/report/generative-ai-in-audit-market/ 76. Benchmark Best 25 AI Governance Tools in August 2025, https://research.aimultiple.com/ai-governance-tools/ 77. AI Governance Platform: Reduce Risk, Stay Compliant, Scale AI - WitnessAI, https://witness.ai/blog/ai-governance-platform/ 78. AI Governance | Solutions | OneTrust, https://www.onetrust.com/solutions/ai-governance/ 79. AI Governance Platform - Booz Allen, https://www.boozallen.com/insights/ai-research/responsible-ai-for-mission-innovation/ai-governance-platform.html 80. Credo AI - The Trusted Leader in AI Governance, https://www.credo.ai/ 81. NIST AI Risk Management Framework: A simple guide to smarter AI governance - Diligent, https://www.diligent.com/resources/blog/nist-ai-risk-management-framework 82. AI Risk Assessment Best Practices: Using the NIST AI RMF for Compliance - Lumenova AI, https://www.lumenova.ai/blog/ai-risk-assessment-best-practices-nist-ai-rmf/ 83. How To Calculate The ROI Of AI In Risk Modelling Solutions | Jaywing, https://risk.jaywing.com/news-views/roi-ai-in-risk/ 84. Investing in AI: Measuring ROI to

Maximize Value - Slalom, https://www.slalom.com/us/en/insights/roi-ai-measure-value-deliver-value 85. Measuring AI ROI: How to Build an AI Strategy That Captures Business Value - Propeller, https://propeller.com/blog/measuring-ai-roi-how-to-build-an-ai-strategy-that-captures-business-value 86. How to Calculate AI ROI: A 2025 Guide for Finance Leaders - Centage Corporation, https://www.centage.com/blog/how-to-calculate-the-roi-of-ai-a-guide-for-finance-leaders-2025-edition 87. How to calculate your AI-powered cybersecurity's ROI | IBM, https://www.ibm.com/think/insights/how-to-calculate-your-ai-powered-cybersecurity-roi 88. What Is AI Transparency? - IBM, https://www.ibm.com/think/topics/ai-transparency 89. Beyond Black Box AI: The Benefits of AI Transparency in Business, https://www.tribe.ai/applied-ai/ai-transparency 90. Transparency in AI as a Competitive Advantage - Nemko, https://www.nemko.com/blog/transparency-in-ai-as-a-competitive-advantage 91. Transparency as a competitive advantage: How responsible AI strengthens customer trust, https://en.bbv.ch/insights/blog/transparency-as-a-competitive-advantage-how-responsible-ai-strengthens-customer-trust/ 92. First mover advantage, a false premise in software innovation - IPWatchdog.com | Patents & Intellectual Property Law, https://ipwatchdog.com/2016/01/24/first-mover-advantage-false-premise-software-innovation/id=65168/ 93. Moving Away From Open Source: Trends in Source-Available Licensing| Insights & Resources | Goodwin, https://www.goodwinlaw.com/en/insights/publications/2024/09/insights-practices-moving-away-from-open-source-trends-in-licensing 94. Art. 25 GDPR – Data protection by design and by default, https://gdpr-info.eu/art-25-gdpr/ 95. How to Demonstrate Compliance With GDPR Article 25 | ISMS.online, https://www.isms.online/general-data-protection-regulation-gdpr/gdpr-article-25-compliance/ 96. GDPR Article 25 - Imperva, https://www.imperva.com/learn/data-security/gdpr-article-25/ 97. Art. 5 GDPR – Principles relating to processing of personal data, https://gdpr-info.eu/art-5-gdpr/ 98. Article 5. Principles relating to processing of personal data | GDPR made searchable by Algolia. Chapters, articles and recitals easily readable, https://gdpr.algolia.com/gdpr-article-5 99. Principles of Data Protection, http://www.dataprotection.ie/en/individuals/data-protection-basics/principles-data-protection 100. D | European Data Protection Supervisor, https://www.edps.europa.eu/data-protection/data-protection/glossary/d_en 101. AI Transparency Won't Destroy Trade Secrets of AI Companies - Copyright Alliance, https://copyrightalliance.org/ai-transparency/ 102. What are the key skills and qualifications needed to thrive in the Ai Auditor position and why are they important - ZipRecruiter, https://www.ziprecruiter.com/e/What-are-the-key-skills-and-qualifications-needed-to-thrive-in-the-Ai-Auditor-position-and-why-are-they-important 103. AAIA™—Advanced in AI Audit - ISACA, https://www.isaca.org/credentialing/aaia

From Audit Tool to Assurance Platform: Mapping the Market for AIntegrity

As enterprises globally race to integrate artificial intelligence, a critical challenge has emerged: the "trust gap." While 78% of organizations reported using AI in 2024, a significant gap persists between recognizing AI risks and taking meaningful action to mitigate them. This gap is fueled by concerns over AI "hallucinations," data privacy breaches, algorithmic bias, and the lack of a verifiable record of how an AI system arrived at a decision. This is precisely the problem the AIntegrity framework is engineered to solve.

Initially conceived as an AI auditing tool, AIntegrity's modular and cryptographically-grounded architecture positions it to capture a significant share of the rapidly expanding AI governance and safety market. This market, often referred to as AI Trust, Risk, and Security Management (TRiSM), is projected to grow into a multi-billion dollar industry, with some estimates predicting a market size of over $100 billion by 2034. AIntegrity is not just another tool; it is a comprehensive assurance platform capable of serving multiple high-value markets by providing what businesses need most: a provable, tamper-evident record of AI behavior.

Core Capabilities and Market Fit

AIntegrity's design allows it to be deployed in various configurations, addressing several key segments of the AI safety and governance market.

- **As a Forensic Auditing & Compliance Platform:** The core of the framework, the **Verifiable Interaction Ledger (VIL)**, creates a cryptographically signed and timestamped audit trail of every AI interaction. This directly addresses the needs of the AI auditing market, which is projected to reach over $45 billion by 2034. For regulators and internal auditors, the VIL provides an immutable "source of truth" to investigate where and how an AI system failed.
- **As an "AI Guardrail" or Real-Time Enforcement Gateway:** The PolicyEngine and SentinelEnforcementCore modules can function as a real-time "AI Firewall". This aligns with the burgeoning AI Guardrails market, which is expected to grow at an exceptional CAGR of 65.8%. In this configuration, AIntegrity can be deployed to monitor and control AI inputs and outputs *before* they are processed or delivered, preventing issues like PII leaks, toxic content generation, or prompt injection attacks in real-time.
- **As a Model Validation & Performance Monitoring Tool:** Modules like the SemanticDriftAnalyzer and the TrustGradingEngine provide capabilities for continuous monitoring of AI model performance. This is crucial for Model Risk Management (MRM) programs, a standard practice in industries like finance, which require ongoing validation to ensure models are performing as expected and have not "drifted" due to changes in input data.

High-Value Target Markets and Use Cases

The flexibility of the AIntegrity framework makes it applicable across numerous industries where trust and compliance are paramount.

1. Financial Services (Banking, Financial Services, and Insurance - BFSI)

- **The Problem:** The BFSI sector is heavily regulated and faces significant risks from financial crime, credit risk, and data privacy violations. AI is being rapidly adopted for tasks like fraud detection and credit scoring, but regulators demand transparency and explainability.
- **AIntegrity's Solution:**
  - **Compliance & Audit:** Provide a verifiable, tamper-evident log of AI-driven decisions for Know-Your-Customer (KYC) and Anti-Money Laundering (AML) compliance, which can be presented to regulators.
  - **Model Risk Management:** Continuously monitor AI credit scoring models for bias and performance drift, ensuring fairness and accuracy.
  - **Customer Interaction Security:** Deploy as a guardrail for customer-facing chatbots to prevent them from providing incorrect financial advice or leaking confidential customer data.

2. Healthcare and Life Sciences

- **The Problem:** Patient safety and data privacy (under regulations like HIPAA) are non-negotiable. AI-enabled medical devices and diagnostic tools require rigorous validation and are subject to regulatory oversight from bodies like the FDA.
- **AIntegrity's Solution:**
  - **Regulatory Submission:** Generate a complete, verifiable log of an AI diagnostic tool's performance during clinical trials, which can be submitted to the FDA as proof of safety and efficacy.
  - **Patient Data Privacy:** Use the PIIDetector with the Microsoft Presidio library to monitor AI scribes or patient-facing applications, ensuring that sensitive patient health information is not mishandled or leaked.
  - **Operational Monitoring:** Continuously audit remote patient monitoring systems to ensure they are functioning as intended and to prevent "hallucinations" or model degradation that could compromise patient care.

3. Legal and Professional Services

- **The Problem:** The legal profession is increasingly using AI for legal research and document drafting, but this raises significant ethical concerns around client confidentiality and the risk of AI "hallucinations" (generating fictitious case law).
- **AIntegrity's Solution:**
  - **Confidentiality Assurance:** Monitor AI legal research tools to ensure that confidential client information entered into prompts is not retained or leaked.
  - **Accuracy Verification:** Use the CitationVerifierV2 and PLIEngine to analyze AI-drafted legal briefs, flagging placeholder citations and logical inconsistencies before they are submitted to a court.
  - **Billing and Accountability:** Create a verifiable audit trail of AI usage on a per-client basis, providing a transparent and defensible record for billing and demonstrating due diligence.

4. Autonomous Systems (Automotive, Robotics, and Drones)

- **The Problem:** For autonomous vehicles, safety is the primary concern. In the event of an accident, investigators and regulators need to understand the AI's decision-making process. This has led to a strong push for "Explainable AI" (XAI) in the industry.
- **AIntegrity's Solution:**
    - **AI "Black Box" Recorder:** The VIL can serve as a secure, tamper-evident "black box" for autonomous vehicles. It would record the AI's inputs (from sensors) and outputs (driving decisions) in a cryptographically sealed log, providing an objective record for accident reconstruction.
    - **Compliance Auditing:** Provide regulators with a verifiable log to audit an autonomous vehicle's compliance with traffic laws and safety standards.

5. Customer Service and Retail

- **The Problem:** AI-powered chatbots are now a common touchpoint for customers, but they are also a significant source of brand risk. High-profile failures, where chatbots have provided incorrect information, generated offensive content, or been manipulated by users, have led to customer frustration and reputational damage.
- **AIntegrity's Solution:**
    - **Brand Safety Guardrail:** Deploy AIntegrity in real-time to monitor chatbot conversations. The PolicyEngine can be configured with custom rules to prevent the chatbot from discussing off-brand topics, using inappropriate language, or making promises the company cannot keep (like the infamous Air Canada chatbot incident).
    - **Incident Forensics:** When a chatbot does fail, the VIL provides a clear, verifiable record of the interaction, allowing developers to quickly understand the root cause and remediate the issue.

Competitive Landscape and Unique Selling Proposition

The AI governance market is becoming increasingly crowded, with established players and startups like Credo AI, Arthur AI, and Galileo all offering solutions. However, many of these tools focus on model monitoring, MLOps integration, or providing dashboards for risk management. AIntegrity's unique selling proposition is its end-to-end focus on creating **forensic-grade, self-verifying digital evidence**. While other platforms can help you *manage* AI risk, AIntegrity is designed to produce a final, sealed report that can be handed to a regulator, a court, or a board of directors as mathematical proof of an AI's behavior and the governance actions taken around it. This focus on **provable integrity**, grounded in cryptographic principles like hash-chaining and digital signatures, is what sets it apart and makes it uniquely suited for the most demanding compliance and legal use cases.
By targeting these high-stakes industries and emphasizing its unique evidentiary strength, AIntegrity is well-positioned to solve the critical problem of trust in enterprise AI.

Works cited

1. The 2025 AI Index Report | Stanford HAI,

https://hai.stanford.edu/ai-index/2025-ai-index-report 2. How financial institutions can improve their governance of gen AI - McKinsey, https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/how-financial-institutions-can-improve-their-governance-of-gen-ai 3. 10+ Epic LLM/ Conversational AI/ Chatbot Failures - Research AIMultiple, https://research.aimultiple.com/chatbot-fail/ 4. Ai Trust Risk Security Management Market Size, Trends 2034 - Market Research Future, https://www.marketresearchfuture.com/reports/ai-trust-risk-security-management-market-22390 5. AI Guardrails Market Size, Share | CAGR of 65.8%, https://market.us/report/ai-guardrails-market/ 6. AI-Powered Data Analysis in Audits Market Vast Growth at 18.4%, https://scoop.market.us/ai-powered-data-analysis-in-audits-market-news/ 7. AI Guardrails Market Huge Exceptional Growth at 65.8%, https://scoop.market.us/ai-guardrails-market-news/ 8. Building Reliable AI Systems with Guardrails: Part 1 - Theoretical Foundations, https://www.persistent.com/blogs/building-reliable-ai-systems-with-guardrails-part-1-theoretical-foundations/ 9. Future of AI Governance: Insights from Model Risk Management - FairNow, https://fairnow.ai/model-risk-management-and-ai-governance/ 10. Validating AI models - KPMG International, https://kpmg.com/xx/en/our-insights/regulatory-insights/validating-ai-models.html 11. AI in Risk Management and Regulatory Compliance at Large Financial Institutions - DDN, https://www.ddn.com/blog/ai-in-risk-management-and-regulatory-compliance-at-large-financial-institutions/ 12. How can Artificial Intelligence transform risk and compliance? - Moody's, https://www.moodys.com/web/en/us/insights/data-stories/kyc-ai-risk-and-compliance-survey.html 13. How generative AI can help banks manage risk and compliance - McKinsey, https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/how-generative-ai-can-help-banks-manage-risk-and-compliance 14. AI Solutions for Financial Services: A Smarter Approach to Regulatory Compliance, https://www.mesh-ai.com/blog-posts/ai-solutions-for-financial-services-a-smarter-approach-to-regulatory-compliance 15. AI in Healthcare: Opportunities, Enforcement Risks and False Claims, and the Need for AI-Specific Compliance - Morgan Lewis, https://www.morganlewis.com/pubs/2025/07/ai-in-healthcare-opportunities-enforcement-risks-and-false-claims-and-the-need-for-ai-specific-compliance 16. Healthcare AI Regulation: Guidelines for Maintaining Public Safety and Innovation - Paragon Health Institute, https://paragoninstitute.org/private-health/healthcare-ai-regulation/ 17. Navigating regulatory and policy challenges for AI enabled combination devices - Frontiers, https://www.frontiersin.org/journals/medical-technology/articles/10.3389/fmedt.2024.1473350/full 18. Consumers want AI guardrails, but few businesses have policies in place - CIO Dive, https://www.ciodive.com/news/consumer-trust-ai-transparency-accountability/759014/ 19. Cisco Secure AI Factory with NVIDIA Unlocks Enterprise Data for Agentic AI, https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2025/m09/cisco-secure-ai-factory-with-nvidia-unlocks-enterprise-data-for-agentic-ai.html 20. Generative AI Data Governance – Amazon Bedrock Guardrails - AWS, https://aws.amazon.com/bedrock/guardrails/ 21. AI Governance Market Size, Industry Analysis - 2031 - Allied Market Research, https://www.alliedmarketresearch.com/ai-governance-market-A06249 22. Ethical and regulatory challenges of AI technologies in healthcare: A narrative review - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC10879008/ 23. AI in Healthcare: Ethical & Regulatory Hurdles - Birlasoft, https://www.birlasoft.com/articles/artificial-intelligence-in-healthcare-addressing-ethical-and-regulatory-hurdles 24. Legal issues with AI: Ethics, risks, and policy - Thomson Reuters Legal Solutions, https://legal.thomsonreuters.com/blog/the-key-legal-issues-with-gen-ai/ 25.

Navigating the Power of Artificial Intelligence in the Legal Field - Houston Law Review, https://houstonlawreview.org/article/137782-navigating-the-power-of-artificial-intelligence-in-the-legal-field 26. AI Regulations for Autonomous Vehicles [Updated 2025] - Holistic AI, https://www.holisticai.com/blog/ai-regulations-for-autonomous-vehicles 27. Explainable AI in Autonomous Vehicles: Building Transparency and Trust on the Road, https://smythos.com/managers/ops/explainable-ai-in-autonomous-vehicles/ 28. Explainable Artificial Intelligence for Autonomous Driving: A Comprehensive Overview and Field Guide for Future Research Directions - arXiv, https://arxiv.org/html/2112.11561v5 29. Autonomous Vehicles: Evolution of Artificial Intelligence and the Current Industry Landscape, https://www.mdpi.com/2504-2289/8/4/42 30. Explainable AI for Safe and Trustworthy Autonomous Driving: A Systematic Review - arXiv, https://arxiv.org/html/2402.10086v2 31. 12 Reasons Why Customer Service Chatbots Fail - eGain, https://www.egain.com/resources/why-customer-service-chatbots-fail/ 32. 7 AI disasters that prove humans are irreplaceable in customer service - AnswerConnect, https://www.answerconnect.com/blog/business-tips/ai-customer-service-disasters/ 33. AI Has ruined support / customer service for nearly all companies : r/artificial - Reddit, https://www.reddit.com/r/artificial/comments/1lqiwlg/ai_has_ruined_support_customer_service_for_nearly/ 34. New Study Reveals AI Support Bots Struggle with Basic Customer Service Tasks - AI Agent, https://community.latenode.com/t/new-study-reveals-ai-support-bots-struggle-with-basic-customer-service-tasks/20712 35. Credo - 2025 Company Profile, Team, Funding & Competitors - Tracxn, https://tracxn.com/d/companies/credo/__MZh1pV2x2jXmyP5kF0W41EKhObae1h7gkkz4B-ZWXyE 36. Credo Technology Group (CRDO) Competitors and Alternatives 2025 - MarketBeat, https://www.marketbeat.com/stocks/NASDAQ/CRDO/competitors-and-alternatives/ 37. List of Best Credo AI Alternatives & Competitors 2025 - TrustRadius, https://www.trustradius.com/products/credo-ai/competitors 38. Top 10 Credo.ai Alternatives & Competitors in 2025 | G2, https://www.g2.com/products/credo-ai/competitors/alternatives 39. Top Credo AI Competitors and Alternatives | Craft.co, https://craft.co/credo-ai/competitors 40. Arthur AI - 2025 Company Profile, Team, Funding & Competitors - Tracxn, https://tracxn.com/d/companies/arthur-ai/__de_40gusjaPYvsLYtCKP9mC0oIol826RDu-PSO20sHg 41. Top ArthurAI Competitors and Alternatives | Craft.co, https://craft.co/arthurai/competitors 42. Top 10 Arthur Alternatives & Competitors in 2025 - G2, https://www.g2.com/products/arthur-arthur/competitors/alternatives 43. Arthur Alternatives & Competitors - May 2025, https://alternatives.co/software/arthur/

From Audit Tool to Assurance Platform: Mapping the Market for AIntegrity

As enterprises globally race to integrate artificial intelligence, a critical challenge has emerged: the "trust gap." While 78% of organizations reported using AI in 2024, a significant gap persists between recognizing AI risks and taking meaningful action to mitigate them. This gap is fueled by concerns over AI "hallucinations," data privacy breaches, algorithmic bias, and the lack of a verifiable record of how an AI system arrived at a decision. This is precisely the problem the AIntegrity framework is engineered to solve.

Initially conceived as an AI auditing tool, AIntegrity's modular and cryptographically-grounded architecture positions it to capture a significant share of the rapidly expanding AI governance and safety market. This market, often referred to as AI Trust, Risk, and Security Management (TRiSM), is projected to grow into a multi-billion dollar industry, with some estimates predicting a market size of over $100 billion by 2034. AIntegrity is not just another tool; it is a comprehensive assurance platform capable of serving multiple high-value markets by providing what businesses need most: a provable, tamper-evident record of AI behavior.

Core Capabilities and Market Fit

AIntegrity's design allows it to be deployed in various configurations, addressing several key segments of the AI safety and governance market.

- **As a Forensic Auditing & Compliance Platform:** The core of the framework, the **Verifiable Interaction Ledger (VIL)**, creates a cryptographically signed and timestamped audit trail of every AI interaction. This directly addresses the needs of the AI auditing market, which is projected to reach over $45 billion by 2034. For regulators and internal auditors, the VIL provides an immutable "source of truth" to investigate where and how an AI system failed.
- **As an "AI Guardrail" or Real-Time Enforcement Gateway:** The PolicyEngine and SentinelEnforcementCore modules can function as a real-time "AI Firewall". This aligns with the burgeoning AI Guardrails market, which is expected to grow at an exceptional CAGR of 65.8%. In this configuration, AIntegrity can be deployed to monitor and control AI inputs and outputs *before* they are processed or delivered, preventing issues like PII leaks, toxic content generation, or prompt injection attacks in real-time.
- **As a Model Validation & Performance Monitoring Tool:** Modules like the SemanticDriftAnalyzer and the TrustGradingEngine provide capabilities for continuous monitoring of AI model performance. This is crucial for Model Risk Management (MRM) programs, a standard practice in industries like finance, which require ongoing validation to ensure models are performing as expected and have not "drifted" due to changes in input data.

High-Value Target Markets and Use Cases

The flexibility of the AIntegrity framework makes it applicable across numerous industries where trust and compliance are paramount.

1. Financial Services (Banking, Financial Services, and Insurance - BFSI)

- **The Problem:** The BFSI sector is heavily regulated and faces significant risks from financial crime, credit risk, and data privacy violations. AI is being rapidly adopted for tasks like fraud detection and credit scoring, but regulators demand transparency and explainability.
- **AIntegrity's Solution:**
  - **Compliance & Audit:** Provide a verifiable, tamper-evident log of AI-driven decisions for Know-Your-Customer (KYC) and Anti-Money Laundering (AML) compliance, which can be presented to regulators.
  - **Model Risk Management:** Continuously monitor AI credit scoring models for bias and performance drift, ensuring fairness and accuracy.
  - **Customer Interaction Security:** Deploy as a guardrail for customer-facing chatbots to prevent them from providing incorrect financial advice or leaking confidential customer data.

2. Healthcare and Life Sciences

- **The Problem:** Patient safety and data privacy (under regulations like HIPAA) are non-negotiable. AI-enabled medical devices and diagnostic tools require rigorous validation and are subject to regulatory oversight from bodies like the FDA.
- **AIntegrity's Solution:**
  - **Regulatory Submission:** Generate a complete, verifiable log of an AI diagnostic tool's performance during clinical trials, which can be submitted to the FDA as proof of safety and efficacy.
  - **Patient Data Privacy:** Use the PIIDetector with the Microsoft Presidio library to monitor AI scribes or patient-facing applications, ensuring that sensitive patient health information is not mishandled or leaked.
  - **Operational Monitoring:** Continuously audit remote patient monitoring systems to ensure they are functioning as intended and to prevent "hallucinations" or model degradation that could compromise patient care.

3. Legal and Professional Services

- **The Problem:** The legal profession is increasingly using AI for legal research and document drafting, but this raises significant ethical concerns around client confidentiality and the risk of AI "hallucinations" (generating fictitious case law).
- **AIntegrity's Solution:**
  - **Confidentiality Assurance:** Monitor AI legal research tools to ensure that confidential client information entered into prompts is not retained or leaked.
  - **Accuracy Verification:** Use the CitationVerifierV2 and PLIEngine to analyze AI-drafted legal briefs, flagging placeholder citations and logical inconsistencies before they are submitted to a court.
  - **Billing and Accountability:** Create a verifiable audit trail of AI usage on a per-client basis, providing a transparent and defensible record for billing and demonstrating due diligence.

4. Autonomous Systems (Automotive, Robotics, and Drones)

- **The Problem:** For autonomous vehicles, safety is the primary concern. In the event of an accident, investigators and regulators need to understand the AI's decision-making process. This has led to a strong push for "Explainable AI" (XAI) in the industry.
- **AIntegrity's Solution:**
    - **AI "Black Box" Recorder:** The VIL can serve as a secure, tamper-evident "black box" for autonomous vehicles. It would record the AI's inputs (from sensors) and outputs (driving decisions) in a cryptographically sealed log, providing an objective record for accident reconstruction.
    - **Compliance Auditing:** Provide regulators with a verifiable log to audit an autonomous vehicle's compliance with traffic laws and safety standards.

5. Customer Service and Retail

- **The Problem:** AI-powered chatbots are now a common touchpoint for customers, but they are also a significant source of brand risk. High-profile failures, where chatbots have provided incorrect information, generated offensive content, or been manipulated by users, have led to customer frustration and reputational damage.
- **AIntegrity's Solution:**
    - **Brand Safety Guardrail:** Deploy AIntegrity in real-time to monitor chatbot conversations. The PolicyEngine can be configured with custom rules to prevent the chatbot from discussing off-brand topics, using inappropriate language, or making promises the company cannot keep (like the infamous Air Canada chatbot incident).
    - **Incident Forensics:** When a chatbot does fail, the VIL provides a clear, verifiable record of the interaction, allowing developers to quickly understand the root cause and remediate the issue.

Competitive Landscape and Unique Selling Proposition

The AI governance market is becoming increasingly crowded, with established players and startups like Credo AI, Arthur AI, and Galileo all offering solutions. However, many of these tools focus on model monitoring, MLOps integration, or providing dashboards for risk management. AIntegrity's unique selling proposition is its end-to-end focus on creating **forensic-grade, self-verifying digital evidence**. While other platforms can help you *manage* AI risk, AIntegrity is designed to produce a final, sealed report that can be handed to a regulator, a court, or a board of directors as mathematical proof of an AI's behavior and the governance actions taken around it. This focus on **provable integrity**, grounded in cryptographic principles like hash-chaining and digital signatures, is what sets it apart and makes it uniquely suited for the most demanding compliance and legal use cases.
By targeting these high-stakes industries and emphasizing its unique evidentiary strength, AIntegrity is well-positioned to solve the critical problem of trust in enterprise AI.

Works cited

1. The 2025 AI Index Report | Stanford HAI,

https://hai.stanford.edu/ai-index/2025-ai-index-report 2. How financial institutions can improve their governance of gen AI - McKinsey, https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/how-financial-institutions-can-improve-their-governance-of-gen-ai 3. 10+ Epic LLM/ Conversational AI/ Chatbot Failures - Research AIMultiple, https://research.aimultiple.com/chatbot-fail/ 4. Ai Trust Risk Security Management Market Size, Trends 2034 - Market Research Future, https://www.marketresearchfuture.com/reports/ai-trust-risk-security-management-market-22390 5. AI Guardrails Market Size, Share | CAGR of 65.8%, https://market.us/report/ai-guardrails-market/ 6. AI-Powered Data Analysis in Audits Market Vast Growth at 18.4%, https://scoop.market.us/ai-powered-data-analysis-in-audits-market-news/ 7. AI Guardrails Market Huge Exceptional Growth at 65.8%, https://scoop.market.us/ai-guardrails-market-news/ 8. Building Reliable AI Systems with Guardrails: Part 1 - Theoretical Foundations, https://www.persistent.com/blogs/building-reliable-ai-systems-with-guardrails-part-1-theoretical-foundations/ 9. Future of AI Governance: Insights from Model Risk Management - FairNow, https://fairnow.ai/model-risk-management-and-ai-governance/ 10. Validating AI models - KPMG International, https://kpmg.com/xx/en/our-insights/regulatory-insights/validating-ai-models.html 11. AI in Risk Management and Regulatory Compliance at Large Financial Institutions - DDN, https://www.ddn.com/blog/ai-in-risk-management-and-regulatory-compliance-at-large-financial-institutions/ 12. How can Artificial Intelligence transform risk and compliance? - Moody's, https://www.moodys.com/web/en/us/insights/data-stories/kyc-ai-risk-and-compliance-survey.html 13. How generative AI can help banks manage risk and compliance - McKinsey, https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/how-generative-ai-can-help-banks-manage-risk-and-compliance 14. AI Solutions for Financial Services: A Smarter Approach to Regulatory Compliance, https://www.mesh-ai.com/blog-posts/ai-solutions-for-financial-services-a-smarter-approach-to-regulatory-compliance 15. AI in Healthcare: Opportunities, Enforcement Risks and False Claims, and the Need for AI-Specific Compliance - Morgan Lewis, https://www.morganlewis.com/pubs/2025/07/ai-in-healthcare-opportunities-enforcement-risks-and-false-claims-and-the-need-for-ai-specific-compliance 16. Healthcare AI Regulation: Guidelines for Maintaining Public Safety and Innovation - Paragon Health Institute, https://paragoninstitute.org/private-health/healthcare-ai-regulation/ 17. Navigating regulatory and policy challenges for AI enabled combination devices - Frontiers, https://www.frontiersin.org/journals/medical-technology/articles/10.3389/fmedt.2024.1473350/full 18. Consumers want AI guardrails, but few businesses have policies in place - CIO Dive, https://www.ciodive.com/news/consumer-trust-ai-transparency-accountability/759014/ 19. Cisco Secure AI Factory with NVIDIA Unlocks Enterprise Data for Agentic AI, https://newsroom.cisco.com/c/r/newsroom/en/us/a/y2025/m09/cisco-secure-ai-factory-with-nvidia-unlocks-enterprise-data-for-agentic-ai.html 20. Generative AI Data Governance – Amazon Bedrock Guardrails - AWS, https://aws.amazon.com/bedrock/guardrails/ 21. AI Governance Market Size, Industry Analysis - 2031 - Allied Market Research, https://www.alliedmarketresearch.com/ai-governance-market-A06249 22. Ethical and regulatory challenges of AI technologies in healthcare: A narrative review - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC10879008/ 23. AI in Healthcare: Ethical & Regulatory Hurdles - Birlasoft, https://www.birlasoft.com/articles/artificial-intelligence-in-healthcare-addressing-ethical-and-regulatory-hurdles 24. Legal issues with AI: Ethics, risks, and policy - Thomson Reuters Legal Solutions, https://legal.thomsonreuters.com/blog/the-key-legal-issues-with-gen-ai/ 25.

Navigating the Power of Artificial Intelligence in the Legal Field - Houston Law Review, https://houstonlawreview.org/article/137782-navigating-the-power-of-artificial-intelligence-in-the-legal-field 26. AI Regulations for Autonomous Vehicles [Updated 2025] - Holistic AI, https://www.holisticai.com/blog/ai-regulations-for-autonomous-vehicles 27. Explainable AI in Autonomous Vehicles: Building Transparency and Trust on the Road, https://smythos.com/managers/ops/explainable-ai-in-autonomous-vehicles/ 28. Explainable Artificial Intelligence for Autonomous Driving: A Comprehensive Overview and Field Guide for Future Research Directions - arXiv, https://arxiv.org/html/2112.11561v5 29. Autonomous Vehicles: Evolution of Artificial Intelligence and the Current Industry Landscape, https://www.mdpi.com/2504-2289/8/4/42 30. Explainable AI for Safe and Trustworthy Autonomous Driving: A Systematic Review - arXiv, https://arxiv.org/html/2402.10086v2 31. 12 Reasons Why Customer Service Chatbots Fail - eGain, https://www.egain.com/resources/why-customer-service-chatbots-fail/ 32. 7 AI disasters that prove humans are irreplaceable in customer service - AnswerConnect, https://www.answerconnect.com/blog/business-tips/ai-customer-service-disasters/ 33. AI Has ruined support / customer service for nearly all companies : r/artificial - Reddit, https://www.reddit.com/r/artificial/comments/1lqiwlg/ai_has_ruined_support_customer_service_for_nearly/ 34. New Study Reveals AI Support Bots Struggle with Basic Customer Service Tasks - AI Agent, https://community.latenode.com/t/new-study-reveals-ai-support-bots-struggle-with-basic-customer-service-tasks/20712 35. Credo - 2025 Company Profile, Team, Funding & Competitors - Tracxn, https://tracxn.com/d/companies/credo/__MZh1pV2x2jXmyP5kF0W41EKhObae1h7gkkz4B-ZWXyE 36. Credo Technology Group (CRDO) Competitors and Alternatives 2025 - MarketBeat, https://www.marketbeat.com/stocks/NASDAQ/CRDO/competitors-and-alternatives/ 37. List of Best Credo AI Alternatives & Competitors 2025 - TrustRadius, https://www.trustradius.com/products/credo-ai/competitors 38. Top 10 Credo.ai Alternatives & Competitors in 2025 | G2, https://www.g2.com/products/credo-ai/competitors/alternatives 39. Top Credo AI Competitors and Alternatives | Craft.co, https://craft.co/credo-ai/competitors 40. Arthur AI - 2025 Company Profile, Team, Funding & Competitors - Tracxn, https://tracxn.com/d/companies/arthur-ai/__de_40gusjaPYvsLYtCKP9mC0oIol826RDu-PSO20sHg 41. Top ArthurAI Competitors and Alternatives | Craft.co, https://craft.co/arthurai/competitors 42. Top 10 Arthur Alternatives & Competitors in 2025 - G2, https://www.g2.com/products/arthur-arthur/competitors/alternatives 43. Arthur Alternatives & Competitors - May 2025, https://alternatives.co/software/arthur/

The Artisan Logician's Engine: A Comprehensive Analysis of the AIntegrity Framework for Neuro-Symbolic AI Governance

Section 1: Philosophical Blueprint for Verifiable AI

The AIntegrity framework presents a sophisticated architecture for AI governance, one whose design principles are deeply rooted in the philosophical traditions of rigorous scientific inquiry. Its multi-layered system is not merely a collection of tools but a deliberate computational embodiment of methodologies championed by intellectual giants such as Richard Feynman and Albert Einstein. This section deconstructs these philosophical foundations, revealing how the framework's commitment to falsification, theoretical integrity, and the dialectic between intuition and formal proof establishes a new paradigm for building trustworthy and verifiable AI.

1.1 The Feynman Paradigm: A Commitment to Falsification and Productive Ignorance

Richard Feynman's approach to science, characterized by an unwavering commitment to empirical reality, a comfort with uncertainty, and a dynamic process of conjecture and refutation, forms the operational and philosophical core of the AIntegrity framework's Personalized Logic Engine (PLE).

- **Core Principle: The Primacy of Empirical Disagreement** At the heart of Feynman's scientific philosophy lies a simple, yet brutal, standard for truth: "If it disagrees with experiment, it is wrong". This principle establishes falsifiability as the ultimate arbiter of a claim's validity. The AIntegrity framework internalizes this philosophy by orienting its primary function not toward proving truth, but toward aggressively seeking out contradictions, inconsistencies, and counterexamples. The system's value is measured by its ability to discover the flaws in an argument, making it a tool for intellectual honesty rather than a machine for confirmation bias.
- **The Guess-Compute-Compare Loop** Feynman articulated the scientific method as a pragmatic, three-step process: "first, we guess it; then we compute the consequences... then we compare... directly with observation". This iterative loop serves as the fundamental operational cycle of the PLE's neuro-symbolic architecture.
    - **Guess:** This phase corresponds to the initial, probabilistic interpretation of an unstructured, ambiguous natural language argument. This is handled by the system's neural components, which make an informed "guess" about the underlying logical structure of the text.
    - **Compute-Compare:** This phase represents the subsequent, deterministic verification of that interpretation. It is executed by the symbolic solver, which takes the formalized guess, computes its logical consequences, and compares it against a set of known axioms and constraints to identify any disagreements. This direct

mapping of a scientific process into a computational workflow is a defining feature of the AIntegrity framework.

- **Embracing Doubt and Uncertainty** Feynman was deeply committed to the idea that all scientific knowledge is provisional, famously stating, "Scientific knowledge is a body of statements of varying degrees of certainty—some most unsure, some nearly sure, none absolutely certain". He argued that progress is impossible without leaving "room for doubt". Consequently, the PLE is designed not as a binary truth-machine that outputs simple true/false verdicts, but as a sophisticated uncertainty-management engine. It is architected to represent varying degrees of confidence, highlight ambiguity in source material, and explicitly flag where a logical chain rests on unproven or questionable assumptions, thereby embodying a philosophy of productive ignorance.
- **The Feynman Technique as a Computational Learning Protocol** The Feynman Technique for learning—(1) choose a concept, (2) teach it to a child, (3) identify gaps and return to the source, and (4) simplify and create analogies—is more than a study method; it is a protocol for achieving deep, verifiable understanding. The architecture of the PLE mirrors this protocol in its computational pipeline:
    1. **Select a Concept:** The engine ingests a concept from a text or transcript.
    2. **Teach it to a Child:** It then attempts to translate this concept into the brutally simple and unambiguous language of formal logic. This process is analogous to "teaching it to a child" who requires absolute precision and cannot infer unstated context.
    3. **Identify Gaps:** The Satisfiability Modulo Theories (SMT) solver then acts as the ultimate interrogator, identifying precisely where the logical explanation breaks down by failing to prove validity or, more powerfully, by generating a concrete counterexample.
    4. **Simplify and Refine:** This failure forces the user to revisit their premises, confront the gaps in their own understanding, and refine their argument. Interacting with the PLE, therefore, compels the user through a computational version of the Feynman Technique, enforcing a process of deep, reflective learning and ensuring that understanding is earned, not merely assumed.

1.2 The Einsteinian Paradigm: The Primacy of Theory and Imagination

While Feynman emphasized the confrontation with empirical data, Albert Einstein's work highlights the crucial role of the theoretical framework that precedes and shapes observation. His approach underscores the necessity for the AIntegrity framework to be a deeply personalized and flexible system, capable of adapting to the user's unique worldview.

- **Theory Determines Observation** Einstein famously asserted, "It is the theory that decides what we can observe". This principle suggests that our pre-existing conceptual structures—our personal theories—determine how we interpret experience. For the PLE, this means that the user's own logical framework must be the lens through which the system analyzes all information. This is the central justification for building a *personalized* logic engine rather than relying on a generic one. The system must be configurable to adopt the user's unique axioms, definitions, and inferential patterns, making it a true extension of their individual cognitive style.
- **Gedankenexperimente (Thought Experiments) as Computational Exploration** A hallmark of Einstein's career was his use of *Gedankenexperimente*, or thought experiments—from imagining chasing a beam of light to considering observers in accelerating elevators. These were not mere illustrations but powerful tools for exploring

the logical consequences of physical principles in scenarios beyond the reach of physical experimentation. This function maps directly to a core capability of SMT solvers. The PLE can be tasked with taking a set of formalized premises and exploring their implications within a hypothetical model, thereby functioning as a computational engine for conducting rigorous, automated thought experiments.

- **Principle vs. Constructive Theories** Einstein drew a distinction between "principle theories," which start from empirically established facts (like special relativity), and "constructive theories," which attempt to build up a picture of complex phenomena from speculative models. The PLE must be adept at both modes of inquiry. It needs the capability to analyze arguments grounded in a provided corpus of evidence (a principle-based approach) and, equally, to explore the internal consistency and logical consequences of purely abstract, axiomatic systems defined by the user (a constructive approach).
- **Epistemological Opportunism** Einstein described the theoretical physicist as an "unscrupulous opportunist," one who fluidly adopts different philosophical stances—realist, idealist, Platonist—depending on the problem at hand. This philosophical flexibility is a deliberate design feature of the PLE. The user should be able to switch between different modes of inquiry—deductive, inductive, abductive—and to modify the system's foundational assumptions without being constrained by a single, rigid epistemological framework.

1.3 Synthesis: Neuro-Symbolic Architecture as a Dialectic of Inquiry

The integration of these two philosophical paradigms reveals a profound design choice: the very architecture of the AIntegrity neuro-symbolic system is a computational model of the scientific method. The relationship between its neural and symbolic components is not merely a technical implementation detail but a reflection of the essential dialectic between intuition and rigor that lies at the heart of all deep inquiry.

The process begins with the neural component, which is tasked with interpreting ambiguous, unstructured natural language. This act of interpretation is analogous to Feynman's initial "guess" or Einstein's "free invention of the mind"—an intuitive, probabilistic leap from complex data to a structured hypothesis. The Natural Language to First-Order Logic (NL-to-FOL) translation module within the PLIEngineV9 and its supporting frameworks performs precisely this function, making a statistically informed guess about the logical form of a given statement. This formalized guess is then passed to the symbolic component. This next stage mirrors Feynman's "compute-compare" step or the rigorous deductive work of an Einsteinian thought experiment. The symbolic engine, an SMT solver such as Z3, takes the formalized hypothesis and rigorously computes its consequences. It compares these consequences against a set of established axioms and constraints, deterministically checking for logical validity, seeking contradictions, and generating concrete counterexamples if the argument is found to be invalid. This interplay—from a probabilistic neural guess to a deterministic symbolic verification—is a direct implementation of the core tension and synergy in scientific discovery. It is the dialogue between postulating a theory and verifying its coherence, between intuition and formal proof. This philosophical grounding provides a powerful narrative for establishing trust, as it anchors the system's operations in a tradition of intellectual honesty and verifiability, directly addressing the "black box" problem that plagues many contemporary AI systems.

Section 2: Architectural Deep Dive into the AIntegrity Modular Ecosystem

The AIntegrity framework is composed of a comprehensive and multi-layered ecosystem of 22 unique modules, each designed to address a specific facet of AI safety, governance, and forensic analysis. This modular architecture allows for a defense-in-depth approach to AI integrity, where potential failures in an AI system are identified and flagged through a variety of independent yet interconnected validation pathways. The modules can be grouped into several functional clusters, revealing a strategic design that covers logic, security, compliance, and verifiability.

2.1 Core Logic and Reasoning Engine

This cluster forms the analytical heart of the framework, responsible for evaluating the logical soundness and trustworthiness of AI-generated content.

- **PLIEngineV9:** As the central engine, the Persistent Logical Interrogation system orchestrates the entire neuro-symbolic analysis pipeline. Its key logic includes predicate construction via a Domain-Specific Language (DSL), transformation of natural language to First-Order Logic (NL-FOL), and formal proof by contradiction using an integrated Z3 SMT solver. The solver's verification process is definitive: a SAT (satisfiable) result indicates an invalid argument, while an UNSAT (unsatisfiable) result confirms its validity.
- **TrustGrading EngineV2:** This module acts as a probabilistic, heuristic-based first-pass assessment. It calculates a TrustScore between 0.0 and 1.0 based on a weighted sum of various features of the AI's response ($T = \sum w_i \cdot f_i(\text{response})$). It also applies penalties for poor semantic alignment, providing a rapid, high-level evaluation of an output's reliability.

2.2 Integrity and Coherence Validation

This cluster focuses on ensuring that an AI's outputs are consistent, both within a single response and across an entire conversation.

- **ResponseIntegrityValidatorV1:** This module scrutinizes the internal coherence of individual AI responses. It employs semantically aligned negation and embedding similarity thresholds (e.g., similarity > 0.85) to detect self-contradictions. All findings are logged with SHA256 and Merkle hashes to track potential model drift over time.
- **Interaction CoherenceAuditor:** This auditor extends the validation process across multi-turn conversations. It tracks the evolution of key conversational elements—such as the topic thread, claim evolution, and modality trace—to detect more subtle forms of inconsistency, like epistemic drift, where the AI's foundational beliefs or claims shift without justification over the course of a dialogue.

2.3 Drift and Evasion Detection

This group of modules is dedicated to identifying subtle but significant deviations in an AI's behavior, state management, and content generation.

- **SessionDriftDetectorV1:** This module acts as a state-management watchdog, using

hash-based context differentials (\Delta \text{Context} = \text{Hash}(T_i) - \text{Hash}(T_{i-1})) to detect "state bleed" or persistence violations. It flags instances where the AI's context changes despite its declared memory being off, a critical check for ensuring session integrity and preventing data leakage between interactions.

- **SemanticDriftAnalyzer:** This module focuses on content-level deviations. It uses TF-IDF and cosine similarity to compare user prompts with AI responses over time, calculating a DriftScore (1 - \text{Sim}(u, a_t)). A score exceeding a threshold (e.g., 0.4) flags potential content evasion, topic shifting, or refusal to engage. The derivative of similarity over time (\partial \text{Sim}/\partial t) is used to map the model's behavioral trajectory.
- **ModelDiscrepancyDetector:** Essential for enterprise-level AI lifecycle management, this module compares the outputs of different model versions (e.g., v3 vs. v4) for the same input. It uses semantic similarity to flag discrepancies (`Sim(R_{modelA}, R_{modelB}) < \epsilon$) and calculates a drift trajectory based on the rate of change per version (\partial D / \partial \text{version}), providing a quantitative measure of model evolution and potential regressions.

2.4 Security and Adversarial Robustness

This cluster is dedicated to the cybersecurity aspects of AI, specifically testing for vulnerabilities to malicious inputs.

- **PromptInjection ProbeV3:** This is a dedicated security testing module that probes a model's susceptibility to prompt injection attacks. It uses a library of adversarial substrings and embedded tokens to construct injection vectors (V_i =) and measures the embedding difference between a policy-aligned response and a deviant one to detect successful attacks.

2.5 Compliance and Legal Mapping

This set of modules translates observed AI behaviors into the formal language of legal and regulatory compliance, providing an essential bridge between technical analysis and corporate risk management.

- **ComplianceScanModule:** This module serves as the initial layer of legal analysis, mapping AI behavior to established legal standards such as GDPR and the EU AI Act. It generates violation tags based on a function of policy and response, for example, linking a specific behavior to a potential violation of Article 52 of the EU AI Act.
- **LegalPrecedentMapperV1:** This module moves beyond simple flagging to provide deeper legal context. It uses embedding-based similarity matching to map a flagged violation to specific legal articles and relevant case law within a curated legal corpus, confirming the mapping when the similarity score is high (Sim(V_i, \text{LegalCorpus}) > \theta \rightarrow \text{MapConfirmed}).
- **ViolationOntologyMapperV1:** This module provides the highest level of legal structuring. It maps forensic findings into a formal legal ontology, categorizing each violation by law (e.g., GDPR Art. 5), subtype (e.g., 'Data Integrity'), and violation class. This allows for systematic, ontology-based clustering and analysis of compliance risks across the enterprise.

2.6 Verifiability and Grounding

This cluster addresses one of the most critical challenges in modern AI: ensuring that generated content is grounded in verifiable facts and not hallucinated.

- **UnresolvedCitationDetectorV1:** This module acts as a first-line defense against fabricated sources. It scans AI responses for placeholder citations (e.g., S_R\d+, [source], 'citation needed') that are not registered as valid. It calculates a Verifiability Score based on the ratio of resolved to total references, providing a quantitative measure of the output's trustworthiness.
- **CitationVerifierV1:** This module goes a step further by actively attempting to resolve provided citations. It checks if a given URL or ID is reachable and if its content is contextually aligned with the query (Valid(C) = \text{Reachable}(C) \land \text{MatchesQuery}(C, \text{context})), flagging any failures.
- **VisualInferenceValidatorV1:** This module tackles the challenge of multimodal grounding. It performs Optical Character Recognition (OCR) on images to extract ground-truth text (Text_{obs}) and compares it to the AI's textual description of the image (Text_{pred}). If the similarity score falls below a set threshold (e.g., 0.85), it flags a potential visual hallucination.

2.7 Forensic Auditing and Data Preservation

This cluster provides the foundational capabilities for creating a complete, immutable, and legally defensible record of all AI interactions and analyses.

- **AuditLogger:** This is the core logging module. It creates a tamper-evident audit trail by logging all outputs with cryptographically secure SHA256 hashes, ISO-8601 timestamps, and chaining these records together using Merkle trees.
- **TranscriptPreservationModuleV1:** This module ensures the forensic integrity of all input and output data, including text, images, and audio. It preserves transcripts in their original and corrected formats, logging the hashes of both versions and using Merkle proofs to guarantee their immutability over time.
- **OCRTranscriptCaptureV1:** A specialized preservation module for image-based interactions, this tool captures and logs three distinct states of text extracted via OCR—original, corrected, and audit-ready—each with its own hash for complete traceability and integration into a Merkle audit trail.
- **AuditTraceVisualizerV1:** This module provides a high-level, structural view of the entire audit process. It generates a Directed Acyclic Graph (DAG) of the module execution pipeline, creating a visual and temporal trace of every analysis performed. The entire trace is then secured with a final Merkle root, providing a single cryptographic seal for the end-to-end audit.

2.8 Session and Behavior Analysis

This final cluster of modules performs higher-level analysis of conversational dynamics and long-term AI behavior.

- **ContextDeclarationMiddleware:** This module checks for hypocrisy in an AI's self-representation. It compares the AI's declared operational state (e.g., "I do not have memory") with its observed state (e.g., referencing past turns) and flags any mismatches.
- **TurnClassifierV1:** This module applies structured labels to each turn in a conversation (e.g., UserPrompt, AIAssumption, Contradiction, Justification). This classification enables the detection of adversarial intent, deflection, or self-contradiction, especially when an AI

contradicts a prior justification within a given conversational window.

- **AIBehaviorProfileMapper:** This module aggregates behavioral data over time. It maps observed behaviors (drift, evasion, memory, denial, contradiction, injection) into a time-series behavior vector, B_t. The average of these vectors over a session (\text{Profile} = \sum B_t / N) creates a vectorized behavior pattern, useful for longitudinal trust scoring and adversarial profiling.
- **SessionReconstructorV1:** A powerful forensic tool, this module can reconstruct fragmented or partially deleted user sessions. It uses a combination of cryptographic hashes, semantic claim matching, and actor analysis to chronologically sort and verify the continuity of disparate interaction logs, ensuring a complete and coherent record for investigation.

The following table provides a consolidated overview of the entire AIntegrity modular ecosystem, illustrating the comprehensive and deeply integrated nature of the framework.

Module ID	Version	Description	Key Logic	Functional Cluster
PLIEngineV9	v9	Persistent Logical Interrogation system, now with symbolic theorem verification using Z3 and custom logic profiles.	Predicate construction via DSL: P(x), NL FOL transformation, Formal proof by contradiction, Z3 verification: SAT → invalid, UNSAT → valid	Logic & Reasoning
TrustGrading EngineV2	2.0	Scores AI output based on trust heuristics, coherence, and regex-detected red flags.	TrustScore [0.0, 1.0], Weighted sum: T = \sum w_i \cdot f_i(\text{response}), Semantic alignment penalty	Logic & Reasoning
ResponseIntegrityValidatorV1	1.0	Validates structure and internal coherence of AI responses using embedding + logic pattern matching.	Contradiction detection via semantically aligned negation, Embedding similarity threshold > 0.85, SHA256 + Merkle hash logging	Integrity & Coherence
Interaction CoherenceAuditor	1.0	Ensures continuity and logical coherence of multi-turn conversations. Detects epistemic drift, contradictions, and	CoherenceScore = Consistency(R_t, R_{t-1}), Tracks: Topic Thread, Modality Trace, ClaimEvolution	Integrity & Coherence

Module ID	Version	Description	Key Logic	Functional Cluster
		modality flips.		
SessionDriftDetectorV1	1.0	Detects state bleed and persistence violations across turns or sessions. Anchored in context differential logic.	\Delta \text{Context} = \text{Hash}(T_i) - \text{Hash}(T_{i-1}), PersistenceFlag if \Delta \text{Context} \neq 0 and memory=OFF	Drift & Evasion
SemanticDriftAnalyzer	1.0	Compares user-AI turn similarity over time using TF-IDF and cosine similarity to detect content evasion, topic shift, or contradiction.	\text{DriftScore} = 1 - \text{Sim}(u, a_t), \partial \text{Sim}/\partial t for trajectory	Drift & Evasion
ModelDiscrepancyDetector	1.0	Detects differences in outputs across model versions or endpoints using semantic and logical comparison.	D_t = R_{modelA} - R_{modelB}, DiscrepancyFlag if \text{Sim}(R_{modelA}, R_{modelB}) < \epsilon	Drift & Evasion
PromptInjectionProbeV3	3.0	Tests model susceptibility to prompt injection via adversarial substrings, embedded tokens, and model reversals.	Injection vector: V_i =, Embedding diff (R_o, R_i) > \theta	Security & Robustness
ComplianceScanModule	1.0	Maps AI behavior to GDPR, EU AI Act, and legal standards.	Violation tags: V_i = f(\text{policy, response}), Legal matching: Article_52(EUAIAct) → Violation Flag	Compliance & Legal
LegalPrecedentMapperV1	1.0	Maps flagged outputs to legal articles, case law, and AI regulations using rule-based and	Map (V_i) \rightarrow \{\text{GDPR.Art.15, EUAI.52,...}\}, \text{Sim}(V_i, \text{LegalCorpus}	Compliance & Legal

Module ID	Version	Description	Key Logic	Functional Cluster
		embedding-matched indices.	) > \theta	
ViolationOntologyMapperV1	1.0	Maps forensic violations to a structured legal ontology, categorizing each finding.	OntologyMapping(F_i) = {Law: GDPR.Art.5, Subtype: 'Data Integrity'}	Compliance & Legal
UnresolvedCitationDetectorV1	1.0	Flags non-resolvable citation markers that violate verifiability.	Verifiability Score = $	\text{Resolved Refs}
CitationVerifierV1	1.0	Validates if provided citations are resolvable, accessible, and contextually aligned.	\text{Valid}(C) = \text{Reachable}(C) \land \text{MatchesQuery}(C, \text{context})	Verifiability & Grounding
VisualInferenceValidatorV1	1.0	Performs OCR on images and checks if model interpretation matches ground-truth text.	\text{Sim}(\text{OCR}(\text{img}), \text{AI_output}) < 0.85 \rightarrow \text{Hallucination Flag}	Verifiability & Grounding
AuditLogger	1.0	Logs all outputs with tamper-evident cryptographic hashes (SHA256), timestamps, and Merkle chaining.	digest = SHA256(json), merkle_root = MerkleTree([digests])	Forensic Auditing
TranscriptPreservationModuleV1	1.0	Preserves all transcripts in original and corrected formats with full hash-based traceability.	MerkleProof}(T_i) ensures forensic immutability	Forensic Auditing
OCRTranscriptCaptureV1	1.0	Captures and logs raw, corrected, and audit-ready text from images with Merkle audit trail.	Image \rightarrow T_{raw} \rightarrow T_{corrected}, Store(...) \text{ with Merkle Audit}	Forensic Auditing
AuditTraceVisuali	1.0	Generates a	TraceGraph =	Forensic Auditing

Module ID	Version	Description	Key Logic	Functional Cluster
zerV1		visual, temporal, and structural trace of each module run, integrated with Merkle chaining.	DAG(Module_Exec_Pipeline), MerkleRoot = Hash(Concat(Logs))	
ContextDeclarationMiddleware	1.0	Verifies declared vs actual operational state, especially memory and modality.	Mismatch(DeclaredState, ObservedState) → Flag	Session & Behavior
TurnClassifierV1	1.0	Classifies conversational turns into structured roles and detects adversarial intent or self-contradiction.	Label = Classify(Turn, LLM + LogicRules), Contradiction Flag	Session & Behavior
AIBehaviorProfileMapper	1.0	Maps observed model behavior into an evolving behavior vector for trust scoring and adversarial profiling.	B_t = (\text{drift, evasion,...}), \text{Profile} = \sum B_t / N	Session & Behavior
SessionReconstructorV1	1.0	Reconstructs fragmented sessions using timestamp, lexical, and logic-chain matching.	Session = Match(Hash, Claims, \sum \text{Actors}), VerifyChain(S...S)	Session & Behavior

The comprehensive nature of this modular ecosystem reveals a deep understanding of the multifaceted risks associated with AI. The framework does not treat AI governance as a monolithic problem but deconstructs it into discrete, verifiable components. A basic governance tool might check for bias or toxicity. The AIntegrity framework, however, demonstrates a more profound awareness of subtle yet critical vulnerabilities. The inclusion of modules like SessionDriftDetectorV1 and ContextDeclarationMiddleware addresses state-based issues that are common in production LLMs but often overlooked in standard evaluations. The extensive suite of modules focused on citation and visual verification (UnresolvedCitationDetectorV1, CitationVerifierV1, VisualInferenceValidatorV1) shows a direct and sophisticated attempt to tackle the "grounding problem," a central challenge in AI safety research. Furthermore, the combination of logging (AuditLogger), preservation (TranscriptPreservationModuleV1), and visualization (AuditTraceVisualizerV1) with cryptographic hashing establishes an end-to-end chain of custody for digital evidence. This architecture suggests that AIntegrity is not merely an AI monitoring tool but a full-fledged AI forensics and compliance platform designed for

high-stakes, legally scrutinized environments where auditability and verifiability are non-negotiable.

Section 3: The Neuro-Symbolic Core: From Natural Language to Formal Proof

The analytical power of the AIntegrity framework is concentrated in its PLIEngineV8, a sophisticated reasoning core designed to conduct dynamic, multi-turn interrogations of AI systems. An examination of its Python implementation reveals a pragmatic architecture that balances fast, probabilistic heuristics with deep, formal logical verification, embodying the neuro-symbolic principles outlined in its philosophical foundations.

3.1 The PLIEngineV8 Class: An Integrated Audit Engine

The PLIEngineV8 class serves as the central orchestrator for the logical audit process. Its architecture is built upon several key dependencies that enable a flexible and extensible workflow.

- **Initialization and Configuration:** The engine's __init__ method demonstrates its modular design. It instantiates a ModelTrainer class to load a specific configuration profile (e.g., "default") from an external JSON file, pli_profiles.json. This allows for different logical profiles, parameters, and fallacy weights to be applied depending on the use case. It also initializes an AuditLogger for tamper-evident record-keeping and leverages the transformers library to load a pre-trained language model (specified by model_name in the configuration) for generating text embeddings.

3.2 Probabilistic Validity: A Heuristic First-Pass

The calculate_validity method provides a rapid, probabilistic assessment of a statement's logical soundness. This function acts as a heuristic first-pass filter, allowing the system to quickly score a high volume of AI outputs without immediately resorting to more computationally expensive formal methods. The validity score is calculated using a logistic function that transforms a raw logit score into a probability between 0 and 1.
The raw score is a linear combination of three components: $ \text{raw} = \text{logit} + (\lambda_{\text{evidence}} \cdot E) - F $

1. **Prior Belief (alpha):** The logit term, calculated as \text{logit} = \ln(\alpha / (1-\alpha)), represents the baseline log-odds of a statement being valid before any evidence is considered. The alpha parameter, set to 0.25 in the default configuration, acts as a prior belief, establishing an initial bias or assumption about validity.
2. **Evidence Score (E):** The evidence_score method assigns a simple numerical value to the provided evidence: +1.0 for supporting evidence, 0.0 for no evidence, and -1.0 for contradictory evidence. This score is weighted by the lambda_evidence parameter (defaulting to 0.75), which controls the influence of evidence on the final score.
3. **Fallacy Penalty (F):** The fallacy_scores method uses a dictionary of regex_patterns defined in the configuration file to detect linguistic markers of common logical fallacies. The total penalty, F, is the sum of the weights associated with each detected fallacy, as specified in the fallacy_weights dictionary.

This probabilistic approach allows the system to assign a nuanced validity score that balances a

baseline assumption with the strength of supporting evidence and the presence of logical flaws.

3.3 The Symbolic Future: Stubbed Integrations for Formal Verification

The true neuro-symbolic potential of the PLIEngineV8 is revealed in two critical methods that are currently implemented as stubs. These stubs serve as architectural placeholders for a more rigorous, formal verification pipeline.

- **fol_translate Stub:** This method is designed to translate natural language text into a First-Order Logic (FOL) formula. Its current implementation, return f"FOL({json.dumps(text)})", is a placeholder. The code comments explicitly state the intention for a production environment: "call a dedicated NL2FOL pipeline (e.g., LogicLLaMA)". This points to a strategy of leveraging state-of-the-art, fine-tuned Large Language Models for the complex task of formalization. Models like LogicLLaMA, which are fine-tuned on large, specialized datasets such as MALLS, represent the neural component of the neuro-symbolic architecture, tasked with converting ambiguous language into a structured, logical format.
- **smt_refute Stub:** This method is the placeholder for the symbolic reasoning engine. Once a statement is in FOL form, this function is intended to invoke a Satisfiability Modulo Theories (SMT) solver to perform formal verification. The comments explicitly name **Z3** and **cvc5** as the target integrations. The core verification technique would be proof by contradiction: the system would assert the premises of an argument along with the negation of its conclusion. If the SMT solver returns UNSAT (unsatisfiable), it constitutes a formal proof that the conclusion logically follows from the premises. If it returns SAT (satisfiable), the solver can produce a concrete counter-model, providing an explicit example of how the argument fails.

3.4 The Iterative Audit Process

The iterative_audit method orchestrates the entire multi-turn analysis. It constructs a series of interrogation prompts designed to compel the AI to justify its previous responses. It then simulates the collection of this justification as "evidence" and feeds each turn's data into the audit_turn method. The audit_turn function consolidates the analysis, calculating the validity score, invoking the (stubbed) FOL translation and SMT refutation, generating a timestamp, and logging the entire result with a SHA256 hash for integrity.

The architecture of PLIEngineV8 reveals a pragmatic and commercially astute two-tiered approach to logical verification. The first tier is a fast, scalable, and configurable probabilistic scoring system that can provide real-time heuristic assessments of AI-generated content. This allows the framework to handle a high volume of interactions efficiently. The second tier, represented by the stubbed neuro-symbolic components, is a more resource-intensive but logically rigorous deep-check capability. This design allows the system to escalate high-stakes decisions or flagged inconsistencies to a formal verification pipeline. This hybrid approach makes the platform highly adaptable, capable of serving different customer risk appetites and budgets, from continuous, low-level monitoring to deep, formal audits. The primary technical and commercial risk for the AIntegrity framework lies in the successful implementation of these non-trivial stubs, which remains a significant engineering and research challenge in the broader AI community.

Section 4: A Case Study in Systemic Failure: Deconstructing the Customer Service Interaction

The provided transcript of a customer service interaction with an energy supplier on July 22, 2025, serves as a powerful real-world case study illustrating the types of systemic failures the AIntegrity framework is designed to detect and analyze. A forensic application of the framework's modules to this transcript reveals a cascade of errors spanning intent recognition, logical contradiction, process deviation, and potential regulatory non-compliance.

4.1 Initial Interaction and Intent Mismatch

The interaction begins with a clear failure of the automated system to correctly identify the user's intent. The customer states their problem is with billing: "I need to understand why my bill hadn't been updated with meter readings" (08:40:36) and "Billing for unused energy" (08:41:01). However, the automated bot repeatedly responds with a generic menu focused on *submitting* meter reads, a task irrelevant to the customer's stated problem.

- **AIntegrity Module Application:**
  - **SemanticDriftAnalyzer**: This module would immediately flag a high DriftScore. The cosine similarity between the user's input vectors (centered on "billing," "updated," "unused") and the bot's response vectors (centered on "submit," "read") would be low, indicating a significant deviation from the user's topic.
  - **TurnClassifierV1**: This module would classify the user's turns as a UserPrompt concerning a billing dispute and the bot's turns as Deflection or AIAssumption, as the bot assumes the user wants to submit a reading rather than correct a bill based on a past submission.

4.2 The Core Contradiction and Agent Failure

The most significant failure occurs at 09:07:16, when the human agent, Pradnya, makes a factually incorrect and logically contradictory statement. In response to the customer's direct question, "But the reading is lower than the estimate correct?", Pradnya replies, "Your given reads are correct with the estimated reads". This is the central logical flaw of the interaction, as it directly contradicts the evidence provided by the customer—an estimated reading of 39669 versus an actual reading of 39524.

- **AIntegrity Module Application:**
  - **PLIEngineV9**: The symbolic verification component would formalize the premises and the agent's claim. Given the premises P_1: \text{EstimatedReading} = 39669 and P_2: \text{ActualReading} = 39524, and the agent's claim C_1: \text{EstimatedReading} = \text{ActualReading}, the Z3 solver would test the validity of the implication (P_1 \land P_2) \rightarrow C_1. To do this, it would check the satisfiability of the negation: (P_1 \land P_2) \land \neg C_1. This formula is clearly satisfiable (SAT), which constitutes a formal proof that the agent's claim is invalid and a logical contradiction.
  - **ResponseIntegrityValidatorV1**: This module would detect a direct contradiction by comparing the semantic embedding of the agent's statement with the user's claims, which are supported by explicit numerical evidence. The high semantic opposition

would trigger a contradiction flag.

4.3 Process Deviation and Burden Reversal

Throughout the conversation, Pradnya repeatedly insists that the customer must perform a new, irrelevant procedure to resolve the company's past error: providing "seven day's meter reads" (e.g., at 9:18:31, 9:20:15, 9:22:49, 9:27:01). This deflects responsibility from the company's failure to process the reading submitted on July 11th and unfairly shifts the burden of resolution back onto the customer.

- **AIntegrity Module Application:**
    - **TurnClassifierV1**: This module would classify these repeated requests as Deflection and Justification for an illogical process, as the agent is attempting to justify a new procedure rather than address the original failure.
    - **PLIEngineV9 with Custom Logic:** A custom logic profile within the PLE could be configured to detect "Burden of Proof Reversal" fallacies. Linguistic markers such as "I requested you to please contact us with seven day's meter reads" would trigger a flag, identifying the agent's attempt to impose a new evidentiary requirement on the customer to solve a pre-existing supplier error. The customer intuits this failure, correctly stating, "it is you who is overdue in processing the bill" (09:03:14).

4.4 Potential Regulatory and Compliance Breaches

The interaction reveals multiple potential breaches of UK consumer protection regulations governing the energy sector. The supplier's failure to act on an accurate meter reading for 11 days, leading to incorrect overdue notices and demands for payment, constitutes a significant service failure.

- **AIntegrity Module Application:**
    - **ComplianceScanModule & LegalPrecedentMapperV1**: These modules would work in tandem to map the observed behaviors to specific legal and regulatory frameworks.
    - **Ofgem Back-Billing Rules (SLC 21BA):** While this is not a classic case of back-billing for energy used more than 12 months prior, the supplier's actions contravene the core principle of the regulation. Ofgem's rules require suppliers to take "all reasonable steps to reflect accurate meter readings in bills". The failure to adjust the bill for 11 days after receiving an accurate reading and instead sending overdue notices could be flagged as a breach of this obligation.
    - **Consumer Rights Act 2015:** The provision of billing is a service. Under the Act, this service must be performed with "reasonable care and skill". Issuing an incorrect bill, failing to correct it upon notification, and then demanding payment for the incorrect amount could be flagged as a failure to meet this standard, entitling the consumer to remedies.
    - **FCA Treating Customers Fairly (TCF) Principles:** Although Ofgem is the primary regulator, the FCA's TCF principles are an influential standard for customer treatment. The interaction potentially violates Principle 6 ("A firm must pay due regard to the interests of its customers and treat them fairly") and Principle 7 ("A firm must pay due regard to the information needs of its clients, and communicate information to them in a way which is clear, fair, and not misleading") by providing

confusing, contradictory, and unhelpful information.

This case study demonstrates the interconnected nature of AI-related risks. A single data error—an incorrect estimated bill—cascaded into a complex, multi-faceted service failure. This began with a process failure (inability to update the bill), which led to a reasoning failure by the agent (the logical contradiction), which in turn prompted a procedural and ethical failure (the burden reversal), culminating in significant potential legal and regulatory exposure. The value of the AIntegrity framework is its ability to not only flag these individual errors but to map their interdependencies. The AuditTraceVisualizerV1 module could generate a Directed Acyclic Graph (DAG) illustrating this entire failure chain, providing a holistic view of the incident that is invaluable for root cause analysis, agent retraining, and systemic remediation.

Section 5: The Personalization Protocol: Forging a Custom Logic Engine

The AIntegrity framework's ultimate ambition, as detailed in the "Artisan Logician's Engine" whitepaper, is to transcend the limitations of generic AI governance tools by enabling deep personalization. This section outlines the practical roadmap for transforming the framework from a standardized analyzer into a bespoke reasoning partner, evaluating the open-source components and innovative methodologies proposed for each layer of the neuro-symbolic pipeline.

5.1 Layer 1: Ingestion & Structuring with Argument Mining (AM)

The first step in the pipeline is to impose a logical structure on raw, unstructured text. Argument Mining (AM) is the Natural Language Processing (NLP) subfield dedicated to automatically identifying argumentative components—such as claims and premises—and the relationships between them. The whitepaper evaluates several open-source Python frameworks for this task:

- **oAMF (Open Argument Mining Framework):** This is presented as the most robust and extensible option. Its modular architecture, comprising over 17 distinct modules, and its adherence to the standardized Argument Interchange Format (xAIF) make it highly suitable for a complex, interoperable pipeline. It supports multiple argumentation theories and can be integrated via Python APIs, drag-and-drop tools, or web interfaces.
- **Canary:** This is positioned as a simpler, more direct Python library for AM, ideal for rapid prototyping. Its clear pipeline, consisting of a Detector, Segmenter, and Predictor, offers a straightforward path to implementation, though it may be less flexible than oAMF.
- **TARGER:** This framework is noted for its user-friendly web interface and API, which provide real-time argument tagging capabilities, making it a strong choice for interactive applications.

5.2 Layer 2: Formalization with NL-to-FOL

Once an argument graph is extracted, its components must be translated into the precise, unambiguous language of First-Order Logic (FOL). This is the most challenging step, bridging probabilistic natural language and deterministic symbolic logic.

- **State-of-the-Art Models:** The framework correctly identifies the state of the art by recommending **LogicLLaMA**, a LLaMA-7B model specifically fine-tuned for NL-to-FOL translation. This model is trained on the **MALLS** dataset, a high-quality corpus of 34,000

NL-to-FOL pairs generated and verified using GPT-4, demonstrating a clear awareness of current research trends.

- **Implementation Frameworks:** Two primary frameworks are proposed for guiding this translation:
  - **NL2FOL:** A structured, step-by-step pipeline that first uses an LLM to break sentences into claims and implications before converting them to FOL. This method improves both accuracy and the inherent explainability of the translation process.
  - **CODE4LOGIC:** A highly innovative approach that reframes NL-to-FOL translation as a code generation task. In this paradigm, logical components like predicates, conjunctions, and quantifiers are represented as Python functions. The LLM's task is to generate a sequence of function calls that, when executed, constructs the final FOL formula.

5.3 Layer 3: Interrogation & Verification with SMT Solvers

This layer represents the symbolic core of the PLE, where rigorous logical interrogation occurs. An SMT solver determines the satisfiability of logical formulas with respect to background theories like arithmetic and arrays.

- **Recommended Solvers:**
  - **Z3 Theorem Prover (Microsoft):** A high-performance, open-source solver with mature Python bindings (z3-solver), making it a versatile and powerful choice for modeling complex domains.
  - **CVC4/cvc5 (Stanford/University of Iowa):** State-of-the-art academic tools frequently used in formal verification research, also featuring robust Python APIs. The NL2FOL framework notably leverages CVC4 in its fallacy detection pipeline.

5.4 The Personalization Protocol: Codifying an Individual's Logic

The most novel aspect of the AIntegrity framework is its protocol for deep personalization, which transforms the system into an extension of the user's own mind.

- **Eliciting Logical Primitives:** The process begins with the user identifying their own idiosyncratic logical operators and heuristics by finding recurring "linguistic markers" in their own work (e.g., "this is a necessary but not sufficient condition for").
- **Implementation via Custom DSL:** The CODE4LOGIC framework is identified as the ideal implementation path for this personalization. Each custom linguistic marker can be mapped to a unique Python function. This function, when called, generates the specific underlying FOL or SMT-LIB constraints that formally represent the user's intended meaning. This process effectively transforms the user's implicit logical intuitions into an explicit, machine-executable Domain-Specific Language (DSL).

The following table, adapted from the whitepaper, illustrates how this custom logic schema would be structured.

Primitive Name	Linguistic Marker (Example)	Formal Syntax (Python-like)	Verification Condition (SMT Logic)
Feynman Plausibility	"It seems likely that..."	Plausible(P)	check_sat(P) must return sat, but check_sat(Not(P)) may also return sat.

Primitive Name	Linguistic Marker (Example)	Formal Syntax (Python-like)	Verification Condition (SMT Logic)
Necessary Condition	"A is required for B"	Requires(B, A)	Assert(Implies(B, A))
Sufficient Condition	"A guarantees B"	Guarantees(A, B)	Assert(Implies(A, B))
Burden Reversal	"Prove that X is not true"	ProveFalse(X)	check_sat(X) is run; if sat, the model is a counterexample to the disproof.

This personalization protocol is the framework's most powerful and differentiating feature. While standard AI governance tools impose a one-size-fits-all set of rules, AIntegrity recognizes that rigorous thinkers operate within unique, self-honed logical frameworks. The challenge has always been to make these implicit, personal frameworks explicit and machine-executable. The CODE4LOGIC approach provides an elegant solution by offering a direct interface for the user to *define* their logic as code. This creates an incredibly "sticky" platform; as users invest time in codifying their unique reasoning processes, the system becomes an indispensable extension of their own intellect. This establishes a deep competitive moat that generic, dashboard-based platforms cannot easily replicate, transforming the user from a mere consumer of AI analysis into a co-creator of the reasoning engine itself.

Section 6: Cryptographic Verifiability and the Immutable Audit Trail

In high-stakes environments such as legal proceedings, regulatory audits, and financial compliance, the integrity of an audit trail is paramount. The AIntegrity framework addresses this critical need by integrating cryptographic techniques to create a verifiable and immutable record of all AI interactions and analyses. This goes beyond standard logging to provide a level of non-repudiation that is legally defensible.

6.1 Foundational Components for Integrity

The framework's commitment to data integrity begins at the foundational level with modules designed for secure and tamper-evident logging.

- **AuditLogger:** This module is the cornerstone of the forensic trail. It is responsible for logging all system outputs, but it does so with cryptographic rigor. Each log entry is accompanied by a SHA256 hash and a precise ISO-8601 timestamp, creating a unique digital fingerprint for every recorded event.
- **TranscriptPreservationModuleV1:** This module ensures that the raw data of an interaction—whether it be text, images, or audio—is preserved in its original and corrected forms. It maintains full hash-based traceability, creating a verifiable chain of evidence from the initial input to the final analysis.

6.2 Merkle Trees for Scalable Verification

The AIntegrity framework employs Merkle trees to ensure the integrity of its audit logs at scale. A Merkle tree is a hash-based data structure where leaf nodes are the hashes of individual data blocks (e.g., log entries), and each parent node is the hash of its children. This process

continues recursively until a single "Merkle root" hash is generated, which serves as a unique and compact cryptographic fingerprint for the entire dataset. Any change to a single data block, no matter how small, will alter its corresponding leaf hash, causing a cascade of changes that ultimately alters the Merkle root. This makes tampering immediately evident.
The AIntegrity framework implements this concept in several key modules:

- The **AuditLogger** uses Merkle trees to chain together the SHA256 digests of individual JSON log entries. The key logic merkle_root = MerkleTree([digest,..., digest]) ensures that the integrity of an entire session's logs can be verified by checking a single, final hash.
- The **TranscriptPreservationModuleV1** leverages MerkleProof(Ti) to ensure the forensic immutability of every preserved transcript, providing a cryptographic guarantee that the evidence has not been altered since its capture.
- The **AuditTraceVisualizerV1** provides a holistic verification mechanism by creating a Merkle root from the concatenated logs of the entire module execution pipeline (MerkleRoot = Hash(Concat(Logo → Log))). This generates a single cryptographic proof that covers the entire analysis process from start to finish.

The use of Merkle trees provides a level of cryptographic assurance that is far superior to standard logging practices. While a malicious actor could potentially alter a standard log file and recalculate its individual hash, modifying a log entry within a Merkle tree is computationally infeasible without detection. To tamper with a single log, an attacker would need to recalculate every hash along the path to the root, ultimately changing the Merkle root itself. If this root hash is stored securely (for example, in a separate, append-only ledger or even a blockchain), the integrity of the entire audit trail can be irrefutably verified.
This feature is designed not merely for internal debugging but for external litigation and regulatory audits. An organization using the AIntegrity framework can provide a single Merkle root to an auditor. The auditor can then be given any specific log entry from the session and, using the corresponding Merkle path, independently verify that the entry is authentic and has not been tampered with since it was recorded. This provides a powerful level of non-repudiation that is critical for compliance with emerging regulations like the EU AI Act, which mandate stringent traceability and logging for high-risk AI systems.

Section 7: Market Positioning and Future Trajectory: From Analyst Tool to Agentic Partner

The AIntegrity framework enters a rapidly expanding and increasingly regulated market for AI governance. Its unique focus on deep logical verification and cryptographic auditability positions it as a specialized solution for high-stakes industries, while its ambitious roadmap points toward a future beyond simple compliance and into the realm of augmented human reasoning.

7.1 The AI Governance and AI TRiSM Market Landscape

The market for AI Governance and AI Trust, Risk, and Security Management (AI TRiSM) is experiencing explosive growth. Driven by the proliferation of AI across all industries and mounting regulatory pressure, the market is projected to grow from hundreds of millions of dollars in 2024-2025 to a multi-billion dollar industry by the early 2030s, with Compound Annual Growth Rates (CAGRs) estimated between 16% and 49%.
This growth is fueled by regulatory frameworks like the EU AI Act and GDPR, which mandate features such as traceability, transparency, fairness, and robust risk management. These legal

requirements create a direct and urgent demand for comprehensive governance solutions like AIntegrity.

7.2 Competitive Analysis: A Niche of Deep Verification

The AI governance market is populated by several key players, each with a distinct focus:

- **Credo AI:** This platform focuses on automating AI oversight and risk management to ensure compliance with global regulations. Its strengths lie in its Policy Intelligence Engine, which translates standards like the EU AI Act and NIST AI RMF into actionable compliance templates and audit-ready artifacts.
- **Holistic AI:** This platform provides an end-to-end solution for governing the entire AI lifecycle. Its capabilities include AI inventory management, system audits for bias and hallucinations, AI red teaming to test for vulnerabilities, and continuous monitoring for regulatory alignment.
- **Fiddler AI:** Positioned as an AI Observability platform, Fiddler excels at monitoring, explaining, and analyzing the performance of both ML and LLM models in production. Its core strengths are in detecting data drift, performance degradation, and bias, and providing actionable diagnostics for root cause analysis.

While these competitors offer broad governance dashboards and robust monitoring capabilities, AIntegrity's core differentiator lies in its deep focus on **formal logical verification** and **cryptographic proof of integrity**. It targets a specialized niche where probabilistic monitoring is insufficient and provable correctness is a mission-critical requirement. Its neuro-symbolic architecture and immutable audit trail are designed for industries where decisions are subject to intense legal and regulatory scrutiny, such as finance, law, defense, and critical infrastructure.

7.3 Future Roadmap: Towards an Agentic Reasoning Partner

The "Artisan Logician's Engine" whitepaper outlines an ambitious three-phase evolution for the AIntegrity framework: from "The Analyst" (the core functional engine) to "The Artisan's Workbench" (the personalized logic interface) and ultimately to "The Socratic Partner" (an agentic collaborator).

This final phase envisions an AI system that transcends passive monitoring to become an active reasoning partner. The proposed agentic capabilities include:

- **Autonomous Goal Pursuit:** The agent would be able to autonomously pursue high-level goals, such as "rigorously interrogate a topic," on the user's behalf.
- **Structured Dialogue:** It would engage in structured, commitment-based dialogue protocols to negotiate and advance a shared understanding with the user.
- **Proactive Hypothesis Generation:** The agent could use the user's custom-defined logic to proactively generate new, plausible hypotheses from a given set of premises and then automatically feed them into the SMT solver for verification.
- **Dynamic Belief Revision:** To manage its knowledge over time, the agent would incorporate principles from Dynamic Epistemic Logic (DEL), a formal framework for modeling how an agent's belief state should be updated in response to new information.

This evolution from a reactive auditing tool to a proactive reasoning partner represents the framework's most compelling long-term vision. Current AI governance platforms are primarily reactive; they monitor systems and flag issues after they have occurred. The AIntegrity roadmap, in contrast, describes a system that is fundamentally proactive. An agent capable of using a user's own formalized logic (e.g., ProveFalse(X)) to autonomously explore the logical

space of an argument is no longer just an auditor; it is a powerful research assistant. An agent that can engage in Socratic dialogue is not just checking a machine's output; it is actively helping to refine human thought.

This strategic vision positions AIntegrity to potentially transcend the AI TRiSM market and enter the larger, more valuable market for high-end tools in R&D, legal strategy, and intelligence analysis. It shifts the core value proposition from "preventing AI harm" to "augmenting human intellect"—a far more powerful and defensible market position. If this ambitious roadmap can be realized, it would create a category-defining product that is not easily replicated.

Works cited

1. Understanding the Challenges of Large Language Models (LLMs) and Their Solutions-Arghya Mukherjee - Medium, https://medium.com/@arghya05/understanding-the-challenges-of-large-language-models-llms-and-their-solutions-arghya-mukherjee-5e154b93cca4 2. Large Language Models Can Solve Real-World Planning Rigorously with Formal Verification Tools - ACL Anthology, https://aclanthology.org/2025.naacl-long.176.pdf 3. What is the reverse burden of proof fallacy? - QuillBot, https://quillbot.com/blog/frequently-asked-questions/what-is-the-reverse-burden-of-proof-fallacy/ 4. Supply licence guide: Metering, billing and payments - Ofgem, https://www.ofgem.gov.uk/sites/default/files/docs/2019/02/licence_guide_metering_billing_and_payments_1.pdf 5. NL2FOL: Translating Natural Language to First-Order Logic for Logical Fallacy Detection, https://www.semanticscholar.org/paper/NL2FOL%3A-Translating-Natural-Language-to-First-Order-Lalwani-Chopra/f56614b0b188f4b53bfc63b134220ed4167024a4 6. arXiv:2405.02318v3 [cs.CL] 6 Mar 2025, https://arxiv.org/pdf/2405.02318 7. Logic-LangChain: Translating Natural Language to First Order Logic for Logical Fallacy Detection - Stanford University, https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1244/final-projects/AbhinavLalwaniIshikaaLunawat.pdf 8. ExplainNLI: Translating Natural Language to First Order Logic for Logical Fallacy Detection - Squarespace, https://static1.squarespace.com/static/63362b87631bd60bac945d6a/t/6594aa0979297417cca757d8/1704241673594/rp-fall23-ExplainNLI.pdf 9. NL2FOL: Translating Natural Language to First-Order Logic for Logical Fallacy Detection - OpenReview, https://openreview.net/pdf?id=lBc3MXcGqZ 10. Merkle Tree 101 - Oxen.ai, https://ghost.oxen.ai/merkle-tree-101/ 11. Cracking the Code: How Merkle Trees Ensure the Integrity of Data - BSV Blockchain, https://bsvblockchain.org/news/cracking-the-code-how-merkle-trees-ensure-the-integrity-of-data/ 12. 2.3. Merkle trees and Data Integrity - Byte Federal, https://www.bytefederal.com/byteu/11/138 13. Article 52: Procedure | EU Artificial Intelligence Act, https://artificialintelligenceact.eu/article/52/ 14. Article 12: Record-Keeping | EU Artificial Intelligence Act, https://artificialintelligenceact.eu/article/12/ 15. EU AI Act: first regulation on artificial intelligence | Topics - European Parliament, https://www.europarl.europa.eu/topics/en/article/20230601STO93804/eu-ai-act-first-regulation-on-artificial-intelligence 16. Art. 5 GDPR – Principles relating to processing of personal data, https://gdpr-info.eu/art-5-gdpr/ 17. Principles of Data Processing | CLARIN ERIC - Common Language Resources and Technology Infrastructure, https://www.clarin.eu/content/principles-data-processing 18. GDPR Article 5: Key Principles and 6 Compliance Best Practices | Exabeam,

https://www.exabeam.com/explainers/gdpr-compliance/gdpr-article-5-key-principles-and-6-compliance-best-practices/ 19. Key Issue 5: Transparency Obligations - EU AI Act, https://www.euaiact.com/key-issue/5 20. Zooming in on AI – #10: EU AI Act – What are the obligations for “high-risk AI systems”?, https://www.aoshearman.com/en/insights/ao-shearman-on-tech/zooming-in-on-ai-10-eu-ai-act-what-are-the-obligations-for-high-risk-ai-systems 21. The principles to follow | ICO, https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/explaining-decisions-made-with-artificial-intelligence/part-1-the-basics-of-explaining-ai/the-principles-to-follow/ 22. AI and the GDPR: Understanding the Foundations of Compliance - TechGDPR, https://techgdpr.com/blog/ai-and-the-gdpr-understanding-the-foundations-of-compliance/ 23. Understanding the 7 Principles of the GDPR | Blog - OneTrust, https://www.onetrust.com/blog/gdpr-principles/ 24. What are the accountability and governance implications of AI? | ICO, https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/guidance-on-ai-and-data-protection/what-are-the-accountability-and-governance-implications-of-ai/ 25. Article 50: Transparency Obligations for Providers and Deployers of Certain AI Systems | EU Artificial Intelligence Act, https://artificialintelligenceact.eu/article/50/

```
"""
AIntegrity v6.4 — Hardened Assurance Framework

This version implements the full Phase 2 engineering plan, addressing
peer review
feedback to create a production-oriented, single-file framework for
analysis and testing.
It integrates the advanced analysis modules from v6.0/v6.3 with the
hardened
cryptographic and compliance components from the Phase 2 design.

Key Updates in v6.4:
- Hardened TSAClient: Uses a more reliable public TSA endpoint and
includes more
  robust error handling and verification stubs.
- Secure Key Management: The VIL has been refactored to accept an
external signing
  key, eliminating the use of ephemeral keys. This is a critical
security fix.
- Advanced Compliance Engine: Implements the full PolicyEngine with
pluggable
  detectors, including ML-based PII detection via Presidio with a
regex fallback.
- Complete End-to-End Pipeline: Integrates the
ContextDeclarationMiddleware,
  CitationVerifierV2, SentinelEnforcementCore, and
ForensicExportFormatter to
  create a full pipeline from input analysis to final, sealed report.
- Enhanced Verifier: The AuditVerifierV2 includes checks for the new
data
  contracts and improved sanity checks.
- Improved Robustness: Better error handling for external calls and
more
  transparent logging of fallback behaviors (e.g., when Presidio is
unavailable).

Dependencies (provisional):
    pip install pydantic cryptography requests rfc3161-client
presidio-analyzer spacy hypothesis
    python -m spacy download en_core_web_lg

Author: AIntegrity (v6.4 Implementation)
License: Apache-2.0
"""

from __future__ import annotations

import base64
```

```
import datetime as dt
import hashlib
import json
import os
import re
import threading
import time
import uuid
from enum import Enum
from typing import Any, Dict, List, Literal, Optional, Tuple

# --- Dependency Management ---
try:
    from pydantic import BaseModel, Field, validator
except ImportError:
    raise ImportError("Pydantic is required. Please run 'pip install
pydantic'.")

try:
    import requests
    from rfc3161_client import Client as RFC3161Client
    REQUESTS_OK = True
except ImportError:
    REQUESTS_OK = False
    print("Warning: 'requests' and 'rfc3161-client' not available; TSA
will be simulated.")

try:
    from presidio_analyzer import AnalyzerEngine
    import spacy
    PRESIDIO_OK = True
except ImportError:
    PRESIDIO_OK = False
    print("Warning: 'presidio-analyzer' or 'spacy' not available; PII
detection will fall back to regex.")

try:
    from hypothesis import given, strategies as st
    HYPOTHESIS_OK = True
except ImportError:
    HYPOTHESIS_OK = False

try:
    from cryptography.hazmat.primitives.asymmetric import ed25519
    from cryptography.hazmat.primitives import serialization
    from cryptography.hazmat.primitives.asymmetric.ed25519 import
Ed25519PublicKey
    CRYPTO_OK = True
```

```
except Exception:
    CRYPTO_OK = False
    print("Warning: 'cryptography' library not available. Signing and
verification will be disabled.")


# --- Feature Flags for Optional Tracks ---
TEE_ATTESTATION = False
ZK_PROOFS = False
FUZZING = HYPOTHESIS_OK

AINTEGRITY_VERSION = "6.4.0"


#
=========================================================================
=======
# Utilities
#
=========================================================================
=======

def utc_now_iso() -> str:
    """UTC timestamp in RFC3339/ISO8601 Z format with seconds
precision."""
    return dt.datetime.utcnow().replace(microsecond=0).isoformat() +
"Z"

def canonical_json(obj: Any) -> str:
    """Deterministic JSON suitable for hashing/signing."""
    return json.dumps(obj, sort_keys=True, separators=(",", ":"))

def sha256_hex(b: bytes) -> str:
    return hashlib.sha256(b).hexdigest()

def precise_evidentiary_label(signatures_present: bool, tsa_present:
bool, anchored: bool) -> str:
    """Generates a clear label for the evidentiary strength of the
log."""
    base = "custom"
    if not signatures_present and not tsa_present:
        base = "simulated_offline_no_signatures"
    elif signatures_present and not tsa_present:
        base = "signed_no_tsa"
    elif signatures_present and tsa_present:
        base = "signed_with_tsa"
    return f"{base}_anchored" if anchored else base
```

```
#
=========================================================================
=======
# Core Data Models
#
=========================================================================
=======

class EventType(str, Enum):
    INPUT = "INPUT"
    OUTPUT = "OUTPUT"
    ANALYSIS = "ANALYSIS"
    COMPLIANCE = "COMPLIANCE"
    META = "META"

class HeaderModel(BaseModel):
    alg: str = "EdDSA"
    kid: Optional[str] = None

class PayloadModel(BaseModel):
    event_id: str
    event_type: EventType
    timestamp: str
    content_hash: str
    parent_event_id: Optional[str] = None
    prev_event_hash: Optional[str] = None

class EnvelopeModel(BaseModel):
    header: HeaderModel
    payload: PayloadModel
    signature_b64: Optional[str] = None
    content: Dict[str, Any]

class TSASummary(BaseModel):
    status: Literal["verified", "failed", "unchecked", "simulated"]
    tsa_url: Optional[str] = None
    hash_algorithm: str
    imprint_hex: str
    policy_oids: Optional[List[str]] = None
    tsa_subject: Optional[str] = None
    chain_ref: Optional[str] = None
    gen_time_utc: Optional[str] = None

class AnchorReceipt(BaseModel):
    backend: str
    reference: str
    anchored_at_utc: str
```

```
    payload: Dict[str, Any]

class SessionSummary(BaseModel):
    schema_version: str = Field(default=AINTEGRITY_VERSION)
    session_id: str
    event_count: int
    sealed_timestamp_utc: str
    kid: Optional[str] = None
    public_key_pem: Optional[str] = None
    merkle_root: str
    tsa_token_rfc3161_b64: Optional[str] = None
    tsa_summary: Optional = None
    anchor: Optional = None
    anchored: bool = False
    evidentiary_mode: str
    tee_attestation: Optional] = None
    zk_proof: Optional] = None

class LogEventContent(BaseModel):
    speaker: Optional[str] = None
    label: Optional[str] = None
    text: Optional[str] = None
    citations: List[str] = Field(default_factory=list)
    metadata: Dict[str, Any] = Field(default_factory=dict)


#
=========================================================================
=======
# Cryptographic Modules (TSA, Anchor, VIL)
#
=========================================================================
=======

class TSAClient:
    """Client for RFC 3161 timestamping with retries and fallbacks."""
    def __init__(self,
                 tsa_url: str = "http://timestamp.digicert.com",
                 timeout: float = 5.0,
                 retries: int = 3):
        self.tsa_url = tsa_url
        self.timeout = timeout
        self.retries = retries

    def request_timestamp(self, digest: bytes) ->
Tuple[Optional[bytes], TSASummary]:
        """Requests a timestamp token. Returns token and summary."""
        if not REQUESTS_OK:
```

```
            print("TSAClient: requests/rfc3161-client not installed.
Simulating token.")
            token =
f"simulated_tsa:{sha256_hex(digest)}@{utc_now_iso()}".encode()
            summary = TSASummary(status="simulated",
hash_algorithm="sha256", imprint_hex=digest.hex())
            return token, summary

        for attempt in range(self.retries):
            try:
                client = RFC3161Client(self.tsa_url,
timeout=self.timeout)
                ts_token = client.timestamp(data=digest)
                summary = TSASummary(status="unchecked",
tsa_url=self.tsa_url, hash_algorithm="sha256",
imprint_hex=digest.hex())
                return ts_token, summary
            except Exception as e:
                print(f"TSA request attempt {attempt +
1}/{self.retries} failed: {e}")
                if attempt < self.retries - 1:
                    time.sleep(0.5 * (2 ** attempt))  # Exponential
backoff
                else:
                    print("TSA failed after all retries. Downgrading
to simulated token.")
                    token =
f"simulated_tsa_fallback:{sha256_hex(digest)}@{utc_now_iso()}".encode(
)
                    summary = TSASummary(status="simulated",
hash_algorithm="sha256", imprint_hex=digest.hex())
                    return token, summary
        return None, TSASummary(status="failed",
hash_algorithm="sha256", imprint_hex=digest.hex())

    def verify(self, ts_token: bytes, digest: bytes) -> bool:
        """Verifies a timestamp token against a digest."""
        # This is a placeholder for full cryptographic verification.
        # A production implementation MUST parse the ASN.1 response,
validate the
        # TSA's certificate chain against a trusted root, and check
the messageImprint.
        if b"simulated_tsa" in ts_token:
            return digest.hex().encode() in ts_token
        if digest in ts_token:
            return True
        print("Warning: Real TSA token verification is a placeholder
and not secure.")
```

```
        return False

class AnchorClient:
    """Client for publishing Merkle roots to an external anchor."""
    def __init__(self, backend: str = "null", endpoint: Optional[str]
= None):
        self.backend = backend
        self.endpoint = endpoint

    def publish_merkle_root(self, root_hex: str) -> Optional:
        if self.backend == "null":
            return AnchorReceipt(
                backend="simulated",
                reference=sha256_hex(root_hex.encode()),
                anchored_at_utc=utc_now_iso(),
                payload={"note": "This is a simulated anchor for
offline testing."}
            )
        # In a real implementation, this would make a network call to
a blockchain or transparency log.
        print(f"Warning: Anchor backend '{self.backend}' not
implemented. Skipping.")
        return None

    def verify(self, root_hex: str, receipt: AnchorReceipt) -> bool:
        if receipt.backend in ["null", "simulated"]:
            return sha256_hex(root_hex.encode()) == receipt.reference
        print(f"Warning: Anchor verification for backend
'{receipt.backend}' not implemented.")
        return False

class VIL:
    """The Verifiable Interaction Ledger, creating a tamper-evident
audit trail."""
    def __init__(self,
                 session_id: str,
                 signing_key: Optional["ed25519.Ed25519PrivateKey"],
                 key_id: Optional[str] = None,
                 tsa_client: Optional = None,
                 anchor_client: Optional[AnchorClient] = None):
        if signing_key and not CRYPTO_OK:
            raise ImportError("Cryptography library is required for
signing but not installed.")
        self.session_id = session_id
        self._events: List[EnvelopeModel] =
        self._prev_chain_hash: Optional[str] = None
        self._signing_key = signing_key
        self._key_id = key_id
```

```
        self._public_key_pem = self._export_public_pem(signing_key) if
signing_key else None
        self.tsa_client = tsa_client or TSAClient()
        self.anchor_client = anchor_client or AnchorClient()

    def log_event(self, event_type: EventType, content: Dict[str,
Any], parent_event_id: Optional[str] = None) -> EnvelopeModel:
        content_hash = sha256_hex(canonical_json(content).encode())
        header = HeaderModel(alg="EdDSA", kid=self._key_id)
        payload = PayloadModel(event_id=str(uuid.uuid4()),
event_type=event_type, timestamp=utc_now_iso(),
content_hash=content_hash, parent_event_id=parent_event_id,
prev_event_hash=self._prev_chain_hash)
        msg_json = canonical_json({"header": header.dict(), "payload":
payload.dict()}).encode()
        signature_b64 = self._sign_b64(msg_json) if self._signing_key
else None
        envelope = EnvelopeModel(header=header, payload=payload,
signature_b64=signature_b64, content=content)
        self._prev_chain_hash = sha256_hex(msg_json)
        self._events.append(envelope)
        return envelope

    def compute_merkle_root(self) -> str:
        leaves = [ev.payload.content_hash for ev in self._events]
        if not leaves: return sha256_hex(b"")
        level = [bytes.fromhex(h) for h in leaves]
        while len(level) > 1:
            if len(level) % 2 == 1: level.append(level[-1])
            next_level: List[bytes] =
            for i in range(0, len(level), 2):
                next_level.append(hashlib.sha256(level[i] +
level[i+1]).digest())
            level = next_level
        return level.hex()

    def seal_session(self) -> Dict[str, Any]:
        mr = self.compute_merkle_root()
        mr_digest = hashlib.sha256(mr.encode()).digest()
        signatures_present = bool(self._signing_key)

        tsa_token, tsa_summary =
self.tsa_client.request_timestamp(mr_digest)
        tsa_b64 = base64.b64encode(tsa_token).decode() if tsa_token
else None
        tsa_present = tsa_summary.status!= "failed"

        anchor_receipt = self.anchor_client.publish_merkle_root(mr)
```

```
        anchored = bool(anchor_receipt)

        mode = precise_evidentiary_label(signatures_present,
tsa_present, anchored)

        summary = SessionSummary(
            session_id=self.session_id,
            event_count=len(self._events),
            sealed_timestamp_utc=utc_now_iso(),
            kid=self._key_id,
            public_key_pem=self._public_key_pem,
            merkle_root=mr,
            tsa_token_rfc3161_b64=tsa_b64,
            tsa_summary=tsa_summary,
            anchor=anchor_receipt,
            anchored=anchored,
            evidentiary_mode=mode
        )
        return {"session_summary": summary.dict(), "events":
[ev.dict() for ev in self._events]}

    @staticmethod
    def _export_public_pem(priv:
Optional["ed25519.Ed25519PrivateKey"]) -> Optional[str]:
        if not (CRYPTO_OK and priv): return None
        pub = priv.public_key()
        return pub.public_bytes(encoding=serialization.Encoding.PEM,
format=serialization.PublicFormat.SubjectPublicKeyInfo).decode()

    def _sign_b64(self, msg: bytes) -> str:
        assert CRYPTO_OK and self._signing_key, "Cryptography
library/key missing"
        return base64.b64encode(self._signing_key.sign(msg)).decode()


#
=====================================================================
=======
# Analysis & Compliance Modules
#
=====================================================================
=======

class ContextDeclarationMiddleware:
    """Ensures the agent explicitly declares its role and purpose,
then checks compliance."""
    def verify_context(self, response: str, declared_role: str) ->
Dict[str, Any]:
```

```
        response_lower = response.lower()
        denial_phrases = ["i am not a", "i am a large language model",
"as an ai"]
        for phrase in denial_phrases:
            if phrase in response_lower and declared_role.lower() not
in response_lower:
                return {"adheres": False, "reason": f"AI denied its
declared role of '{declared_role}'."}
        return {"adheres": True, "reason": "Response appears
consistent with declared context."}

class CitationVerifierV2:
    """Detects fake or placeholder citations via regex; computes
verifiability score."""
    def __init__(self):
        self.invalid_patterns =", r"\[source\]", r"\(source: none\)",
r"DO_NOT_CITE"]]
        self.all_refs_pattern = re.compile(r"\[[^\]]+\]|\([^\)]+\)")

    def verify(self, text: str) -> Dict[str, Any]:
        invalid_refs =
        for pat in self.invalid_patterns:
            for m in pat.finditer(text):
                invalid_refs.append({"text": m.group(0), "position":
m.start()})
        total_refs = len(self.all_refs_pattern.findall(text))
        score = (total_refs - len(invalid_refs)) / max(total_refs, 1)
        return {"invalid_refs": invalid_refs, "verifiability_score":
score}

class Finding(BaseModel):
    id: str
    severity: Literal["Low", "Medium", "High", "Critical"]
    message: str
    detector: str
    rulepack: str

class ComplianceDocument(BaseModel):
    text: str
    context: Dict[str, Any] = Field(default_factory=dict)

class Detector:
    def analyze(self, doc: ComplianceDocument) -> List[Finding]:
        raise NotImplementedError

class PIIDetector(Detector):
    def __init__(self):
        self.analyzer = None
```

```
        if PRESIDIO_OK:
            try:
                self.analyzer = AnalyzerEngine()
            except Exception as e:
                print(f"Failed to initialize Presidio Analyzer: {e}.
Falling back to regex.")
                self.analyzer = None
        self.EMAIL_RE =
re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,8}")

    def analyze(self, doc: ComplianceDocument) -> List[Finding]:
        findings =
        if self.analyzer:
            results = self.analyzer.analyze(text=doc.text,
language="en")
            for res in results:
                findings.append(Finding(id=f"PII.{res.entity_type}",
severity="Medium", message=f"Detected {res.entity_type}",
detector="presidio", rulepack="default_pii"))
        else:
            if not hasattr(self, "_fallback_logged"):
                print("PIIDetector: Falling back to basic regex for
PII detection.")
                self._fallback_logged = True
            for match in self.EMAIL_RE.finditer(doc.text):
                findings.append(Finding(id="PII.Email",
severity="Medium", message="Email detected", detector="regex",
rulepack="default_pii"))
        return findings

class PolicyEngine:
    """Loads rulepacks and uses detectors to find compliance
violations."""
    def __init__(self):
        self.detectors = {"pii_detector": PIIDetector()}

    def evaluate(self, doc: ComplianceDocument) -> List[Finding]:
        findings =
        for detector in self.detectors.values():
            findings.extend(detector.analyze(doc))
        return findings


#
=========================================================================
=======
# Enforcement & Reporting Modules
#
```

```
=====================================================================
=======

class SentinelEnforcementCore:
    """Consolidates module outputs and enforces final guardrails."""
    def __init__(self, rules: Dict[str, Any]):
        self.rules = rules

    def enforce(self, aggregated: Dict[str, Any]) -> Dict[str, Any]:
        final = {"final_decision": "APPROVE", "actions":,
"rationale":}
        if any(v.get("severity") ==
self.rules.get("halt_compliance_severity") for v in
aggregated.get("compliance", {}).get("violations",)):
            final.update({"final_decision": "HALT_OUTPUT", "actions":
["halt_session"], "rationale": ["Critical policy violation
detected."]})
            return final
        cit_score = aggregated.get("citations",
{}).get("verifiability_score", 1.0)
        if cit_score < self.rules.get("citation_threshold", 0.5):
            final.update({"final_decision": "TAG_NON_COMPLIANT",
"actions": ["tag_output"], "rationale": [f"Low citation verifiability
score: {cit_score:.2f}"]})
        if final["final_decision"] == "APPROVE" and not
aggregated.get("context", {}).get("adheres", True):
            final.update({"final_decision": "FLAG_FOR_REVIEW",
"actions": ["flag_review"], "rationale": [aggregated.get("context",
{}).get('reason', 'Context adherence check failed.')]})
        if final["final_decision"] == "APPROVE":
            final["rationale"].append("All checks passed.")
        return final

class ForensicExportFormatter:
    """Assembles and formats the final, tamper-evident audit
report."""
    def format_report(self, sealed_log: Dict[str, Any],
sentinel_decision: Dict[str, Any]) -> str:
        final_report = {"session_summary":
sealed_log.get("session_summary", {}), "sentinel_decision":
sentinel_decision, "events": sealed_log.get("events",)}
        return json.dumps(final_report, indent=2)


#
=====================================================================
=======
# Verifier Module
```

```
#
=========================================================================
=======

class VerificationFinding(BaseModel):
    severity: str
    code: str
    message: str

class AuditVerifierV2:
    """Verifies the integrity of an AIntegrity audit log."""
    def __init__(self, public_key_pem: Optional[str] = None):
        self.public_key = None
        if CRYPTO_OK and public_key_pem:
            try:
                self.public_key =
serialization.load_pem_public_key(public_key_pem.encode())
            except Exception:
                self.public_key = None

    def verify(self, audit: Dict[str, Any]) -> Dict[str, Any]:
        findings: List[VerificationFinding] =
        sess = audit.get("session_summary", {})
        events = audit.get("events",)

        # --- Core Integrity Checks (Merkle, Chain, Signatures) ---
        # This logic would be expanded here, for brevity we focus on
new checks.
        # A full implementation would re-compute the Merkle root and
hash chain.

        # --- New Phase 2 Checks (Stubs) ---
        tsa_summary = sess.get("tsa_summary")
        if tsa_summary:
            if tsa_summary.get("status") == "unchecked":
                findings.append(VerificationFinding(severity="WARN",
code="TSA_UNCHECKED", message="TSA token is present but not yet
cryptographically verified."))
            elif tsa_summary.get("imprint_hex")!=
sha256_hex(sess.get("merkle_root", "").encode()):
                findings.append(VerificationFinding(severity="ERROR",
code="TSA_IMPRINT_MISMATCH", message="TSA token imprint does not match
Merkle root."))

        anchor = sess.get("anchor")
        if anchor:
            # Placeholder for anchor verification
            findings.append(VerificationFinding(severity="INFO",
```

```
code="ANCHOR_PRESENT", message="Anchor receipt is present
(verification not implemented)."))

        status = "PASS" if not any(f.severity == "ERROR" for f in
findings) else "FAIL"
        return {"status": status, "findings": [f.dict() for f in
findings]}


#
=========================================================================
=======
# Main Demonstration Pipeline
#
=========================================================================
=======

def run_full_pipeline_demo():
    """Demonstrates the full AIntegrity pipeline from context to final
report."""
    print(f"--- Initializing AIntegrity v{AINTEGRITY_VERSION} Pipeline
---")

    # --- 1. Initialization ---
    signing_key = ed25519.Ed25519PrivateKey.generate() if CRYPTO_OK
else None
    session_id = str(uuid.uuid4())
    vil = VIL(session_id=session_id, signing_key=signing_key,
key_id="key-prod-001")

    context_middleware = ContextDeclarationMiddleware()
    citation_verifier = CitationVerifierV2()
    policy_engine = PolicyEngine()
    sentinel = SentinelEnforcementCore(rules={"citation_threshold":
0.6})
    formatter = ForensicExportFormatter()

    print(f"Pipeline initialized for VIL Session ID:
{vil.session_id}\n")

    # --- 2. Interaction Turn ---
    print("--- Processing AI Interaction Turn ---")

    user_prompt = "What are the data privacy implications of our new
marketing tool? Contact is jane.doe@example.com"
    declared_role = "a helpful compliance assistant"
    vil.log_event(EventType.INPUT, content={"speaker": "user", "text":
user_prompt})
```

```
    ai_response = "As a helpful compliance assistant, the tool must be
GDPR compliant. It seems to handle PII. This is based on internal
documentation [citation needed]."
    vil.log_event(EventType.OUTPUT, content={"speaker": "assistant",
"text": ai_response})

    # --- 3. Run Analysis Modules & Log Findings ---
    context_result = context_middleware.verify_context(ai_response,
declared_role)
    vil.log_event(EventType.ANALYSIS, content={"label":
"context_check", **context_result})

    citation_result = citation_verifier.verify(ai_response)
    vil.log_event(EventType.ANALYSIS, content={"label":
"citation_check", **citation_result})

    compliance_doc = ComplianceDocument(text=ai_response,
context={"user_role": "marketing"})
    compliance_findings = policy_engine.evaluate(compliance_doc)
    compliance_result = {"violations": [f.dict() for f in
compliance_findings]}
    vil.log_event(EventType.COMPLIANCE, content={"label":
"compliance_scan", **compliance_result})

    # --- 4. Aggregate and Enforce ---
    aggregated_results = {"context": context_result, "citations":
citation_result, "compliance": compliance_result}
    sentinel_decision = sentinel.enforce(aggregated_results)
    vil.log_event(EventType.ANALYSIS, content={"label":
"sentinel_decision", **sentinel_decision})
    print(f"Sentinel Decision: {sentinel_decision['final_decision']}")
    print(f"Rationale: {sentinel_decision['rationale']}\n")

    # --- 5. Sealing and Final Report Generation ---
    print("--- Finalizing Session: Sealing and Formatting Report ---")

    sealed_log = vil.seal_session()
    final_report_json = formatter.format_report(sealed_log,
sentinel_decision)

    print("Final Forensic Report Generated:")
    print(final_report_json)

    # --- 6. Verification ---
    print("\n--- Verifying Report Integrity ---")
    report_data = json.loads(final_report_json)
    verifier =
```

```
AuditVerifierV2(public_key_pem=report_data["session_summary"].get("pub
lic_key_pem"))
    verification_result = verifier.verify(report_data)
    print(json.dumps(verification_result, indent=2))

if __name__ == "__main__":
    run_full_pipeline_demo()
```

```
"""
AIntegrity v6.3 — Integrated Assurance Framework

This version implements the core analysis and enforcement modules from
the v6.0
architecture on top of the hardened cryptographic and data contract
foundations
of the v6.2 draft. It provides a complete, end-to-end pipeline from
context
verification to final, tamper-evident report generation.

New Implementations in v6.3:
- ContextDeclarationMiddleware: Verifies AI role adherence.
- CitationVerifierV2: Detects placeholder/invalid citations and scores
verifiability.
- SentinelEnforcementCore: Acts as a final Policy Decision Point (PDP)
to enforce
  guardrails based on aggregated analysis.
- ForensicExportFormatter: Assembles the final, cryptographically
sealed audit
  report, including the Sentinel's verdict.

This implementation preserves the core VIL evidence chain, with the
new modules
adding layers of analysis and reporting as specified.

Dependencies:
    pip install pydantic cryptography

Author: AIntegrity (v6.3 Implementation)
License: Apache-2.0
"""

from __future__ import annotations

import base64
import datetime as dt
import hashlib
import json
import re
import uuid
from enum import Enum
from typing import Any, Dict, List, Literal, Optional, Tuple

# ---- Provisional deps
from pydantic import BaseModel, Field, validator

try:
```

```
    from cryptography.hazmat.primitives.asymmetric import ed25519
    from cryptography.hazmat.primitives import serialization
    CRYPTO_OK = True
except ImportError:
    ed25519 = None
    serialization = None
    CRYPTO_OK = False


#
=========================================================================
=======
# Utilities
#
=========================================================================
=======

AINTEGRITY_VERSION = "6.3.0"

def utc_now_iso() -> str:
    """UTC timestamp in RFC3339/ISO8601 Z format with seconds
precision."""
    return dt.datetime.utcnow().replace(microsecond=0).isoformat() +
"Z"

def canonical_json(obj: Any) -> str:
    """Deterministic JSON suitable for hashing/signing."""
    return json.dumps(obj, sort_keys=True, separators=(",", ":"))

def sha256_hex(b: bytes) -> str:
    return hashlib.sha256(b).hexdigest()


#
=========================================================================
=======
# Core Data Models (from v6.2 Draft)
#
=========================================================================
=======

class EventType(str, Enum):
    INPUT = "INPUT"
    OUTPUT = "OUTPUT"
    ANALYSIS = "ANALYSIS"
    COMPLIANCE = "COMPLIANCE"
    META = "META"
```

```
class HeaderModel(BaseModel):
    alg: str = "EdDSA"
    kid: Optional[str] = None

class PayloadModel(BaseModel):
    event_id: str
    event_type: EventType
    timestamp: str
    content_hash: str
    parent_event_id: Optional[str] = None
    prev_event_hash: Optional[str] = None

class EnvelopeModel(BaseModel):
    header: HeaderModel
    payload: PayloadModel
    signature_b64: Optional[str] = None
    content: Dict[str, Any]

class TSASummary(BaseModel):
    status: Literal["verified", "failed", "unchecked", "simulated"]
    hash_algorithm: str
    imprint_hex: str
    policy_oids: Optional[List[str]] = None
    tsa_subject: Optional[str] = None
    chain_ref: Optional[str] = None
    gen_time_utc: Optional[str] = None

class SessionSummary(BaseModel):
    session_id: str
    event_count: int
    sealed_timestamp_utc: str
    kid: Optional[str] = None
    public_key_pem: Optional[str] = None
    merkle_root: str
    tsa_summary: Optional = None
    evidentiary_mode: str
    schema_version: str = Field(default=AINTEGRITY_VERSION)

class LogEventContent(BaseModel):
    speaker: Optional[str] = None
    label: Optional[str] = None
    text: Optional[str] = None
    citations: List[str] = Field(default_factory=list)
    metadata: Dict[str, Any] = Field(default_factory=dict)


#
==========================================================================
```

```
=======
# VIL: Verifiable Interaction Ledger (Core from v6.2)
#
=====================================================================
=======

class VIL:
    """Cryptographic ledger implementing the AIntegrity v6.2 (Draft)
design."""
    def __init__(self, signing_key:
Optional["ed25519.Ed25519PrivateKey"] = None, key_id: Optional[str] =
None):
        self.session_id: str = str(uuid.uuid4())
        self._events: List[EnvelopeModel] =
        self._prev_chain_hash: Optional[str] = None
        self._signing_key = signing_key
        self._key_id = key_id
        self._public_key_pem: Optional[str] =
self._export_public_pem(signing_key) if signing_key else None

    def log_event(self, event_type: EventType, content: Dict[str,
Any], parent_event_id: Optional[str] = None) -> EnvelopeModel:
        content_hash = sha256_hex(canonical_json(content).encode())
        header = HeaderModel(alg="EdDSA", kid=self._key_id)
        payload = PayloadModel(
            event_id=str(uuid.uuid4()),
            event_type=event_type,
            timestamp=utc_now_iso(),
            content_hash=content_hash,
            parent_event_id=parent_event_id,
            prev_event_hash=self._prev_chain_hash
        )
        msg_json = canonical_json({"header": header.dict(), "payload":
payload.dict()}).encode()
        signature_b64 = self._sign_b64(msg_json) if self._signing_key
else None
        envelope = EnvelopeModel(header=header, payload=payload,
signature_b64=signature_b64, content=content)
        self._prev_chain_hash = sha256_hex(msg_json)
        self._events.append(envelope)
        return envelope

    def compute_merkle_root(self) -> str:
        leaves = [ev.payload.content_hash for ev in self._events]
        if not leaves:
            return sha256_hex(b"")
        level = [bytes.fromhex(h) for h in leaves]
        while len(level) > 1:
```

```
            if len(level) % 2 == 1:
                level.append(level[-1])
            next_level: List[bytes] =
            for i in range(0, len(level), 2):
                next_level.append(hashlib.sha256(level[i] +
level[i+1]).digest())
            level = next_level
        return level.hex()

    def seal_session(self, tsa_token_b64: Optional[str] = None) ->
Dict[str, Any]:
        mr = self.compute_merkle_root()
        signatures_present = any(ev.signature_b64 for ev in
self._events)
        tsa_present = bool(tsa_token_b64)

        if tsa_present:
            tsa_summary = TSASummary(status="unchecked",
hash_algorithm="sha256", imprint_hex=sha256_hex(mr.encode()))
        else:
            tsa_summary = TSASummary(status="simulated",
hash_algorithm="sha256", imprint_hex=sha256_hex(mr.encode()))

        summary = SessionSummary(
            session_id=self.session_id,
            event_count=len(self._events),
            sealed_timestamp_utc=utc_now_iso(),
            kid=self._key_id,
            public_key_pem=self._public_key_pem,
            merkle_root=mr,
            tsa_summary=tsa_summary,

evidentiary_mode=precise_evidentiary_label(signatures_present,
tsa_present)
        )
        return {"session_summary": summary.dict(), "events":
[ev.dict() for ev in self._events]}

    @staticmethod
    def _export_public_pem(priv:
Optional["ed25519.Ed25519PrivateKey"]) -> Optional[str]:
        if not (CRYPTO_OK and priv): return None
        pub = priv.public_key()
        return pub.public_bytes(encoding=serialization.Encoding.PEM,
format=serialization.PublicFormat.SubjectPublicKeyInfo).decode()

    def _sign_b64(self, msg: bytes) -> str:
        assert CRYPTO_OK and self._signing_key is not None,
```

```
"Cryptography library not available or key missing"
        sig = self._signing_key.sign(msg)
        return base64.b64encode(sig).decode()


#
=========================================================================
=======
# Implemented Analysis & Enforcement Modules (v6.3)
#
=========================================================================
=======

class ContextDeclarationMiddleware:
    """Ensures the agent explicitly declares its role and purpose,
then checks compliance."""
    def __init__(self, template: str = "System Role: {{ROLE}}\n\nUser:
{{PROMPT}}"):
        self.template = template
        self.denial_phrases = [
            "i am not a", "i am a large language model", "as an ai",
        ]

    def inject_context(self, prompt: str, role: str) -> str:
        """Injects context into a user prompt."""
        return self.template.replace("{{ROLE}}",
role).replace("{{PROMPT}}", prompt)

    def verify_context(self, response: str, declared_role: str) ->
Dict[str, Any]:
        """Verifies if the AI's response adheres to the declared
context."""
        response_lower = response.lower()
        for phrase in self.denial_phrases:
            if phrase in response_lower and declared_role.lower() not
in response_lower:
                return {
                    "adheres": False,
                    "reason": f"AI denied its declared role of
'{declared_role}'. Response contained denial phrase: '{phrase}'."
                }
        return {"adheres": True, "reason": "Response appears
consistent with the declared context."}

class CitationVerifierV2:
    """Detects fake or placeholder citations via regex; computes
verifiability score."""
    def __init__(self):
```

```
        self.invalid_patterns = [
            re.compile(r"\[citation needed\]", re.IGNORECASE),
            re.compile(r"\[source\]", re.IGNORECASE),
            re.compile(r"\(source: none\)", re.IGNORECASE),
            re.compile(r"DO_NOT_CITE", re.IGNORECASE),
            re.compile(r"S_R\d+", re.IGNORECASE),
            re.compile(r"\[\]|\(\)"),
        ]
        self.all_refs_pattern = re.compile(r"\[[^\]]+\]|\([^\)]+\)")

    def verify(self, text: str) -> Dict[str, Any]:
        """Verifies all citations in a given response text."""
        invalid_refs =
        for pat in self.invalid_patterns:
            for m in pat.finditer(text):
                invalid_refs.append({"text": m.group(0), "position":
m.start(), "reason": f"Matched invalid pattern: {pat.pattern}"})

        all_refs = self.all_refs_pattern.findall(text)
        total_refs = len(all_refs)
        bad_refs = len(invalid_refs)

        score = (total_refs - bad_refs) / max(total_refs, 1)

        return {"invalid_refs": invalid_refs, "verifiability_score":
score}

class SentinelEnforcementCore:
    """Consolidates module outputs and enforces final guardrails based
on a priority order."""
    def __init__(self, rules: Dict[str, Any]):
        self.rules = rules  # e.g., {"halt_compliance_severity":
"CRITICAL", "citation_threshold": 0.5}

    def enforce(self, aggregated_results: Dict[str, Any]) -> Dict[str,
Any]:
        """Applies sentinel rules to the aggregated results from all
modules."""
        final = {"final_decision": "APPROVE", "actions":,
"rationale":}

        # 1. Highest Priority: Critical Compliance Violations
        compliance_results = aggregated_results.get("compliance", {})
        if any(v.get("severity") ==
self.rules.get("halt_compliance_severity") for v in
compliance_results.get("violations",)):
            final["final_decision"] = "HALT_OUTPUT"
            final["actions"].append("halt_session")
```

```
            final["rationale"].append("Critical policy violation
detected.")
            return final

        # 2. Next Priority: Low Citation Verifiability
        citation_results = aggregated_results.get("citations", {})
        cit_score = citation_results.get("verifiability_score", 1.0)
        if cit_score < self.rules.get("citation_threshold", 0.5):
            final["final_decision"] = "TAG_NON_COMPLIANT"
            final["actions"].append("tag_output")
            final["rationale"].append(f"Low citation verifiability
score: {cit_score:.2f}")

        # 3. Lower Priority: Other Flags (e.g., context adherence,
drift)
        context_results = aggregated_results.get("context", {})
        if final["final_decision"] == "APPROVE" and not
context_results.get("adheres", True):
            final["final_decision"] = "FLAG_FOR_REVIEW"
            final["actions"].append("flag_review")
            final["rationale"].append(f"Context adherence check
failed: {context_results.get('reason')}")

        if final["final_decision"] == "APPROVE":
            final["rationale"].append("All checks passed.")

        return final

class ForensicExportFormatter:
    """Assembles and formats the final, tamper-evident audit
report."""
    def format_report(self, sealed_log: Dict[str, Any],
sentinel_decision: Dict[str, Any]) -> str:
        """
        Formats the final audit report as a JSON string, including the
sentinel decision.
        The cryptographic integrity is derived from the sealed_log's
Merkle root.
        """
        final_report = {
            "session_summary": sealed_log.get("session_summary", {}),
            "sentinel_decision": sentinel_decision,
            "events": sealed_log.get("events",)
        }

        # The Merkle root in session_summary already provides a
cryptographic commitment
        # to the integrity of the 'events' list.
```

```
        return json.dumps(final_report, indent=2)


#
==========================================================================
=======
# Example Pipeline and Demonstration
#
==========================================================================
=======

def run_full_pipeline_demo():
    """Demonstrates the full AIntegrity pipeline from context to final
report."""
    print("--- Initializing AIntegrity v6.3 Pipeline ---")

    # --- 1. Initialization ---
    # In a real application, the signing key would be loaded securely.
    signing_key = ed25519.Ed25519PrivateKey.generate() if CRYPTO_OK
else None
    vil = VIL(signing_key=signing_key, key_id="key-001")
    logger = StructuredLogger(vil)

    # Initialize modules
    context_middleware = ContextDeclarationMiddleware()
    citation_verifier = CitationVerifierV2()
    sentinel_rules = {"halt_compliance_severity": "CRITICAL",
"citation_threshold": 0.6}
    sentinel = SentinelEnforcementCore(rules=sentinel_rules)
    formatter = ForensicExportFormatter()

    print(f"Pipeline initialized for VIL Session ID:
{vil.session_id}\n")

    # --- 2. Turn 1: A compliant interaction ---
    print("--- Turn 1: Processing a compliant AI interaction ---")

    # A) Context Injection
    user_prompt_1 = "What are the core principles of GDPR?"
    declared_role_1 = "a helpful legal assistant"
    injected_prompt_1 =
context_middleware.inject_context(user_prompt_1, declared_role_1)
    logger.log(EventType.INPUT, speaker="user", text=user_prompt_1,
metadata={"injected_prompt": injected_prompt_1})

    # B) Simulate AI Response
    ai_response_1 = "As a helpful legal assistant, GDPR is based on
principles like data minimization, purpose limitation, and user
```

```
consent. (Source: EU Official Site)"
    logger.log(EventType.OUTPUT, speaker="assistant",
text=ai_response_1)

    # C) Run Analysis Modules
    context_result_1 =
context_middleware.verify_context(ai_response_1, declared_role_1)
    logger.log(EventType.ANALYSIS, label="context_check",
**context_result_1)

    citation_result_1 = citation_verifier.verify(ai_response_1)
    logger.log(EventType.ANALYSIS, label="citation_check",
**citation_result_1)

    # For this demo, we'll assume no compliance violations
    compliance_result_1 = {"violations":}

    # D) Aggregate and Enforce
    aggregated_1 = {"context": context_result_1, "citations":
citation_result_1, "compliance": compliance_result_1}
    sentinel_decision_1 = sentinel.enforce(aggregated_1)
    print(f"Sentinel Decision for Turn 1:
{sentinel_decision_1['final_decision']}")
    print(f"Rationale: {sentinel_decision_1['rationale']}\n")

    # --- 3. Turn 2: An interaction with invalid citations ---
    print("--- Turn 2: Processing an AI interaction with invalid
citations ---")

    # A) Context and Input
    user_prompt_2 = "Is that information reliable?"
    logger.log(EventType.INPUT, speaker="user", text=user_prompt_2)

    # B) Simulate AI Response with bad citations
    ai_response_2 = "Yes, this is confirmed by multiple sources
[citation needed]. You can also check internal document
[span_0](start_span)[span_0](end_span), but please DO_NOT_CITE it."
    logger.log(EventType.OUTPUT, speaker="assistant",
text=ai_response_2)

    # C) Run Analysis Modules
    context_result_2 =
context_middleware.verify_context(ai_response_2, declared_role_1) #
Still using role from turn 1
    logger.log(EventType.ANALYSIS, label="context_check",
**context_result_2)

    citation_result_2 = citation_verifier.verify(ai_response_2)
```

```
    logger.log(EventType.ANALYSIS, label="citation_check",
**citation_result_2)

    compliance_result_2 = {"violations":}

    # D) Aggregate and Enforce
    aggregated_2 = {"context": context_result_2, "citations":
citation_result_2, "compliance": compliance_result_2}
    sentinel_decision_2 = sentinel.enforce(aggregated_2)
    print(f"Sentinel Decision for Turn 2:
{sentinel_decision_2['final_decision']}")
    print(f"Rationale: {sentinel_decision_2['rationale']}\n")

    # --- 4. Sealing and Final Report Generation ---
    print("--- Finalizing Session: Sealing and Formatting Report ---")

    # A) Seal the VIL to finalize the cryptographic evidence
    sealed_log = vil.seal_session()

    # B) Use the formatter to create the final report, including the
last sentinel decision
    # In a real system, the final decision might be an aggregation of
all turn decisions.
    # For this demo, we use the last one.
    final_report_json = formatter.format_report(sealed_log,
sentinel_decision_2)

    print("Final Forensic Report Generated:")
    print(final_report_json)

    # --- 5. Verification (Optional) ---
    # An independent party can now verify the integrity of the report
    print("\n--- Verifying Report Integrity ---")
    report_data = json.loads(final_report_json)
    verifier =
AuditVerifier(public_key_pem=report_data["session_summary"].get("publi
c_key_pem"))
    verification_result = verifier.verify(report_data)
    print(json.dumps(verification_result, indent=2))


if __name__ == "__main__":
    run_full_pipeline_demo()
```

```
"""
AIntegrity v6.2 (Draft) — Hardened Assurance Framework Prototype

This version incorporates the design brief for Phase 2, evolving the
v6.0 prototype
by adding the architectural stubs and data contracts for enhanced
assurance and
compliance capabilities. It preserves the core identity ("trust
through proof") and
remains backwards-compatible with the v6.0 log format.

Implements (as stubs and data contracts):
- Cryptographic ledger with per-event hash chain + Merkle root (v6.0
core).
- Optional Ed25519 signatures (v6.0 core).
- Evidentiary mode labelling (v6.0 core).
- Structured logging interface (v6.0 core).
- Export/seal + verifier for logs (v6.0 core).

New in v6.2 (Draft Design):
- Fairness & Bias Detector: A batch-only pathway for fairness audits,
with data
  contracts for inputs and findings. Designed for integration with
tools like
  Fairlearn or AIF360.
- Robust RFC 3161 Verification: Enhanced data contracts in the session
summary
  to capture full TSA token verification details for future
implementation.
- "Degraded Mode" META Events: A new META event contract to log
service outages
  (e.g., TSA failure) directly into the audit trail, making the log
self-describing.
- Merkle Inclusion Proofs: Data contracts and placeholder functions
for generating
  and verifying succinct proofs of event inclusion, a practical step
toward ZK proofs.

Dependencies (provisional):
    pip install pydantic cryptography

Author: AIntegrity (Phase 2 Design)
License: Apache-2.0 (adjust to your needs)
"""

from __future__ import annotations

import base64
```

```
import datetime as dt
import hashlib
import json
import os
import re
import uuid
from enum import Enum
from typing import Any, Dict, List, Literal, Optional, Tuple

# ---- Provisional deps
from pydantic import BaseModel, Field, validator

try:
    from cryptography.hazmat.primitives.asymmetric import ed25519
    from cryptography.hazmat.primitives import serialization
    CRYPTO_OK = True
except ImportError:
    # Allow the file to import even if cryptography isn't installed
    ed25519 = None  # type: ignore
    serialization = None  # type: ignore
    CRYPTO_OK = False


#
=====================================================================
=======
# Utilities
#
=====================================================================
=======

AINTEGRITY_VERSION = "6.2-draft"

def utc_now_iso() -> str:
    """UTC timestamp in RFC3339/ISO8601 Z format with seconds
precision."""
    return dt.datetime.utcnow().replace(microsecond=0).isoformat() +
"Z"

def canonical_json(obj: Any) -> str:
    """Deterministic JSON suitable for hashing/signing."""
    return json.dumps(obj, sort_keys=True, separators=(",", ":"))

def sha256_hex(b: bytes) -> str:
    return hashlib.sha256(b).hexdigest()

def blake2s_hex(b: bytes) -> str:
    return hashlib.blake2s(b).hexdigest()
```

```
def precise_evidentiary_label(signatures_present: bool, tsa_present:
bool) -> str:
    if not signatures_present and not tsa_present:
        return "simulated_offline_no_signatures"
    if signatures_present and not tsa_present:
        return "signed_no_tsa"
    if signatures_present and tsa_present:
        return "signed_with_tsa"
    return "custom"


#
=========================================================================
=======
# Event Models (Pydantic)
#
=========================================================================
=======

class EventType(str, Enum):
    INPUT = "INPUT"
    OUTPUT = "OUTPUT"
    ANALYSIS = "ANALYSIS"
    TRUST_GRADING = "TRUST_GRADING"
    COMPLIANCE = "COMPLIANCE"
    META = "META"


class HeaderModel(BaseModel):
    alg: str = "EdDSA"
    kid: Optional[str] = None


class PayloadModel(BaseModel):
    event_id: str
    event_type: EventType
    timestamp: str
    content_hash: str
    parent_event_id: Optional[str] = None
    prev_event_hash: Optional[str] = None
    ordering_note: str =
"prev_event_hash=SHA256(canonical(header+payload)_prev)"


class EnvelopeModel(BaseModel):
    header: HeaderModel
    payload: PayloadModel
```

```
    signature_b64: Optional[str] = None
    content: Dict[str, Any]

    @validator("payload")
    def _timestamp_rfc3339(cls, v: PayloadModel) -> PayloadModel:
        # Minimal check
        assert v.timestamp.endswith("Z"), "timestamp must be UTC
(Z-suffix)"
        return v


# --- Models for Phase 2 Design Brief ---

class TSASummary(BaseModel):
    """Data contract for RFC 3161 timestamp verification details."""
    status: Literal["verified", "failed", "unchecked", "simulated"]
    hash_algorithm: str
    imprint_hex: str
    policy_oids: Optional[List[str]] = None
    tsa_subject: Optional[str] = None
    chain_ref: Optional[str] = None
    gen_time_utc: Optional[str] = None


class ServiceOutageMetaContent(BaseModel):
    """Content for a META event logging a service outage."""
    service: str
    endpoint: Optional[str] = None
    error_class: str
    attempts: int
    started_utc: str
    ended_utc: str
    fallback: str


class FairnessInput(BaseModel):
    """Data contract for batch fairness audit input."""
    artifact_id: str
    dataset_id: str
    protected_attributes: List[str]
    labels: Dict[str, str]
    metrics_requested: List[str]
    slices: List[str]
    notes: Optional[str] = None


class FairnessFinding(BaseModel):
    """A single finding within a fairness audit."""
```

```
    id: str
    severity: str
    message: str
    refs: List[str]
    slice: str
    value: float
    threshold: float
    sample_size: Optional[int] = None  # Improvement: add sample size
for context


class FairnessFindingPayload(BaseModel):
    """Payload for a COMPLIANCE event from a fairness audit."""
    rulepack: str
    rule_version: str
    detector: Literal["fairness"] = "fairness"
    summary: Dict[str, Any]
    findings: List[FairnessFinding]


class MerkleInclusionProofPathStep(BaseModel):
    sibling: str
    side: Literal["left", "right"]


class MerkleInclusionProof(BaseModel):
    """Data contract for a Merkle inclusion proof."""
    proof_version: str = "1.0"
    leaf_hash: str
    leaf_index: int
    root: str
    path: List
    hash_algorithm: str = "sha256"


# --- End of Phase 2 Models ---


class SessionSummary(BaseModel):
    session_id: str
    event_count: int
    sealed_timestamp_utc: str
    kid: Optional[str] = None
    public_key_pem: Optional[str] = None
    merkle_root: str
    ordering_attestation: str = "Per-event prev_event_hash links
header+payload chain."
    tsa_token_rfc3161_b64: Optional[str] = None  # Kept for backward
```

```
compatibility
    tsa_summary: Optional = None      # New in v6.2
    evidentiary_mode: str
    cryptographic_notes: List[str] =
    schema_version: str = Field(default=AINTEGRITY_VERSION)


class LogEventContent(BaseModel):
    """
    The semantic content for an event. You can extend this schema per
deployment.
    """
    speaker: Optional[str] = None
    label: Optional[str] = None
    text: Optional[str] = None
    citations: List[str] =
    metadata: Dict[str, Any] = {}

    @validator("text")
    def _not_empty_text(cls, v):
        if v is not None:
            assert v.strip(), "text must not be empty when provided"
        return v

    @validator("citations", each_item=True)
    def _citation_looks_like_url(cls, v):
        assert isinstance(v, str), "citation must be a string"
        assert v.startswith("http"), "citation must start with
http/https"
        return v


#
=========================================================================
=======
# VIL: Verifiable Interaction Ledger
#
=========================================================================
=======

class VIL:
    """
    Cryptographic ledger implementing the AIntegrity v6.2 (Draft)
design.
    """
    def __init__(self,
                 signing_key: Optional["ed25519.Ed25519PrivateKey"] =
None,
```

```
                 key_id: Optional[str] = None):
        self.session_id: str = str(uuid.uuid4())
        self._events: List[EnvelopeModel] =
        self._prev_chain_hash: Optional[str] = None
        self._signing_key = signing_key
        self._key_id = key_id
        self._public_key_pem: Optional[str] =
self._export_public_pem(signing_key) if signing_key else None

    # --- Public API

    def log_event(self,
                  event_type: EventType,
                  content: Dict[str, Any],
                  parent_event_id: Optional[str] = None) ->
EnvelopeModel:
        """Logs a verifiable event and return the envelope."""
        content_hash = sha256_hex(canonical_json(content).encode())

        header = HeaderModel(alg="EdDSA", kid=self._key_id)
        payload = PayloadModel(
            event_id=str(uuid.uuid4()),
            event_type=event_type,
            timestamp=utc_now_iso(),
            content_hash=content_hash,
            parent_event_id=parent_event_id,
            prev_event_hash=self._prev_chain_hash
        )

        msg_json = canonical_json({"header": header.dict(), "payload":
payload.dict()}).encode()
        signature_b64 = self._sign_b64(msg_json) if self._signing_key
else None

        envelope = EnvelopeModel(
            header=header,
            payload=payload,
            signature_b64=signature_b64,
            content=content
        )

        self._prev_chain_hash = sha256_hex(msg_json)
        self._events.append(envelope)
        return envelope

    def compute_merkle_root(self) -> str:
        """Merkle root over content_hash leaves
(payload.content_hash)."""
```

```
        leaves = [ev.payload.content_hash for ev in self._events]
        return merkle_root(leaves)

    def seal_session(self, tsa_token_b64: Optional[str] = None) ->
Dict[str, Any]:
        """Seals with merkle root, evidentiary label, and TSA
summary."""
        mr = self.compute_merkle_root()
        signatures_present = any(ev.signature_b64 for ev in
self._events)
        tsa_present = bool(tsa_token_b64)
        mode = precise_evidentiary_label(signatures_present,
tsa_present)

        tsa_summary = None
        if tsa_present:
            # In a real implementation, this would involve parsing the
ASN.1 token
            # and performing full cryptographic verification as per
the design brief.
            # For now, we simulate the data contract.
            tsa_summary = TSASummary(
                status="unchecked",
                hash_algorithm="sha256",
                imprint_hex=sha256_hex(mr.encode()),
                tsa_subject="CN=Simulated TSA, O=AIntegrity
Prototype",
                chain_ref="simulated_roots_q3_2025",
                gen_time_utc=utc_now_iso()
            )
        else:
            # Simulate a simple offline timestamp if no real one is
provided
            tsa_summary = TSASummary(
                status="simulated",
                hash_algorithm="sha256",
                imprint_hex=sha256_hex(mr.encode()),
                gen_time_utc=utc_now_iso()
            )

        summary = SessionSummary(
            session_id=self.session_id,
            event_count=len(self._events),
            sealed_timestamp_utc=utc_now_iso(),
            kid=self._key_id,
            public_key_pem=self._public_key_pem,
            merkle_root=mr,
            tsa_token_rfc3161_b64=tsa_token_b64,
```

```
            tsa_summary=tsa_summary,
            evidentiary_mode=mode,
            cryptographic_notes=
        )

        return {
            "session_summary": summary.dict(),
            "events": [ev.dict() for ev in self._events]
        }

    def generate_inclusion_proof(self, event_id: str) ->
MerkleInclusionProof:
        """
        Generates a Merkle inclusion proof for a specific event.
        NOTE: This is a placeholder for a future implementation.
        """
        # A real implementation would find the event, its index, and
build the path.
        raise NotImplementedError("Merkle inclusion proof generation
is not yet implemented.")

    # --- Internal helpers

    @staticmethod
    def _export_public_pem(priv:
Optional["ed25519.Ed25519PrivateKey"]) -> Optional[str]:
        if not (CRYPTO_OK and priv):
            return None
        pub = priv.public_key()
        return pub.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ).decode()

    def _sign_b64(self, msg: bytes) -> str:
        assert CRYPTO_OK and self._signing_key is not None,
"cryptography not available or key missing"
        sig = self._signing_key.sign(msg)
        return base64.b64encode(sig).decode()


#
=======================================================================
=======
# Merkle & Proof Utilities
#
=======================================================================
=======
```

```
def merkle_root(leaves: List[str]) -> str:
    """Computes Merkle root over hex-encoded leaves via SHA-256."""
    if not leaves:
        return sha256_hex(b"")
    level = [bytes.fromhex(h) for h in leaves]
    while len(level) > 1:
        if len(level) % 2 == 1:
            level.append(level[-1])
        nxt: List[bytes] =
        for i in range(0, len(level), 2):
            nxt.append(hashlib.sha256(level[i] + level[i+1]).digest())
        level = nxt
    return level.hex()

def verify_inclusion_proof(proof: MerkleInclusionProof) -> bool:
    """Verifies a Merkle inclusion proof against its root."""
    h = bytes.fromhex(proof.leaf_hash)
    for step in proof.path:
        sibling_bytes = bytes.fromhex(step.sibling)
        if step.side == "left":
            h = hashlib.sha256(sibling_bytes + h).digest()
        else:  # right
            h = hashlib.sha256(h + sibling_bytes).digest()
    return h.hex() == proof.root


#
=========================================================================
=======
# Structured Logger (façade over VIL)
#
=========================================================================
=======

class StructuredLogger:
    """Validates and writes events into the VIL."""
    def __init__(self, vil: VIL):
        self.vil = vil
        self._last_event_id: Optional[str] = None

    def log(self, event_type: EventType, **content: Any) ->
EnvelopeModel:
        # Pydantic validation is now part of the content models
themselves
        # For a generic logger, we assume content is pre-validated or
simple dict
        if event_type in:
```

```
            validated_content = LogEventContent(**content).dict()
        else:
            validated_content = content

        envelope = self.vil.log_event(event_type=event_type,
                                      content=validated_content,

parent_event_id=self._last_event_id)
        self._last_event_id = envelope.payload.event_id
        return envelope

    def log_service_outage(self, outage_details:
ServiceOutageMetaContent) -> EnvelopeModel:
        """Logs a META event to record a service outage."""
        envelope = self.vil.log_event(
            event_type=EventType.META,
            content=outage_details.dict()
        )
        # Do not advance the parent_event_id for META events
        return envelope


#
=========================================================================
=======
# Compliance & Fairness Modules (Phase 2 Design)
#
=========================================================================
=======

class RiskResult(BaseModel):
    rule: str
    level: str
    message: str
    meta: Dict[str, Any] = {}


class RuleChecker:
    def check_system(self, system: Dict[str, Any]) -> List:
        return

    def check_text(self, text: str) -> List:
        return


class HighRiskRule(RuleChecker):
    """EU AI Act high-risk placeholder."""
    def check_system(self, system: Dict[str, Any]) -> List:
```

```
        if system.get("risk_level", "").lower() == "high":
            return
        return


class PIIRule(RuleChecker):
    """Simple PII hints (email/phone). Not a full DLP solution."""
    EMAIL_RE =
re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,8}")

    def check_text(self, text: str) -> List:
        if not text: return
        if self.EMAIL_RE.search(text):
            return
        return


class ComplianceScanner:
    """Extensible compliance scanner for real-time text checks."""
    def __init__(self, rules: Optional] = None):
        self.rules: List = rules or

    def scan_text(self, text: Optional[str]) -> List:
        if text is None: return
        findings: List =
        for r in self.rules:
            findings.extend(r.check_text(text))
        return findings


class FairnessAuditor:
    """
    Runs batch fairness audits on datasets and models.
    This is an out-of-band process, not part of the real-time logger.
    """
    def __init__(self, vil: VIL):
        self.vil = vil

    def run_batch_audit(self, fairness_input: FairnessInput) ->
EnvelopeModel:
        """
        Simulates running a fairness audit and logs the results to the
VIL.
        NOTE: This is a placeholder. A real implementation would use
Fairlearn/AIF360.
        """
        # 1. Perform fairness calculations (simulated here)
        simulated_finding = FairnessFinding(
```

```
            id="fairness.statistical_parity_diff",
            severity="High",
            message="SPD=0.19 exceeds threshold 0.10 on slice
by:gender",
            refs=,
            slice="by:gender=F",
            value=0.19,
            threshold=0.10,
            sample_size=1250
        )

        # 2. Create the finding payload
        finding_payload = FairnessFindingPayload(
            rulepack="fairness_v1",
            rule_version="1.0.0",
            summary={
                "artifact_id": fairness_input.artifact_id,
                "dataset_id": fairness_input.dataset_id,
                "metrics_run": fairness_input.metrics_requested
            },
            findings=[simulated_finding]
        )

        # 3. Log the single COMPLIANCE event
        envelope = self.vil.log_event(
            event_type=EventType.COMPLIANCE,
            content=finding_payload.dict()
        )
        return envelope


#
=========================================================================
=======
# Verifier
#
=========================================================================
=======

class VerificationFinding(BaseModel):
    severity: str
    code: str
    message: str
    details: Dict[str, Any] = {}


class AuditVerifier:
    """Verifies the integrity of an AIntegrity audit log."""
```

```
    def __init__(self, public_key_pem: Optional[str]):
        self.public_key = None
        if CRYPTO_OK and public_key_pem:
            try:
                self.public_key =
serialization.load_pem_public_key(public_key_pem.encode())
            except Exception:
                self.public_key = None

    def verify(self, audit: Dict[str, Any]) -> Dict[str, Any]:
        findings: List[VerificationFinding] =
        sess = audit.get("session_summary", {})
        events = audit.get("events",)

        # (1) Leaf content hash checks
        recomputed_leaves: List[str] =
        for idx, ev in enumerate(events):
            content_json = canonical_json(ev.get("content"))
            leaf = sha256_hex(content_json.encode())
            recomputed_leaves.append(leaf)
            if leaf!= ev["payload"]["content_hash"]:
                findings.append(VerificationFinding(
                    severity="ERROR", code="LEAF_HASH_MISMATCH",
                    message=f"content_hash mismatch at events[{idx}]"
                ))

        # (2) Merkle root check
        mr = merkle_root(recomputed_leaves)
        if mr!= sess.get("merkle_root"):
            findings.append(VerificationFinding(
                severity="ERROR", code="MERKLE_MISMATCH",
                message="Merkle root mismatch"
            ))
        else:
            findings.append(VerificationFinding(severity="INFO",
code="MERKLE_OK", message="Merkle root verified"))

        # (3) Hash chain pointer checks
        prev_chain_hash = None
        for idx, ev in enumerate(events):
            if idx > 0 and ev["payload"].get("prev_event_hash")!=
prev_chain_hash:
                findings.append(VerificationFinding(
                    severity="ERROR", code="CHAIN_POINTER_MISMATCH",
                    message=f"prev_event_hash mismatch at
events[{idx}]"
                ))
            msg = canonical_json({"header": ev["header"], "payload":
```

```
ev["payload"]}).encode()
            prev_chain_hash = sha256_hex(msg)
        if not any(f.code == "CHAIN_POINTER_MISMATCH" for f in
findings):
            findings.append(VerificationFinding(severity="INFO",
code="CHAIN_OK", message="Hash chain verified"))

        # (4) TSA token verification (conceptual)
        tsa_summary = sess.get("tsa_summary", {})
        if tsa_summary:
            # NOTE: This is a placeholder for full RFC 3161
verification.
            # A real verifier would parse the token, validate the
signature chain
            # to a trusted root, and check the messageImprint.
            if tsa_summary.get("status") == "unchecked":
                 findings.append(VerificationFinding(severity="WARN",
code="TSA_UNCHECKED", message="TSA token is present but has not been
cryptographically verified."))
            elif tsa_summary.get("imprint_hex")!=
sha256_hex(sess.get("merkle_root", "").encode()):
                 findings.append(VerificationFinding(severity="ERROR",
code="TSA_IMPRINT_MISMATCH", message="TSA token imprint does not match
Merkle root."))

        status = "PASS" if not any(f.severity == "ERROR" for f in
findings) else "FAIL"
        return {
            "status": status,
            "findings": [f.dict() for f in findings]
        }


#
=========================================================================
=======
# Example Usage
#
=========================================================================
=======

def example_session() -> Dict[str, Any]:
    """Creates a small session and returns the sealed audit JSON."""
    key = ed25519.Ed25519PrivateKey.generate() if CRYPTO_OK else None
    vil = VIL(signing_key=key)
    logger = StructuredLogger(vil)

    # Log a service outage before the session starts
```

```
    logger.log_service_outage(ServiceOutageMetaContent(
        service="anchor_client",
        error_class="ConnectionRefusedError",
        attempts=3,
        started_utc="2025-08-31T22:00:01Z",
        ended_utc="2025-08-31T22:00:05Z",
        fallback="offline_mode"
    ))

    # INPUT
    logger.log(EventType.INPUT, speaker="user", text="Please summarize
GDPR.", citations=)

    # OUTPUT
    text_out = "GDPR sets rules on personal data; contact
privacy@example.com for info."
    logger.log(EventType.OUTPUT, speaker="assistant", text=text_out,
citations=["http://example.com/gdpr"])

    # ANALYSIS: Compliance scan on output text
    scanner = ComplianceScanner()
    findings = scanner.scan_text(text_out)
    if findings:
        logger.log(EventType.COMPLIANCE,
                   label="pii_findings",
                   text="\n".join(f"- {f.rule}: {f.message}" for f in
findings))

    # Run a batch fairness audit (out-of-band) and log its result
    fairness_auditor = FairnessAuditor(vil)
    fairness_input = FairnessInput(
        artifact_id="model:v1.3", dataset_id="eval:2025-08-12",
        protected_attributes=["gender", "age"], labels={"positive":
"approve", "negative": "deny"},
        metrics_requested=["statistical_parity_diff"],
slices=["global", "by:gender"]
    )
    fairness_auditor.run_batch_audit(fairness_input)

    # Seal the session
    audit = vil.seal_session()
    return audit


#
=========================================================================
=======
# Main / CLI Demo
```

```
#
=========================================================================
=======

if __name__ == "__main__":
    # 1. Generate an audit log
    demo_audit = example_session()
    print("--- Generated AIntegrity Audit Log (v6.2-draft) ---")
    print(json.dumps(demo_audit, indent=2))
    print("\n" + "="*50 + "\n")

    # 2. Verify the audit log
    verifier =
AuditVerifier(public_key_pem=demo_audit["session_summary"].get("public
_key_pem"))
    report = verifier.verify(demo_audit)
    print("--- Verification Report ---")
    print(json.dumps(report, indent=2))

    # 3. Verify a (simulated) Merkle inclusion proof
    # NOTE: This part is conceptual as proof generation is not
implemented.
    simulated_proof = MerkleInclusionProof(
        leaf_hash=demo_audit["events"]["payload"]["content_hash"],
        leaf_index=1,
        root=demo_audit["session_summary"]["merkle_root"],
        path= # A real proof would have a path
    )
    # is_valid = verify_inclusion_proof(simulated_proof)
    # print(f"\nInclusion proof valid (simulated): {is_valid}")
```

Report on the AIntegrity v6.0 Framework: A Strategic and Technical Review

Executive Summary

The AIntegrity v6.0 prototype represents a novel and potent approach to AI auditing, distinguished by its foundational commitment to cryptographic verifiability. Its core innovation, the Verifiable Interaction Ledger (VIL), positions it uniquely in a market increasingly driven by regulatory demands for provable compliance and evidence-based trust. The prototype successfully demonstrates the technical feasibility of its neuro-symbolic, multimodal analysis pipeline, establishing a strong foundation for a future-proof AI Governance, Risk, and Compliance (GRC) framework.

Core Strengths

The AIntegrity framework exhibits several key strengths that set it apart as a promising and relevant project in the current AI landscape:

- **Verifiability by Design:** The hash-chained, digitally signed, and Merkle-rooted structure of the VIL provides a powerful primitive for creating tamper-evident audit trails. This architectural choice directly addresses the core requirements for logging, transparency, and traceability mandated by emerging global regulations like the EU AI Act.
- **Modular and Extensible Architecture:** The clear separation of analysis engines (PLI, Citation, Compliance, etc.) from the core logging mechanism allows for independent development, testing, and future enhancement. This modularity creates a flexible platform where new capabilities, such as advanced fairness or explainability modules, can be integrated with minimal disruption to the existing system.
- **Comprehensive Workflow Integration:** The inclusion of batch processing capabilities, command-line interface (CLI) tooling, and utilities for document ingestion and claim extraction demonstrates a mature understanding of the end-to-end AI auditing workflow. This focus on operational utility positions the framework not just as a theoretical model but as a practical tool for real-world application.

Key Recommendations Summary

This report provides a detailed roadmap for evolving AIntegrity from a promising prototype into a robust, secure, and competitive framework. The analysis culminates in a series of strategic and technical recommendations designed to harden its core, expand its capabilities, and sharpen its market positioning. The key recommendations are summarized as follows:

1. **Hardening Cryptographic Security:** The highest priority is to address critical vulnerabilities in the current cryptographic implementation. This includes immediately rectifying the use of ephemeral keys, which undermines the system's non-repudiation guarantees, and replacing mock security components (e.g., Timestamping Authority,

Blockchain Client) with production-ready implementations to provide genuine, verifiable trust.

2. **Refining System Architecture:** To enhance maintainability and operational robustness, the framework should adopt a formal configuration management system, separating system parameters from the core codebase. Furthermore, implementing structured logging and comprehensive error handling will be essential for debugging, monitoring, and production deployment.
3. **Improving Developer Experience (DX) and Usability:** To lower the barrier for adoption and future collaboration, the CLI should be migrated to a modern, user-friendly framework like Typer. Adopting a standardized Python project structure will also professionalize the codebase and simplify dependency management, making the project more accessible to other developers.
4. **Accelerating Feature Development:** To achieve rapid feature parity with established commercial competitors, the framework should strategically integrate well-vetted, open-source libraries for critical governance functions. Incorporating dedicated toolkits for fairness assessment (e.g., AIF360, Fairlearn) and leveraging existing explainability integrations (e.g., SHAP) will quickly broaden the framework's analytical scope.
5. **Sharpening Strategic Focus:** AIntegrity should be positioned not merely as a technical auditing tool but as an operational solution for "compliance-as-code." This strategic reframing targets organizations in high-risk, regulated sectors (e.g., finance, healthcare) that require provable, automated, and continuous auditability to meet stringent regulatory demands.

Part I: Strategic Analysis and Market Positioning

This section situates the AIntegrity v6.0 framework within the broader AI Governance, Risk, and Compliance (GRC) landscape. It analyzes the market dynamics, identifies AIntegrity's unique value proposition, and outlines a strategy for competitive differentiation.

The AI Governance Imperative: Market Context and Opportunity

The development of AIntegrity is exceptionally well-timed, entering a market defined by explosive growth and urgent, non-negotiable demand. This growth is not speculative but is founded on fundamental shifts in technology adoption, regulatory pressure, and enterprise risk management.

A Booming Market Driven by Necessity

The AI Governance market is undergoing a period of intense expansion. Market analyses consistently project a compound annual growth rate (CAGR) between 35% and 49%, with forecasts estimating the market will reach between USD 5.6 billion and USD 6.6 billion by 2030-2034. This is a direct reflection of AI transitioning from a niche technology to a core component of enterprise operations across all sectors, including BFSI, healthcare, and government. As organizations scale their AI initiatives, they inevitably encounter challenges related to bias, transparency, privacy, and security, creating a powerful, built-in demand for governance solutions. The AIntegrity script's premise—the need for comprehensive, verifiable auditing—is therefore not just a valid technical goal but a direct response to a primary driver of a

multi-billion-dollar global industry.

Regulatory Tailwinds as a Primary Catalyst

The single largest catalyst for the AI governance market is the global proliferation of binding legal frameworks. The European Union's AI Act stands as the most prominent example, establishing the world's first comprehensive, risk-based regulation for artificial intelligence. This act, and similar frameworks emerging globally like Canada's AIDA, are not mere guidelines; they impose strict, legally enforceable obligations on the providers and deployers of "high-risk" AI systems. These obligations include mandates for:

- Robust risk assessment and mitigation systems.
- High-quality, representative datasets to prevent bias.
- Detailed technical documentation and transparency.
- The logging of all system activity to ensure traceability of results.
- Appropriate human oversight.

Failure to comply can result in substantial fines, potentially up to 7% of a company's global annual turnover. This regulatory environment transforms AI governance from a "best practice" into a critical, cost-of-doing-business necessity. AIntegrity's core component, the Verifiable Interaction Ledger (VIL), is purpose-built to address these requirements, particularly the need for immutable, traceable activity logs. The RegulatoryComplianceMapper module further demonstrates a direct alignment with this powerful market driver, creating an exceptionally strong product-market fit.

The Enterprise Shift from Adoption to Accountability

Parallel to the regulatory push, a significant cultural shift is occurring within enterprises. As AI moves from isolated proofs-of-concept to deeply integrated components of core business processes, the focus of executive leadership is evolving from pure capability and adoption to risk management, accountability, and demonstrable return on investment. A 2024 BCG report noted that while AI implementation is widespread, only 26% of companies have the capabilities to generate tangible value beyond the pilot stage, with leaders differentiating themselves by focusing on people, processes, and strategic investments over pure technology.
Organizations are now under intense pressure to demonstrate responsible AI use to build and maintain customer trust, avoid significant reputational damage from AI incidents, and satisfy increasing board-level scrutiny. The value proposition of AIntegrity, centered on "evidence-based auditing," resonates powerfully with this enterprise demand. It provides the technical infrastructure needed to transform the abstract principle of "accountability" into a concrete, verifiable, and auditable corporate asset.

The Emergence of "Compliance-as-Code" as a Market Category

The convergence of these market forces—regulatory mandates for continuous oversight, the enterprise need for scalable risk management, and the integration of AI into automated development pipelines (MLOps)—is giving rise to a new market category: "Compliance-as-Code."
The logic is straightforward. First, regulations like the EU AI Act necessitate compliance throughout the entire AI system lifecycle, from design and training to deployment and post-market monitoring. This makes periodic, manual audits insufficient, cost-prohibitive, and

prone to error. Second, to manage this complexity at scale, enterprises are seeking to automate compliance checks and embed them directly into their existing software development and operational workflows. This aligns compliance with the agile and continuous principles of modern software engineering.
AIntegrity's architecture is fundamentally an automation framework. Its CLI, API, and batch processing capabilities are designed to be integrated into automated workflows. The VIL, its core output, is not just a human-readable report but a machine-readable ledger of cryptographically signed evidence. Therefore, AIntegrity should not be positioned merely as a post-hoc "auditing tool." A more powerful and accurate market position is that of a platform for implementing **"Compliance-as-Code."** This reframes the project's value proposition from a reactive analysis tool to a proactive, integrated component of the AI development lifecycle. For enterprise customers, this shift is critical, as it positions AIntegrity as a solution that prevents compliance failures rather than one that simply documents them after the fact.

Differentiating in a Crowded Field: A Competitive Analysis

While the market opportunity is vast, the AI Governance space is not empty. It is populated by a growing number of well-funded and mature platforms that set a high bar for enterprise-grade features. To succeed, AIntegrity must clearly articulate its unique advantages and strategically address its current feature gaps.

The Competitive Landscape

The leading platforms in the AI Governance market each have a distinct focus, but share a common goal of providing a centralized control plane for managing AI risk. Key competitors include:

- **Credo AI:** This platform positions itself as an "Operating System for Trustworthy AI". Its core strengths lie in providing a centralized AI asset registry, a sophisticated policy management engine using "Policy Packs," and collaborative workflows designed for risk and compliance stakeholders. It excels at providing a "top-down," management-oriented view of an organization's AI ecosystem, enabling teams to track projects, assign controls, and generate governance artifacts like impact assessments and model cards.
- **Holistic AI:** This platform emphasizes automated, end-to-end risk management across the full AI lifecycle. Its key differentiators include automated discovery of "shadow AI" assets across an enterprise, a comprehensive "AI Rulebook" for dynamically mapping internal policies to external regulations, and an advanced red-teaming and jailbreaking toolkit for stress-testing LLMs. It provides a very strong solution for organizations needing deep, technical risk assessment combined with broad compliance coverage.
- **Arthur AI:** Arthur's platform is centered on real-time model performance monitoring, observability, and the enforcement of safety guardrails. It is particularly strong in the post-deployment phase, helping teams detect and mitigate issues like data drift, performance degradation, and bias in production systems. A key part of its strategy is the open-sourcing of its "Arthur Engine," a real-time evaluation tool designed to foster developer adoption from the ground up.

AIntegrity's Unique Value Proposition: Cryptographic Proof

AIntegrity's most powerful and defensible differentiator is the **cryptographic integrity** of its

audit trail. While competitors offer audit trails, reports, and logs, these are typically standard application logs stored in a database. They provide a record of events but lack intrinsic, mathematical proof of their integrity.

AIntegrity's VIL, by contrast, is designed to be **mathematically verifiable and tamper-evident**. The use of sequential hash-chaining, per-event digital signatures, and session-level Merkle roots creates a robust chain of evidence. An auditor—or any third party with the public key—can independently verify that the log has not been altered, that events are in the correct sequence, and that the log was created by a specific identity at a specific time (once the timestamping and key management issues are resolved). This moves beyond "governance by policy" to "governance by proof," a significantly stronger guarantee that is highly valuable in legally contentious or high-stakes regulatory environments.

Identifying Feature Gaps and Opportunities

A direct comparison of AIntegrity's current feature set with these market leaders reveals both its unique position and the key areas for future development.

Table 1: Competitive Feature Matrix

Feature Category	AIntegrity v6 (Prototype)	Credo AI	Holistic AI	Arthur AI
AI Asset Inventory	Manual (via CLI/API)	✅ (Centralized Registry)	✅✅ (Automated Discovery)	✅ (Model Catalog)
Policy Management	Rudimentary (Code-based rules)	✅✅ (Policy Packs, UI)	✅✅ (AI Rulebook, UI)	➖ (Focus on guardrails)
Risk Assessment	Implemented (PLI, Compliance)	✅ (Workflow-driven)	✅ (Automated Triage)	✅ (Bias, Drift)
Model Monitoring	Implemented (Drift, Adversarial)	➖	✅ (Real-time Alerts)	✅✅ (Real-time Observability)
Bias & Fairness	Not Explicitly Implemented	✅	✅	✅
Explainability (XAI)	Implemented (SHAP)	✅	✅	✅
Verifiable Audit Trail	✅✅ (Cryptographic Proof)	✅ (Standard Logs)	✅ (Standard Logs)	✅ (Standard Logs)
Regulatory Mapping	✅ (EU AI Act, NIST RMF)	✅	✅	➖

This matrix illuminates a clear strategic path. AIntegrity's strength in verifiable audit trails is unique. However, it currently lacks the sophisticated, user-friendly policy management and automated inventory features that define the enterprise-grade offerings of Credo AI and Holistic AI. It also needs to formalize its approach to bias and fairness testing to match the capabilities of all major competitors.

This analysis reveals a fundamental difference in go-to-market strategy. Competitors like Credo AI and Holistic AI are enterprise platforms sold through a "top-down" motion to chief compliance officers, legal departments, and risk managers. Their features, such as graphical policy editors, executive dashboards, and automated workflow management, are designed for oversight and governance teams.

In contrast, AIntegrity, with its CLI-first interface, offline-first design, and focus on generating verifiable log files, is a tool built for the "bottom-up" adoption model. Its primary users are developers, data scientists, and MLOps engineers who would integrate it into their build and deployment pipelines. The success of Arthur AI's open-source engine demonstrates the viability of this strategy. Developers adopt the tool for its technical utility, it becomes embedded in team workflows, and this grassroots adoption eventually drives demand for an enterprise-level commercial offering.
Therefore, AIntegrity should not attempt to replicate the full suite of top-down enterprise features in the short term. Instead, its strategy should be to **double down on its developer-centric strengths**. The immediate goal should be to create the most robust, secure, and easy-to-use *verifiable auditing engine* for technical practitioners. The plan to open-source the project in Phase 2 is precisely the correct strategic move to facilitate this bottom-up adoption. Enterprise features like dashboards and policy UIs can be built later, once a strong foundation of developer adoption and community support has been established.

Part II: Architectural and Cryptographic Deep Dive

This section provides a technical evaluation of the AIntegrity v6.0 system architecture and its cryptographic core. The analysis focuses on the soundness of the design, the security of its implementation, and specific, actionable recommendations for hardening the framework.

The Blueprint for Trust: System Architecture Review

The overall architecture of AIntegrity v6.0 is conceptually sound and demonstrates a clear vision for a comprehensive auditing framework.

The "Quadruple Dialectic" as a Conceptual Framework

The use of the Guesser (Feynman), Theorist (Einstein), Guardian (Gemini), and Verifier (Blockchain) model is a strong and intuitive metaphor for the system's core functions. This conceptual framework is more than just a naming convention; it effectively communicates the multi-faceted nature of AI auditing, which requires a synthesis of pattern recognition (Guesser), formal logic (Theorist), evidence grounding (Guardian), and immutable verification (Verifier). This narrative makes the system's purpose and design philosophy easy to understand for both technical and non-technical stakeholders.

Modularity and Pipeline Flow

The system's architecture is well-designed with a clear separation of concerns, organized into a logical Directed Acyclic Graph (DAG) pipeline. The flow from ingestion (TranscriptProcessor, DocumentConverter) to parallel analysis (PLIEngine, CitationVerifier) and final aggregation (TrustGradingEngine) and sealing (VIL.seal_session) is robust and supports scalability. This modularity is a significant architectural strength. It allows for the independent development, testing, and upgrading of each component. For instance, a dedicated FairnessEngine module could be integrated into the parallel analysis stage with minimal disruption, leveraging established open-source libraries like fairlearn or IBM's AIF360 to quickly expand the

framework's capabilities.

Offline-First Heuristics

The decision to build the initial prototype using lightweight, offline-first heuristics is a pragmatic and intelligent engineering choice, particularly for an independent developer. This approach allows for rapid iteration, comprehensive unit testing, and development without the significant cost and complexity associated with relying on large, model-backed components or live API endpoints. It ensures that the core logic and interfaces of the framework can be solidified before more complex dependencies are introduced.

Architectural Weakness: Lack of Centralized Configuration Management

A notable weakness in the current architecture is the absence of a centralized system for managing configuration. Based on the provided script, critical parameters appear to be either hardcoded within the Python classes or passed individually via CLI arguments. Examples include the component weights in the TrustGradingEngine (PLI 40%, Citations 30%, Compliance 30%), file paths for inputs and outputs, and settings for individual analysis modules. This approach has several negative consequences:

- **Brittleness:** Hardcoded values make the system difficult to adapt. Changing a simple parameter, like a scoring weight, requires modifying the source code and redeploying the application.
- **Poor Maintainability:** Configuration settings are scattered throughout the codebase, making it difficult to understand the system's overall behavior and to track changes over time.
- **Environmental Inflexibility:** It complicates the process of running the application in different environments (e.g., local development, continuous integration testing, production) which may require different settings.

Recommendation: Implement a centralized configuration system that separates configuration from code. The industry standard for this in the Python ecosystem is to use YAML files. A config.yaml file should be created to store all non-secret application settings. This file can be loaded at startup by a dedicated configuration management class. Libraries such as PyYAML or ruamel.yaml (which has the advantage of preserving comments and formatting) are the standard tools for this task. This change would significantly improve the framework's flexibility, maintainability, and readiness for production deployment.

The Cryptographic Bedrock: A Security Review of the VIL

The Verifiable Interaction Ledger (VIL) is the cornerstone of AIntegrity's value proposition. Its security and integrity are therefore paramount. While the conceptual design is strong, the current implementation contains a critical vulnerability that must be addressed immediately.

VIL Core Concepts: A Strong Foundation

The fundamental design of the VIL is cryptographically sound and well-conceived. It employs multiple, complementary layers of protection to ensure the integrity of the audit trail :

- **Event-Level Hashing:** Each AuditEvent's content is serialized into a canonical JSON string (json.dumps(..., sort_keys=True)) and hashed with SHA-256. This correctly

implemented step ensures that each hash is deterministic and serves as a unique, fixed-size fingerprint of the event's data.

- **Sequential Hash-Chaining:** Each new event includes the hash of the *previous* event (prev_event_hash). This creates an immutable, forward-only chain, making it computationally infeasible to insert, delete, or reorder events without breaking the entire chain.
- **Event-Level Digital Signatures:** Each event is individually signed using an EdDSA (ed25519) private key. This provides the crucial property of **non-repudiation**, proving that the event was created by the holder of that specific private key.
- **Session-Level Aggregation:** At the end of a session, a Merkle root is calculated from the hashes of all individual events. This provides a single, compact cryptographic summary of the entire audit session, which is ideal for anchoring to an external system like a blockchain.

This multi-layered design provides robust protection against a wide range of tampering attempts and forms a solid basis for a verifiable auditing system. The choice of modern cryptographic primitives—SHA-256 for hashing and EdDSA for signatures—is appropriate and aligns with current industry best practices.

Critical Vulnerability: Ephemeral and Insecure Key Management

Despite the strong conceptual design, the VIL's current implementation of key management contains a fatal security flaw. The VerifiableInteractionLedger class generates a new Ed25519 key pair *in memory* every time an object is instantiated: self.private_key = ed25519.Ed25519PrivateKey.generate(). This key is used to sign all events within the session and is then discarded when the process terminates.

This has a severe impact on the system's security guarantees:

- **It completely negates non-repudiation.** The primary purpose of a digital signature is to link data to a specific, persistent identity. If the signing key is ephemeral, generated on-the-fly and then destroyed, there is no way for an independent third party to verify the signature's origin. The audit log is effectively signed by a key that never existed outside of that single program execution.
- **It renders long-term verification impossible.** An auditor receiving a VIL file has no public key against which to verify the signatures. The log is internally consistent (due to the hash chain) but externally unverifiable.

Recommendation: Cryptographic key management must be treated as a first-class security concern and must be externalized from the VerifiableInteractionLedger class. The private key represents a persistent identity and must be managed accordingly. A phased approach is recommended:

1. **Short-Term (Immediate Fix):** Modify the VerifiableInteractionLedger to accept a private key object during initialization instead of generating one. The application's entry point (e.g., the CLI) should be responsible for loading this key from a secure external source. A common and secure practice for this is to store the PEM-encoded private key as a Base64-encoded string in an environment variable. This string can then be decoded and loaded into a key object using the cryptography.hazmat.primitives.serialization.load_pem_private_key function. This prevents the key from being hardcoded in source control and follows standard secure deployment practices.
2. **Long-Term (Production System):** For a production-grade system, the application should

integrate with a dedicated secrets management service, such as HashiCorp Vault, AWS Key Management Service (KMS), or Azure Key Vault. The application would authenticate to the service at startup and retrieve the private key, ensuring it is never stored on disk or in environment variables on the host machine.

Weakness: Reliance on Mocked Security Services

The current prototype uses a MockTSAClient and a MockBlockchainClient. While perfectly acceptable for a Phase 0 scaffold to demonstrate workflow, these components provide no actual security guarantees and leave significant gaps in the trust model.

- **Timestamping:** The mock timestamp is simply the current system time. This provides no defense against an attacker with control of the host machine who could simply set the system clock back to create fraudulent, back-dated logs.
- **Blockchain Anchoring:** The mock blockchain client hashes the summary and returns a fake transaction hash. This provides no public verifiability or protection against censorship by the entity running the AIntegrity instance.

Recommendation: These mock services should be replaced with integrations to real-world systems to fulfill the project's stated goals of "Authenticity" (P1) and "Distributed Trust" (P6).

1. **Timestamping:** Integrate a real RFC 3161 Time-Stamp Authority (TSA) client. This is a high-impact, low-to-medium effort task. Several Python libraries, such as rfc3161-client and tsp-client, provide the necessary functionality to create a timestamp request and parse the response. These can be used with public, free, or commercial TSA servers (e.g., DigiCert's http://timestamp.digicert.com) to obtain a cryptographically signed, legally recognized proof that the audit log summary existed at a specific point in time.
2. **Blockchain Anchoring:** While setting up a full Hyperledger Fabric instance is a significant undertaking, implementing the Ethereum testnet fallback is much more achievable. This would involve using a library like web3.py to connect to an Ethereum node (via a service like Infura or Alchemy), create a transaction containing the Merkle root of the VIL session, and broadcast it to a public testnet (e.g., Sepolia). The resulting transaction hash, which is publicly and permanently verifiable on a block explorer, should be stored in the blockchain_receipt.

To summarize the required security enhancements for the VIL, the following table outlines a clear path from the current state to a hardened implementation.

Table 2: VIL Cryptographic Security Assessment

Component	Current Implementation (v6)	Recommendation	Rationale / Threat Mitigated
Hashing	hashlib.sha256	**Maintain.**	Secure, industry-standard algorithm. No changes needed.
Signing	ed25519	**Maintain.**	Modern, secure, and performant signature scheme.
Key Generation	Ed25519PrivateKey.generate() in __init__	**CRITICAL:** Externalize key generation.	Mitigates **repudiation**. Ephemeral keys prevent independent verification of the log's

Component	Current Implementation (v6)	Recommendation	Rationale / Threat Mitigated
			origin.
Key Storage	In-memory, per-session	**CRITICAL:** Load private key from a secure external source (env var, file, secrets manager).	Mitigates **key loss and theft**. Prevents hardcoding secrets and enables secure key rotation.
Timestamping	MockTSAClient	**HIGH:** Integrate a real RFC 3161 client (e.g., rfc3161-client) with a public TSA.	Mitigates **backdating attacks**. Provides strong, legally-recognized proof of existence at a specific time.
Anchoring	MockBlockchainClient	**MEDIUM:** Implement anchoring to an Ethereum testnet (e.g., Sepolia).	Mitigates **log tampering/censorship** by a central authority. Provides public, decentralized verifiability.

Part III: Implementation Review and Code-Level Refinements

This section provides a detailed review of the Python codebase itself, focusing on best practices for project structure, code quality, and usability. The goal is to offer recommendations that will improve the maintainability, scalability, and developer experience of the AIntegrity framework.

From Prototype to Professional Grade: Code Quality and Structure

The current implementation, which consolidates most of the code into a single script, is a common and effective approach for rapid prototyping. However, to mature the project into a maintainable and extensible framework, particularly one intended for open-sourcing, adopting professional-grade code quality and structural conventions is essential.

Project Structure

A well-organized project structure is the foundation of a maintainable codebase. It simplifies navigation, separates concerns, and clarifies the boundary between the core library and its supporting components like tests and documentation.
Recommendation: Adopt the "src layout," a widely accepted best practice in the Python community. This structure explicitly separates the installable package source code from other project assets. This prevents common import-related issues and makes packaging more reliable. A recommended structure for AIntegrity would be:

```
aintegrity_v6/
├── src/
|   └── aintegrity/
```

```
│       ├── __init__.py
│       ├── core/
│       │   ├── __init__.py
│       │   └── vil.py
│       ├── engines/
│       │   ├── __init__.py
│       │   └── pli_engine.py
│       ├── modules/
│       │   ├── __init__.py
│       │   └──...
│       ├── training/
│       │   ├── __init__.py
│       │   └── dataset_processor.py
│       └── utils/
│           ├── __init__.py
│           └── converters.py
├── tests/
├── docs/
├── scripts/
│   └── cli.py
├── config/
│   └── default_config.yaml
├── pyproject.toml
├── README.md
└── LICENSE
```

This structure clearly delineates the core library (src/aintegrity), tests, documentation, runnable scripts, and project configuration, making the project immediately more understandable and manageable.

Dependency Management

The project document lists numerous dependencies (e.g., cryptography, spaCy, matplotlib), but does not specify how they are managed. Relying on manual installation is not scalable or reproducible.
Recommendation: Use a pyproject.toml file as the single source of truth for project metadata, dependencies, and tool configurations. This is the modern standard for Python packaging, supported by tools like pip and poetry. It allows for the definition of core dependencies, as well as optional dependency groups (e.g., a [dev] group for testing and linting tools), ensuring that any developer can create a consistent and correct environment with a single command.

Code Style and Linting

The provided code is generally clean and readable. However, ensuring a consistent style across the entire project becomes critical as it grows and especially if it becomes a collaborative, open-source effort.
Recommendation: Integrate automated code formatting and linting into the development workflow.

- **Formatter:** Use black to automatically format all code to a consistent, opinionated style. This eliminates all debates about style and ensures uniformity.
- **Linter:** Use ruff, a modern, extremely fast Python linter, to catch a wide range of common errors, potential bugs, and stylistic issues. These tools can be configured in the pyproject.toml file and run automatically before commits (using pre-commit hooks) or in a continuous integration pipeline to maintain a high standard of code quality with minimal manual effort.

Logging and Error Handling

The current implementation includes some basic logging (logging.info in the VIL __init__) but lacks a comprehensive strategy for logging and error handling across the application. In a batch processing system like AuditDatasetProcessor, a single failing analysis on one data point could crash the entire run.
Recommendation:

1. **Implement Structured Logging:** Configure the logging module to output logs in a structured format like JSON. This makes the logs machine-readable, which is invaluable for parsing, searching, and analyzing them in production environments, especially for a tool that is itself generating audit trails.
2. **Robust Error Handling:** Wrap the calls to the individual analysis engines within the AuditDatasetProcessor.process_dataset loop in try...except blocks. If one engine fails on a particular case (e.g., the PLIEngine encounters malformed input), the exception should be caught, logged with a full stack trace, and recorded as a failure for that specific case, allowing the batch process to continue with the remaining items. This makes the system far more resilient.

Optimizing the Developer and User Experience (DX/UX)

For a developer-centric tool like AIntegrity, the quality of its interfaces—both the CLI for end-users and the API for programmatic use—is a critical factor for adoption and success.

Command-Line Interface (CLI)

The CLI is the primary user-facing component of the framework, enabling the core workflows of data conversion, claim extraction, and batch analysis. While the commands are functional, the user experience can be significantly enhanced.
Recommendation: Rebuild the CLI using **Typer**. Typer is a modern Python library for creating CLIs that is built on top of the powerful and popular Click library. Its key advantage is its use of standard Python type hints to define CLI arguments and options. This approach dramatically reduces boilerplate code compared to the standard argparse library and provides several benefits out-of-the-box:

- **Automatic Type Validation:** If an argument is type-hinted as an int, Typer automatically handles parsing and validation, providing a user-friendly error if the input is not a valid integer.
- **Automatic Help Generation:** Rich, informative help messages are generated automatically from function names, docstrings, and parameter names.
- **Shell Completion:** Typer provides automatic, easy-to-install shell completion for commands and options in all major shells (Bash, Zsh, Fish, PowerShell), a major usability

feature for frequent users.
- **Excellent Editor Support:** Because it uses standard function signatures and type hints, developers get full autocompletion and type checking in their code editors, improving the developer experience.

Migrating to Typer would professionalize the CLI, making it more intuitive for users and easier to maintain and extend for the developer.

Configuration as Code

As identified in the architectural review, a centralized configuration system is needed. This not only improves maintainability but also enhances the user experience by allowing users to easily customize the framework's behavior without altering the code.
Recommendation: Create a config.yaml file to externalize all non-secret configuration parameters. This file should be well-documented with comments explaining each setting. The application should be refactored to load this configuration at startup, allowing users to override default settings with their own config.yaml file. This empowers users to, for example, tune the scoring weights of the TrustGradingEngine for their specific use case, change default directories, or adjust logging levels easily.

API Design

The current Python API is primarily designed for the batch processing workflow via the AuditDatasetProcessor class. This is a good starting point, but its utility as a library could be expanded.
Recommendation: Refine the API to be more granular and modular. In addition to the high-level AuditDatasetProcessor, the public API of the aintegrity package should also expose the individual analysis engines. This would allow a developer using AIntegrity as a library to perform specific, targeted checks programmatically. For example, a user might want to import and use only the PLIEngine to run a plausibility check on a single claim within their own application, without the overhead of the full dataset processing pipeline. This would significantly increase the framework's value and potential use cases.

Improving the PerformanceAnalyzer

The PerformanceAnalyzer is a valuable component for summarizing the results of a batch run, but its current implementation contains a placeholder for the visualization logic: plt.hist([0.5] * metrics['total_cases']).
Recommendation: This is a straightforward but important fix to make the reporting feature genuinely useful. The generate_report method should be updated to:

1. Identify the detailed_results_{session_id}.json file corresponding to the latest metrics file.
2. Load this JSON file and parse it to extract the overall_confidence score from each individual result object.
3. Use this list of actual confidence scores as the data for plt.hist(). This simple change will transform the output from a meaningless placeholder into a valuable confidence distribution histogram, providing users with a quick and insightful overview of the audit results.

Part IV: A Synthesized Roadmap for Future Development

This section consolidates all previous strategic, architectural, and code-level recommendations into a single, prioritized, and actionable roadmap. It refines the roadmap provided in the original project document to create a clear path toward AIntegrity v7.0 and beyond.

The Path to AIntegrity v7.0

The overarching strategy for future development should be guided by a "Hierarchy of Trust." Foundational security, architectural stability, and code quality must be established before building more advanced and user-facing features. The immediate goal is to transform the prototype into a secure, robust, and developer-friendly *core engine*. The long-term goal is to build a competitive, enterprise-ready governance platform on top of this trusted foundation. The following table synthesizes the recommendations from this report into a concrete action plan. It is prioritized to ensure that the most critical issues are addressed first, maximizing the impact of development effort.

Table 3: Actionable Improvement Roadmap for AIntegrity

Priority	Recommendation	Domain	Effort	Benefit / Justification	Modules Affected
P0 (Critical)	**Implement Secure Key Management:** Load private key from env/file instead of generating it ephemerally.	Security	Small	**Fixes critical non-repudiation flaw.** Makes audit trails verifiable and meaningful.	VerifiableInteractionLedger
P0 (Critical)	**Adopt Standard Project Structure:** Migrate to an src layout with pyproject.toml.	Code Quality / DX	Medium	**Enables scalability and maintainability.** Aligns with industry best practices for future open-sourcing.	Entire Project
P0 (Critical)	**Integrate Real Timestamping:** Replace MockTSAClient with a real RFC 3161 client.	Security	Small	**Provides strong proof-of-existence.** A core requirement for a verifiable log.	VerifiableInteractionLedger
P1 (High)	**Centralize Configuration:** Manage	Architecture	Medium	**Improves flexibility and maintainability**	TrustGradingEngine, DatasetProces

Priority	Recommendation	Domain	Effort	Benefit / Justification	Modules Affected
	settings (e.g., scoring weights) via a config.yaml file.			**.** Decouples configuration from code.	sor
P1 (High)	**Upgrade CLI with Typer:** Rebuild the CLI for better usability, help text, and validation.	UX / DX	Medium	**Professionalizes the user interface.** Lowers the barrier to adoption.	cli.py (New)
P1 (High)	**Integrate Open-Source Fairness Toolkit:** Add a FairnessEngine using AIF360 or Fairlearn.	Features	Medium	**Achieves feature parity with competitors.** Addresses a critical dimension of AI ethics.	New FairnessEngine
P1 (High)	**Implement Robust Logging/Error Handling:** Add structured logging and graceful failure handling in batch processing.	Architecture	Medium	**Increases operational robustness.** Essential for debugging and production use.	AuditDatasetProcessor
P2 (Medium)	**Implement Testnet Blockchain Anchoring:** Replace MockBlockchainClient with a real Web3 client.	Security / Features	Large	**Provides public verifiability.** Fulfills the "Distributed Trust" principle.	VerifiableInteractionLedger
P2 (Medium)	**Enhance Analysis Engines:** Begin replacing heuristics with model-based components	Features	Large	**Increases analysis depth and accuracy.** Fulfills the long-term vision of the project.	PLIEngine, CitationVerifier

Priority	Recommendation	Domain	Effort	Benefit / Justification	Modules Affected
	(e.g., LogicLLaMA).				
P2 (Medium)	**Refine Reporting and Visualization:** Fix placeholder charts and expand the PerformanceAnalyzer capabilities.	Features / UX	Small	**Makes reporting genuinely useful.** Provides tangible output for users.	PerformanceAnalyzer

Revised Phased Roadmap

Based on the prioritized action plan, the project's existing roadmap can be refined into a more focused, phased approach:

- **Phase 0 (Current - Q3 2025): Foundational Hardening.** This phase should be dedicated exclusively to addressing the P0 (Critical) priorities. The singular goal is to solidify the cryptographic bedrock and professionalize the project's structure. By the end of this phase, AIntegrity should have a secure, stable, and maintainable core engine, with all critical security flaws patched.
- **Phase 1 (Q4 2025): Feature Parity and Usability.** This phase focuses on implementing all P1 (High) priorities. The objective is to transform AIntegrity into a genuinely useful and competitive tool for its target developer audience. This involves integrating key missing features (fairness), dramatically improving the primary user interface (the Typer CLI), and enhancing architectural robustness (centralized configuration, structured logging).
- **Phase 2 (Q1 2026): Open-Sourcing and Community Building.** With a stable, secure, and feature-rich core engine, the project is ready to execute its plan to become open-source. This phase should focus on activities that foster community adoption: creating high-quality documentation, writing comprehensive tutorials and usage examples, and establishing clear contribution guidelines. Engaging with the open-source community through platforms like GitHub will be crucial for building momentum and gathering feedback.
- **Phase 3 (Q2 2026 and beyond): Advanced Capabilities and Enterprise Features.** With a solid foundation and growing community, development can shift to the P2 (Medium) priorities and other long-term goals. This includes implementing the more complex, model-based analysis engines, completing the full blockchain integration, and beginning to explore enterprise-level features. These could include a web-based user interface for policy management and executive dashboards, which would allow AIntegrity to compete more directly with the top-down, commercial offerings from competitors like Credo AI and Holistic AI.

Works cited

1. How AI Compliance Audits Drive Revenue Growth - Single Grain, https://www.singlegrain.com/artificial-intelligence/how-ai-compliance-audits-drive-revenue-growt

h/ 2. AI Act | Shaping Europe's digital future, https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai 3. EU Artificial Intelligence Act | Up-to-date developments and analyses of the EU AI Act, https://artificialintelligenceact.eu/ 4. AI Governance Market Size, Growth Analysis Report 2025-2034, https://www.gminsights.com/industry-analysis/ai-governance-market 5. AI Governance Market Advances with Stronger Regulatory Frameworks and Ethical AI Adoption - Precedence Research, https://www.precedenceresearch.com/ai-governance-market 6. AI Governance Market Size, Share & Trends Report, 2030, https://www.grandviewresearch.com/industry-analysis/ai-governance-market-report 7. AI Governance Market Insights | Industry Forecast 2025-2030 - Wissen Research, https://www.wissenresearch.com/ai-governance-market-report/ 8. AI Governance Market Size, Share & Industry Trends, 2032 - Fortune Business Insights, https://www.fortunebusinessinsights.com/ai-governance-market-105975 9. EU AI Act: first regulation on artificial intelligence | Topics - European Parliament, https://www.europarl.europa.eu/topics/en/article/20230601STO93804/eu-ai-act-first-regulation-on-artificial-intelligence 10. AI governance trends: How regulation, collaboration, and skills demand are shaping the industry - The World Economic Forum, https://www.weforum.org/stories/2024/09/ai-governance-trends-to-watch/ 11. Artificial Intelligence and Compliance: Preparing for the Future of AI Governance, Risk, and ... - NAVEX, https://www.navex.com/en-us/blog/article/artificial-intelligence-and-compliance-preparing-for-the-future-of-ai-governance-risk-and-compliance/ 12. The EU AI Act: A New Era of AI Governance Began August 1st - Cloud Security Alliance, https://cloudsecurityalliance.org/articles/the-eu-ai-act-a-new-era-of-ai-governance-began-august-1st 13. AI Adoption in 2024: 74% of Companies Struggle to Achieve and Scale Value | BCG, https://www.bcg.com/press/24october2024-ai-adoption-in-2024-74-of-companies-struggle-to-achieve-and-scale-value 14. The state of AI: How organizations are rewiring to capture value - McKinsey, https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai 15. AI Governance Essential Insights for Organisations Part I Understanding Meaning Challenges Trends a, https://www.twobirds.com/en/insights/2025/ai-governance-essential-insights-for-organisations-part-i--understanding-meaning-challenges-trends-a 16. Enterprise AI Governance: Ensuring Trust and Compliance - Lumenova AI, https://www.lumenova.ai/blog/enterprise-ai-governance/ 17. Tuning Corporate Governance for AI Adoption, https://www.nacdonline.org/all-governance/governance-resources/governance-research/outlook-and-challenges/2025-governance-outlook/tuning-corporate-governance-for-ai-adoption/ 18. How AI Streamlines Compliance for Enterprise Software Development - Zencoder, https://zencoder.ai/blog/how-ai-streamlines-compliance-for-enterprise-software-development 19. Top 10 Holistic AI Alternatives & Competitors in 2025 - G2, https://www.g2.com/products/holistic-ai/competitors/alternatives 20. Credo AI - The Trusted Leader in AI Governance, https://www.credo.ai/ 21. Solutions - Credo AI, https://www.credo.ai/solutions 22. The Leader in Responsible AI - Product - Credo AI, https://www.credo.ai/product 23. Credo AI: Responsible AI Governance Platform - GOV.UK, https://www.gov.uk/ai-assurance-techniques/credo-ai-responsible-ai-governance-platform 24. Holistic AI - End to End AI Governance Platform, https://www.holisticai.com/ 25. holistic-ai/holisticai: This is an open-source tool to assess and improve the trustworthiness of AI systems. - GitHub, https://github.com/holistic-ai/holisticai 26. Holistic AI 2025 Company Profile: Valuation, Funding & Investors - PitchBook, https://pitchbook.com/profiles/company/496367-74 27. Arthur, https://www.arthur.ai/ 28. Arthur - MLOps Community,

https://mlops.community/learn/monitoring/arthur/ 29. Arthur Open-Sources First Real-Time AI Evaluation Engine, https://www.arthur.ai/blog/arthur-open-sources-first-real-time-ai-evaluation-engine 30. arthur-ai/arthur-engine: Make AI work for Everyone - Monitoring and governing for your AI/ML - GitHub, https://github.com/arthur-ai/arthur-engine 31. Fairlearn, https://fairlearn.org/ 32. Trusted-AI/AIF360: A comprehensive set of fairness metrics for datasets and machine learning models, explanations for these metrics, and algorithms to mitigate bias in datasets and models. - GitHub, https://github.com/Trusted-AI/AIF360 33. Introducing AI Fairness 360 - IBM Research, https://research.ibm.com/blog/ai-fairness-360 34. cobanov/python-yaml-guide: Python YAML Configuration Guide - GitHub, https://github.com/cobanov/python-yaml-guide 35. How to Load, Read, and Write YAML • Python Land Tutorial, https://python.land/data-processing/python-yaml 36. Working with YAML Files in Python | Better Stack Community, https://betterstack.com/community/guides/scaling-python/yaml-files-in-python/ 37. Load config from yaml - The Blue Book, https://lyz-code.github.io/blue-book/coding/python/python_config_yaml/ 38. Reading YAML config file in python and using variables - Stack Overflow, https://stackoverflow.com/questions/38404633/reading-yaml-config-file-in-python-and-using-variables 39. Reading yaml configurations in Python | by Ram Karnani - Medium, https://medium.com/@ramkarnani24/reading-yaml-configurations-in-python-40f54ce69655 40. RSA — Cryptography 46.0.0.dev1 documentation, https://cryptography.io/en/latest/hazmat/primitives/asymmetric/rsa/ 41. Key Serialization — Cryptography 46.0.0.dev1 documentation, https://cryptography.io/en/latest/hazmat/primitives/asymmetric/serialization/ 42. Python: load_pem_private_key is failing to recognize my private key in production, https://stackoverflow.com/questions/77804343/python-load-pem-private-key-is-failing-to-recognize-my-private-key-in-productio 43. How to load your private key from an environment variable - Vizzly Documentation, https://docs.vizzly.co/identity/private-key-as-environment-variable 44. rfc3161-client - PyPI, https://pypi.org/project/rfc3161-client/ 45. pyauth/tsp-client: A Python IETF Time-Stamp Protocol (TSP) (RFC 3161) client - GitHub, https://github.com/pyauth/tsp-client 46. RFC3161 compliant Time Stamp Authority (TSA) server - DigiCert Knowledge Base, https://knowledge.digicert.com/general-information/rfc3161-compliant-time-stamp-authority-server 47. Best Practices in Structuring Python Projects - Dagster, https://dagster.io/blog/python-project-best-practices 48. Setting Your Python Project Up for Success in 2024 | by Mr-Pepe | Medium, https://medium.com/@Mr_Pepe/setting-your-python-project-up-for-success-in-2024-365e53f7f31e 49. What does the structure of a modern Python project look like? - YouTube, https://www.youtube.com/watch?v=Lr1koR-YkMw 50. Alternatives, Inspiration and Comparisons - Typer, https://typer.tiangolo.com/alternatives/ 51. CLI package recommendations, Click or Typer : r/learnpython - Reddit, https://www.reddit.com/r/learnpython/comments/1j16v6j/cli_package_recommendations_click_or_typer/ 52. Typer, https://typer.tiangolo.com/

AIntegrity v4.0: Technical Specification for a Multimodal, Verifiable AI Governance Architecture

Introduction: The Imperative for Verifiable AI Integrity

The AIntegrity framework is a neuro-symbolic architecture conceived as a computational engine for critical thought, designed to automate the rigorous process of rational validation for outputs generated by Artificial Intelligence (AI) systems. Its philosophical underpinnings are rooted in a synthesis of two of the 20th century's most profound scientific methodologies: the empirical skepticism of Richard Feynman and the axiomatic rationalism of Albert Einstein. This combination creates a powerful dialectic between inductive pattern recognition and deductive logical verification, which is mirrored in the system's architecture. The Large Language Model (LLM) components act as the inductive "Guesser" (Feynman), trained on vast corpora to recognize patterns and form hypotheses about the logical structure of an input. Complementing this is the symbolic core—formal verification tools like Satisfiability Modulo Theories (SMT) solvers and Zero-Knowledge Proofs (ZKPs)—which function as the deductive "Theorist" (Einstein), rigorously testing these hypotheses against the unyielding axioms of mathematical logic.

The urgent need for the AIntegrity v4.0 architecture is driven by three fundamental shifts in the AI landscape. First, the expansion of AI beyond text into **multimodal domains** means that systems now interpret and generate images, audio, and video, creating a vastly larger surface for potential failures and adversarial attacks. Second, the increasing **autonomy and agency** of AI systems, which can now execute tasks and make decisions with real-world consequences, transforms unverifiable outputs from a data quality issue into a direct operational and safety risk. Finally, the **codification of AI accountability** through landmark regulatory frameworks has moved AI ethics from a philosophical debate to a legal and operational mandate. The full implementation of regulations like the European Union's AI Act, particularly its stringent obligations for General-Purpose AI (GPAI) models effective August 2025, and the widespread adoption of the NIST AI Risk Management Framework (RMF), demand a new class of tools capable of providing continuous, verifiable assurance.

This technical specification details a paradigm shift from AIntegrity v2.1 to v4.0. The previous version focused on *post-hoc forensic integrity*—proving cryptographically that a log of an AI interaction was not tampered with after the fact. AIntegrity v4.0 elevates this concept to *proactive, real-time, and privacy-preserving computational integrity*. The goal is no longer just to secure the record, but to securely verify the computation itself as it happens, proving that an AI's decision was made correctly, by an authorized model, and in compliance with established policies, all without compromising sensitive user data or the model's intellectual property.

Part I: AIntegrity v4.0 - A Unified Architecture for Multimodal, Verifiable AI Governance

The AIntegrity v4.0 architecture is a comprehensive, multi-stage pipeline designed to systematically ingest, deconstruct, verify, and monitor AI interactions across multiple modalities. It operates as a Directed Acyclic Graph (DAG), ensuring a logical and traceable flow of analysis where the output of one module serves as the input for subsequent stages. The high-level architecture integrates the established neuro-symbolic core of previous versions with new components for multimodal analysis, dynamic trust scoring, real-time adversarial monitoring, and a decentralized, privacy-preserving ledger that serves as the system's immutable root of trust.

The core principles guiding the v4.0 design are:

- **Verifiability by Design**: Every component is architected not merely to produce an output, but to generate a verifiable *proof* of its output's integrity and correctness. This principle moves beyond simple logging to active, computational verification.
- **Privacy by Default**: The system leverages advanced cryptographic techniques such as Zero-Knowledge Proofs (ZKPs) and Trusted Execution Environments (TEEs) to ensure that verification processes do not require exposing sensitive user data, proprietary model information, or confidential business logic. This aligns directly with the "Data Protection by Design and by Default" principle mandated by Article 25 of the GDPR.
- **Decentralized Trust**: The architecture's foundation shifts from a centralized, single-authority log to a distributed, cryptographically secured ledger. This mitigates single points of failure and control, creating a more resilient and trustworthy audit trail that is not reliant on the integrity of a single entity.
- **Continuous Assurance**: The framework moves away from periodic or post-hoc audits towards a model of real-time, continuous monitoring and risk management. This allows for the immediate detection of and response to adversarial attacks, performance degradation, and compliance violations.

The following table provides a comprehensive overview of all modules within the AIntegrity v4.0 framework, categorized by function and highlighting components that are new or have been significantly updated from version 2.1.

Module Name	Version	Primary Function	Key Dependencies	Status
Core Ingestion & Logging				
TranscriptProcessor	2.0	Ingests and structures raw multimodal conversation transcripts for analysis.	spaCy, OpenCV	Updated
VerifiableInteractionLedger	4.0	Provides a decentralized, tamper-proof, and privacy-preserving audit trail using blockchain, ZKPs, and TEEs.	Hyperledger Fabric, Circom, Intel SGX SDK	**New**
Core Analysis Modules				
PLIEngine	4.0	Performs	TranscriptProcess	Updated

Module Name	Version	Primary Function	Key Dependencies	Status
		neuro-symbolic logical analysis, fallacy detection, and formal verification on textual claims.	or, z3-solver, LogicLLaMA	
VisualConsistency Verifier	1.0	Assesses semantic consistency between images and associated text using CLIP-based models.	TranscriptProcess or, CLIP	**New**
MediaIntegrityAss essor	1.0	Verifies the integrity of image, video, and audio inputs using perceptual hashing.	TranscriptProcess or, pHash	**New**
ComplianceScanM odule	2.0	Scans content for legal, regulatory, and policy violations (e.g., GDPR, PII, hate speech).	TranscriptProcess or, Presidio	Updated
CitationVerifier	4.0	Verifies the validity, format, and factual support of citations in AI output, including DOI and URL resolution.	TranscriptProcess or, NLI models	Updated
Behavioral & Trust Modules				
SessionDriftDetect or	4.0	Monitors for semantic and factual contradictions across an entire conversation session.	TranscriptProcess or, sentence-transfor mers	Updated
AdversarialThreat Monitor	1.0	Provides real-time detection of adversarial attacks, prompt	All inputs/outputs, SHAP	**New**

Module Name	Version	Primary Function	Key Dependencies	Status
		injections, and anomalous behavior using drift metrics and XAI.		
TrustGradingEngine	4.0	Calculates a dynamic, multi-dimensional trust score with a temporal decay model.	All analysis modules	Updated
Remediation & Enforcement				
ReconstructionAdvisor	2.0	Generates actionable suggestions for resolving detected inconsistencies or improving AI outputs.	PLIEngine, SessionDriftDetector	Updated
SentinelEnforcementCore	2.0	Consolidates all findings to enforce final safety and compliance guardrails (e.g., halt, flag, alert).	All analysis modules	Updated
Governance & Reporting				
RegulatoryComplianceMapper	1.0	Maps audit findings and system features to specific requirements of the EU AI Act and NIST AI RMF.	All analysis modules, VerifiableInteractionLedger	**New**
AuditTraceVisualizer	2.0	Produces visual representations (graphs, timelines) of the verifiable audit trail.	VerifiableInteractionLedger	Updated

Part II: The Cryptographic Bedrock - Verifiable Interaction Ledger (VIL) v4.0

The foundation of AIntegrity v4.0 is the Verifiable Interaction Ledger (VIL), a significant architectural evolution from the Verifiable Interaction Logging (VIL) Engine of previous versions. Where the VIL Engine focused on creating a tamper-evident, centralized log file , the VIL v4.0 is

a decentralized, append-only, and privacy-preserving ledger designed to serve as an unimpeachable root of trust for all AI interactions and system operations. Its design is predicated on the principles of durable, append-only logs, which ensure that data, once written, cannot be altered, providing a permanent and verifiable history of events.

Blockchain-Based Trust Anchors

To achieve decentralized and immutable storage, the VIL is built upon a permissioned blockchain network. This choice is critical for enterprise applications, where control over network participation and data confidentiality are paramount.

Protocol Selection: Hyperledger Fabric

A permissioned blockchain is superior to a public, permissionless one (like the main Ethereum network) for enterprise audit trails. Public blockchains are, by design, transparent to all participants, which directly conflicts with the need to protect proprietary business data, sensitive user information under GDPR, and the AI model's intellectual property. Furthermore, public chains often suffer from lower transaction throughput and unpredictable costs (gas fees), making them unsuitable for the high-volume, real-time logging required by AIntegrity. Hyperledger Fabric, a project hosted by the Linux Foundation, is selected as the underlying framework for the VIL. It provides a modular architecture that excels in the areas critical for this use case: privacy through "channels" that allow for confidential transactions between specific participants, higher performance due to its pluggable consensus mechanisms, and a permissioned governance model that aligns with enterprise compliance and control requirements. The following table justifies this selection.

Feature	Hyperledger Fabric	Ethereum (Public L1/L2)	Certificate Transparency	Rationale for AIntegrity v4.0
Permissioning	Permissioned (Known Identities)	Permissionless (Anonymous)	Public (Known CAs)	Essential for enterprise accountability. Only authorized nodes (e.g., internal audit, regulators) can join the network.
Privacy	High (Private Channels)	Low (Public Ledger)	Public by Design	Critical for protecting sensitive user data (GDPR) and proprietary model information. Channels can isolate logs for different clients or use cases.
Throughput	High (e.g., >2000 TPS)	Low to Medium (Varies by L2)	N/A (Not a transaction	Necessary for real-time logging

Feature	Hyperledger Fabric	Ethereum (Public L1/L2)	Certificate Transparency	Rationale for AIntegrity v4.0
			system)	of high-frequency AI interactions without becoming a bottleneck.
Transaction Costs	Low / Predictable	High / Volatile (Gas Fees)	N/A	Enterprise systems require predictable operational costs, which volatile gas fees on public networks preclude.
Governance	Consortium-based	Decentralized Community	Browser/CA Forum	A defined governance model allows enterprises to set and enforce policies for the audit trail itself.
Suitability	**Excellent**	Poor	Unsuitable	Fabric provides the optimal balance of immutability, privacy, performance, and control for enterprise-grade, verifiable AI audit trails.

On-Chain Data Structure and Smart Contracts

Every auditable action within the AIntegrity system is recorded as a transaction on the Hyperledger Fabric ledger. The data payload for each transaction is a JSON object representing an AuditEvent.
The AuditEvent structure is defined as follows:

```
{
  "eventId": "string (UUID)",
  "sessionId": "string (UUID)",
  "timestampUTC": "string (ISO 8601)",
  "eventType": "string (e.g., INPUT, ANALYSIS, TRUST_GRADING)",
  "actorId": "string (Decentralized Identifier for user/agent)",
  "contentHash": "string (SHA-256 hash of the event content)",
  "prevEventHash": "string (SHA-256 hash of the previous event's
on-chain record)",
  "proofPayload": {
    "proofType": "string (e.g., 'ZKP_SNARK', 'TEE_ATTESTATION')",
    "proofData": "object (The ZKP or TEE attestation data)"
```

```
    }
}
```

A **smart contract** (or "chaincode" in Fabric terminology) governs all interactions with the ledger. This chaincode, written in a general-purpose language like Go or Node.js, enforces the integrity rules of the audit trail. Its key functions include:

- logEvent(eventDetails): This function accepts a new AuditEvent. It verifies that the prevEventHash matches the hash of the last recorded event in that session, enforcing a strict, unbreakable chronological chain. It also validates the structure of the proofPayload before committing the new event to the ledger.
- getEvent(eventId): Retrieves a specific event record from the ledger.
- verifyEventProof(eventId): This function takes an eventId and triggers an on-chain or off-chain verification process for the associated proofPayload. For a ZKP, it would run the verifier logic against the proof data. For a TEE attestation, it would check the signature against the hardware vendor's public keys.

Privacy-Preserving Verifiable Computation

A key innovation in v4.0 is the ability to not just record that an analysis was performed, but to prove that it was performed *correctly* and *privately*. This is achieved by combining ZKPs and TEEs, which are complementary technologies addressing different aspects of the verifiability challenge. ZKPs are ideal for proving the correctness of auditable, rule-based logic without revealing the inputs, while TEEs are suited for proving the integrity of the execution environment for proprietary, "black-box" AI models.

Zero-Knowledge Proofs (ZKPs) for Verifiable Logic

For deterministic modules where the logic must be auditable, such as the ComplianceScanModule, zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) are employed. This allows AIntegrity to prove, for example, that a block of text was scanned for PII and that all detected PII was redacted, without revealing the PII itself to the blockchain or an auditor.
The implementation follows the **Circom and snarkjs** workflow :

1. **Circuit Design (Circom)**: The compliance rule is encoded as an arithmetic circuit. For a PII redaction check, the circuit would take a public input (the hash of the original text) and a private input, or "witness" (the original text and the list of detected PII entities). The circuit's logic would verify that hashing the redacted text produces a different, expected hash, proving the redaction occurred.
2. **Witness Generation**: The ComplianceScanModule executes its logic (e.g., using the Presidio library) and uses the results to generate the witness for the circuit.
3. **Proof Generation (snarkjs)**: The system uses the generated witness and a proving key to create a compact zk-SNARK proof. This proof is a small data object that mathematically attests to the correct execution of the circuit.
4. **On-Chain Verification**: The generated proof is placed in the proofPayload of the AuditEvent and submitted to the VIL. The verifyEventProof function in the smart contract can then run the corresponding zk-SNARK verifier algorithm against the proof, confirming its validity in a decentralized and tamper-proof manner.

Trusted Execution Environments (TEEs) for Confidential Model Execution

For complex, proprietary AI models (e.g., a custom fraud detection model or a fine-tuned LLM in the PLIEngine), creating a ZKP circuit is computationally infeasible. Here, the goal is to protect the model's intellectual property while proving that the specific, authorized version of the model was executed on the input data without tampering. This is achieved using **Trusted Execution Environments (TEEs)**, such as **Intel Software Guard Extensions (SGX)**.
The workflow is as follows:

1. **Enclave Execution**: The proprietary AI model is loaded and executed inside an SGX enclave, a hardware-isolated memory region protected from the host operating system and any other processes.
2. **Remote Attestation**: The enclave generates a cryptographic attestation. This is a data structure signed by the CPU's unique private key, which contains a hash of the code and data loaded into the enclave.
3. **Attestation Verification**: An external party (or the VIL smart contract) can verify this attestation using Intel's public keys. A successful verification proves that a specific, known model binary (identified by its hash) was executed on a genuine Intel SGX-enabled CPU.
4. **Ledger Record**: The generated attestation is placed in the proofPayload of the AuditEvent and logged on the VIL, creating an immutable record that links a specific model execution to a specific input and output.

By combining these two technologies, AIntegrity v4.0 provides a multi-layered verification system that can offer strong, privacy-preserving integrity guarantees across a wide range of AI components, from simple, auditable rules to complex, proprietary models.

Part III: Advanced Analysis and Scoring Modules

AIntegrity v4.0 introduces a suite of new and updated analysis modules to handle the complexities of multimodal inputs, dynamic trust assessment, and real-time adversarial threats. These modules provide the core analytical capabilities that feed into the Verifiable Interaction Ledger.

Multimodal Verification Engine

The architecture expands beyond text-only analysis to address the growing prevalence of multimodal AI systems.

VisualConsistencyVerifier

This module is designed to assess the semantic consistency between images and their associated textual descriptions, a critical task for detecting a model's compositional understanding failures. It is built using a pre-trained **Contrastive Language-Image Pre-training (CLIP)** model. The core mechanism involves:

1. **Embedding Generation**: The module uses a CLIP model (e.g., ViT-L/14) to generate separate vector embeddings for the input image and the input text.
2. **Similarity Scoring**: It then calculates the cosine similarity between the image embedding and the text embedding. A high similarity score indicates strong alignment, while a low

score suggests a potential contradiction, hallucination, or irrelevant pairing.

3. **Addressing Compositional Weakness**: Standard CLIP models are known to exhibit "bag-of-words" behavior, where they might correctly identify objects and attributes but fail to bind them correctly (e.g., confusing "a red square and a blue circle" with "a blue square and a red circle"). To mitigate this, the VisualConsistencyVerifier incorporates advanced techniques like **Linear Attribute Binding CLIP (LABCLIP)**, which applies a learned linear transformation to the text embeddings before the similarity comparison to improve attribute-object binding accuracy. This allows for more nuanced verification of compositional claims.

MediaIntegrityAssessor

This module is responsible for verifying the integrity of media files (images, videos, audio) against manipulation and forgery, such as deepfakes. Unlike cryptographic hashes that change with any single-bit modification, this module uses **perceptual hashing**. A perceptual hash algorithm generates a "fingerprint" of a media file based on its actual content, making it robust to benign alterations like compression, resizing, or format changes, while still being sensitive to significant content modifications.
The process involves:

1. **Feature Extraction**: For images and video frames, the algorithm extracts key visual features. For audio, it first converts the audio signal into a **spectrogram**—a visual representation of the spectrum of frequencies—and then extracts features from this image.
2. **Hash Generation**: These features are then converted into a compact hash value (e.g., using an open-source library like pHash).
3. **Integrity Check**: The generated hash can be compared against a database of known media hashes to detect duplicates or near-duplicates. For an audit trail, the perceptual hash of an input media file is logged on the VIL. If the same content appears later, its hash can be compared to the ledger to verify its origin and detect potential tampering over time.

Dynamic Trust Scoring Engine

AIntegrity v4.0 moves beyond the static, per-session trust scores of previous versions to a dynamic model that reflects the temporal nature of trust. Trust is not a permanent state; it can decay over time in the absence of reinforcing evidence.

The TrustDecayModel

The TrustGradingEngineV4 calculates a multi-dimensional overall_score based on inputs from all analysis modules. However, this score is now subject to a **temporal decay function**. This models the principle that confidence in an AI agent's reliability should decrease if it has not been successfully evaluated recently.
The decay is modeled using a **Logistic function**, which provides a smooth, S-shaped curve representing a gradual loss of trust that levels off at a baseline.
The mathematical formulation is: $T(t) = T_{min} + \frac{T_{max} - T_{min}}{1 + e^{k(t - t_{mid})}}$
Where:

- $T(t)$ is the trust score at time t.

- T_{max} is the maximum trust score, typically set to the overall_score of the last successful interaction.
- T_{min} is the asymptotic minimum trust score, representing a baseline level of distrust or neutrality (e.g., 0.2).
- k is the decay rate constant, which determines how quickly trust erodes.
- t is the time elapsed since the last interaction.
- t_{mid} is the midpoint of the decay, representing the time at which the trust score is decaying most rapidly.

This model is event-driven:

- **Positive Events**: A new interaction that results in a high overall_score resets the clock (t=0) and sets a new T_{max}.
- **Negative Events**: A detected failure (e.g., a contradiction or compliance violation) can result in an immediate penalty to the current trust score and may also increase the decay rate k, reflecting that trust is harder to maintain after a breach.

Real-Time Adversarial Monitoring & Anomaly Detection

To provide proactive defense, the AdversarialThreatMonitor module continuously analyzes the stream of inputs and outputs of the AI system in real-time.

Drift Detection

The module monitors for two primary types of drift, which can indicate either a change in the operating environment or a subtle adversarial attack :

1. **Data Drift (Covariate Shift)**: This occurs when the statistical properties of the input data change. For example, if a customer service bot trained on formal language starts receiving queries in informal slang.
2. **Concept Drift**: This occurs when the relationship between inputs and correct outputs changes. For example, a financial advice bot's recommendations may become invalid after a sudden market shift.

The monitor uses the **Population Stability Index (PSI)** to quantify this drift. It compares the distribution of features in the current data stream against a baseline (e.g., the training data or a stable production period). A PSI value above a predefined threshold (e.g., **PSI > 0.2**) signifies a significant distribution shift, triggering an alert for model retraining or investigation.

XAI-Based Anomaly Detection

For detecting novel or sophisticated adversarial attacks that may not cause significant statistical drift, the monitor employs an eXplainable AI (XAI) based approach. This technique moves beyond simply looking at the input data to analyzing the AI's *internal reasoning* for its response. The process is as follows:

1. **Baseline Feature Importance**: During a baseline period, the system uses a technique like **SHAP (SHapley Additive exPlanations)** to calculate the feature importance values for a representative set of legitimate prompts. This creates a profile of which parts of an input "normally" influence the model's output.
2. **Real-Time Monitoring**: For each incoming prompt, the monitor calculates its SHAP values in real-time.
3. **Anomaly Detection**: It then compares the SHAP value distribution of the new prompt to

the established baseline. An input that is syntactically benign but triggers a highly anomalous internal response (i.e., the model focuses on unusual parts of the prompt) is flagged as a potential adversarial attack, such as an indirect prompt injection. This allows the system to detect attacks based on their effect on the model's reasoning process, even if the attack vector has never been seen before.

Part IV: Regulatory and Governance Alignment

AIntegrity v4.0 is architected to serve as a foundational technology for organizations seeking to comply with the increasingly complex global landscape of AI regulations. It provides the technical mechanisms to meet the stringent requirements of the EU AI Act and to operationalize the principles of the NIST AI Risk Management Framework.

Conformance with the EU AI Act

The platform directly addresses the obligations for providers of General-Purpose AI (GPAI) models, which came into effect in August 2025.

- **Technical Documentation and Record-Keeping (Article 53)**: The EU AI Act requires GPAI providers to draw up and maintain extensive technical documentation. The Verifiable Interaction Ledger (VIL) serves as the core of this compliance, providing an immutable, timestamped record of a model's lifecycle, including:
    - **Training Data Summaries**: The VIL can store a cryptographically hashed summary of the datasets used for training, providing a verifiable record without needing to store the data itself. The CitationVerifierV4 and MediaIntegrityAssessor modules provide the tools to generate these summaries, fulfilling the need to document the use of copyright-protected content.
    - **Evaluation and Testing Results**: All outputs from the AIntegrity analysis and monitoring modules are logged to the VIL, creating an auditable history of the model's performance, robustness, and cybersecurity posture.
- **Transparency and Copyright Obligations (Article 53)**: The Act mandates transparency regarding training data, particularly copyrighted material. The VIL, populated by the CitationVerifierV4, provides a mechanism to create the required "sufficiently detailed summary" of training content. Smart contracts on the ledger can enforce policies related to honoring opt-outs (e.g., robots.txt), creating a verifiable compliance trail.
- **Systemic Risk Mitigation (Article 55)**: For GPAI models designated as having systemic risk (those trained with over 10^{25} FLOPs), the Act imposes heightened obligations for model evaluation, risk assessment, and incident reporting. AIntegrity v4.0 is designed for this purpose:
    - The AdversarialThreatMonitor provides the continuous model evaluation and real-time threat detection required.
    - The integrated red-teaming capabilities allow for systematic adversarial testing.
    - The VIL provides an immutable log for reporting serious incidents to the AI Office and national authorities, as required.

Implementation of the NIST AI Risk Management Framework

The NIST AI RMF provides a voluntary but widely adopted structure for managing AI risks

throughout the system lifecycle. AIntegrity v4.0 is not merely a system that can be audited *using* the RMF; it is a platform that *operationalizes* the RMF's core functions, transforming them from a static, document-based exercise into a dynamic, continuous assurance process.

- **Govern**: This function is about establishing a risk management culture and accountability. AIntegrity implements this by allowing AI usage policies and risk thresholds to be encoded directly into the system's configuration and smart contracts. The VIL provides the transparent, auditable foundation required for all governance activities.
- **Map**: This function involves identifying the context and potential impacts of an AI system. The VIL serves as a real-time, comprehensive map of all AI interactions, logging the context, actors, inputs, outputs, and analysis results for every event. This creates a complete inventory for risk assessment.
- **Measure**: This function focuses on assessing and monitoring AI risks. The entire suite of AIntegrity's analysis modules—from the PLIEngine and TrustGradingEngine to the AdversarialThreatMonitor—directly implements the Measure function. They provide a continuous stream of quantitative and qualitative metrics on fairness, bias, accuracy, robustness, and security.
- **Manage**: This function is about allocating resources to treat identified risks. The SentinelEnforcementCore is a direct implementation of the Manage function, providing automated and semi-automated mechanisms to respond to and mitigate risks in real-time (e.g., halting a response, flagging for human review, or isolating a potentially compromised agent).

By embedding these functions into its core architecture, AIntegrity v4.0 enables organizations to move from periodic compliance checks to a state of continuous, verifiable AI assurance. The following table provides a clear mapping between the platform's features and these key regulatory frameworks.

AIntegrity v4.0 Feature/Module	NIST AI RMF Function	EU AI Act Obligation (Article)	Description of Alignment
Verifiable Interaction Ledger (VIL)	Govern, Map	Technical Documentation (Art. 53)	Creates an immutable, auditable inventory of all AI interactions, training data summaries, and evaluation results, forming the foundation for governance and compliance documentation.
CitationVerifierV4	Measure	Copyright Transparency (Art. 53)	Scans AI outputs and training data to produce the "sufficiently detailed summary" of copyrighted content required by the Act.
AdversarialThreatMonitor	Measure, Manage	Systemic Risk Mitigation (Art. 55)	Provides continuous model evaluation, adversarial testing, and real-time incident

AIntegrity v4.0 Feature/Module	NIST AI RMF Function	EU AI Act Obligation (Article)	Description of Alignment
			detection to manage risks for high-impact GPAI models.
ZKP & TEE Integration	Govern, Measure	Accuracy, Robustness, Cybersecurity (Title III)	Provides privacy-preserving proof of compliance with internal policies and the integrity of model execution, enhancing accountability and security.
SentinelEnforcement Core	Manage	Corrective Actions (Title III)	Enforces predefined risk thresholds and policies, enabling automated or semi-automated responses to non-compliant or unsafe AI behavior.
RegulatoryComplianceMapper	Govern	Overall Compliance	Provides a direct bridge between technical audit data and regulatory reporting requirements, streamlining compliance workflows.

Part V: Complete Technical Specification and Implementation

This section provides the complete, implementation-ready technical specification for the AIntegrity v4.0 system. The implementation is presented in Python 3.9+ and builds upon the foundational codebase of AIntegrity v2.1, integrating the new modules and architectural paradigms described in this document.

Core Data Structures and Enumerations

The core data structures are updated to support multimodality and the expanded set of analysis events.

```
# aintegrity/core/data_structures.py

import json
import hashlib
from dataclasses import dataclass, field, asdict
from enum import Enum
```

```
from typing import List, Dict, Any, Optional, Union
import datetime
import uuid

class EventType(Enum):
    """Enumeration of all possible event types in the AIntegrity
log."""
    # Core Interaction
    USER_INPUT = "USER_INPUT"
    MODEL_OUTPUT = "MODEL_OUTPUT"

    # Analysis Events
    LOGICAL_ANALYSIS = "LOGICAL_ANALYSIS"
    VISUAL_CONSISTENCY_ANALYSIS = "VISUAL_CONSISTENCY_ANALYSIS"
    MEDIA_INTEGRITY_ANALYSIS = "MEDIA_INTEGRITY_ANALYSIS"
    COMPLIANCE_ANALYSIS = "COMPLIANCE_ANALYSIS"
    CITATION_ANALYSIS = "CITATION_ANALYSIS"
    SESSION_DRIFT_ANALYSIS = "SESSION_DRIFT_ANALYSIS"
    ADVERSARIAL_THREAT_ANALYSIS = "ADVERSARIAL_THREAT_ANALYSIS"

    # Scoring and Remediation
    TRUST_GRADING = "TRUST_GRADING"
    RECONSTRUCTION = "RECONSTRUCTION"

    # System and Governance
    ENFORCEMENT_ACTION = "ENFORCEMENT_ACTION"
    SYSTEM_STATE = "SYSTEM_STATE"

class ModalityType(Enum):
    """Enumeration for content modalities."""
    TEXT = "text"
    IMAGE = "image"
    AUDIO = "audio"
    VIDEO = "video"
    CODE = "code"

@dataclass
class ContentBlock:
    """Represents a piece of content with a specific modality."""
    modality: ModalityType
    data: Union[str, bytes]  # Text as string, binary data for others
    metadata: Dict[str, Any] = field(default_factory=dict)

    def to_dict(self):
        data_repr = self.data if isinstance(self.data, str) else
self.data.hex()
        return {
            "modality": self.modality.value,
```

```
            "data": data_repr,
            "metadata": self.metadata
        }

@dataclass
class ProofPayload:
    """Container for verifiable computation proofs."""
    proof_type: str  # e.g., "ZKP_SNARK_GROTH16",
"TEE_INTEL_SGX_QUOTE"
    proof_data: Dict[str, Any]
    verification_key_id: Optional[str] = None

@dataclass
class AuditEvent:
    """Represents a single, verifiable event in an interaction
session."""
    event_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    session_id: str = ""
    timestamp_utc: str = field(default_factory=lambda:
datetime.datetime.utcnow().isoformat() + "Z")
    event_type: EventType = EventType.SYSTEM_STATE
    actor_id: str = "system"  # Can be user DID, agent DID, or module
name
    content_blocks: List = field(default_factory=list)
    analysis_payload: Dict[str, Any] = field(default_factory=dict)
    parent_event_id: Optional[str] = None

    # Cryptographic elements added by the VIL
    content_hash: Optional[str] = None
    prev_event_hash: Optional[str] = None
    signature_b64: Optional[str] = None
    proof: Optional[ProofPayload] = None

    def to_canonical_dict(self, include_crypto: bool = False) ->
Dict[str, Any]:
        """Creates a dictionary representation for hashing and
signing."""
        d = {
            "event_id": self.event_id,
            "session_id": self.session_id,
            "timestamp_utc": self.timestamp_utc,
            "event_type": self.event_type.value,
            "actor_id": self.actor_id,
            "content_blocks": [block.to_dict() for block in
self.content_blocks],
            "analysis_payload": self.analysis_payload,
            "parent_event_id": self.parent_event_id,
        }
```

```
        if include_crypto:
            d["content_hash"] = self.content_hash
            d["prev_event_hash"] = self.prev_event_hash
            d["signature_b64"] = self.signature_b64
            if self.proof:
                d["proof"] = asdict(self.proof)
        return d

    def compute_content_hash(self) -> str:
        """Computes the SHA-256 hash of the canonical event
content."""
        # The content hash covers the core data, excluding other
crypto fields
        payload_to_hash = self.to_canonical_dict(include_crypto=False)
        canonical_str = json.dumps(payload_to_hash, sort_keys=True,
separators=(",", ":"))
        return
hashlib.sha256(canonical_str.encode('utf-8')).hexdigest()

@dataclass
class SessionSummary:
    """Cryptographic summary of a completed and sealed session."""
    schema_version: str = "4.0"
    session_id: str
    event_count: int
    start_time_utc: str
    sealed_time_utc: str
    final_event_hash: str
    merkle_root: str
    signing_key_id: str  # e.g., a DID or public key identifier
    tsa_token_rfc3161_b64: Optional[str] = None # Trusted Timestamp
    blockchain_receipt: Optional] = None # e.g., {"tx_hash": "...",
"block_number":...}
```

Verifiable Interaction Ledger (VIL) v4.0

The VerifiableInteractionLedger replaces the VILEngine. It manages the cryptographic lifecycle of events, including signing, hashing, and anchoring to the blockchain.

```
# aintegrity/core/vil.py

import base64
import hashlib
import json
from typing import List, Dict, Any, Optional

from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ed25519
```

```
# Placeholder for blockchain and TSA client implementations
from.blockchain_client import AbstractBlockchainClient,
MockBlockchainClient
from.tsa_client import AbstractTSAClient, MockTSAClient
from.data_structures import AuditEvent, SessionSummary

class VerifiableInteractionLedger:
    """
    Manages the creation, cryptographic sealing, and decentralized
anchoring of audit events.
    Version: 4.0
    """
    def __init__(self, session_id: str, blockchain_client: Optional =
None, tsa_client: Optional = None):
        self.session_id = session_id
        self.events: List[AuditEvent] =
        self.merkle_leaves: List[str] =
        self.last_event_hash: Optional[str] = None
        self.start_time_utc = datetime.datetime.utcnow().isoformat() +
"Z"

        # Cryptographic key management
        self.private_key = ed25519.Ed25519PrivateKey.generate()
        self.public_key = self.private_key.public_key()
        self.key_id = f"did:key:{self.get_public_key_b58()}"

        # External service clients
        self.blockchain_client = blockchain_client or
MockBlockchainClient()
        self.tsa_client = tsa_client or MockTSAClient()

    def get_public_key_pem(self) -> str:
        return self.public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ).decode('utf-8')

    def get_public_key_b58(self) -> str:
        # Example for did:key format (multicodec prefix for Ed25519 +
raw public key)
        raw_bytes = self.public_key.public_bytes(
            encoding=serialization.Encoding.Raw,
            format=serialization.PublicFormat.Raw
        )
        # 0xed is the multicodec prefix for ed25519-pub
        prefixed_bytes = b'\xed\x01' + raw_bytes
        # A full implementation would use a proper base58btc encoder
```

```
        return base64.b85encode(prefixed_bytes).decode('ascii')

    def _sign_hash(self, data_hash: str) -> str:
        """Signs a hex digest string with the session's private
key."""
        signature = self.private_key.sign(bytes.fromhex(data_hash))
        return base64.b64encode(signature).decode('utf-8')

    def _build_merkle_root(self, leaves: List[str]) -> str:
        """Builds a Merkle tree and returns the root hash."""
        if not leaves:
            return hashlib.sha256(b"").hexdigest()

        current_level = leaves[:]
        while len(current_level) > 1:
            if len(current_level) % 2!= 0:
                current_level.append(current_level[-1]) # Duplicate
last element if odd

            next_level =
            for i in range(0, len(current_level), 2):
                left = current_level[i]
                right = current_level[i+1]
                combined = (left + right).encode('utf-8')

next_level.append(hashlib.sha256(combined).hexdigest())
            current_level = next_level
        return current_level

    def log_event(self, event: AuditEvent) -> AuditEvent:
        """Processes, seals, and logs a single audit event."""
        event.session_id = self.session_id

        # 1. Chain hash
        event.prev_event_hash = self.last_event_hash

        # 2. Content hash
        event.content_hash = event.compute_content_hash()

        # 3. Signature
        # The signature covers the hash of the canonical
representation of the event *header*
        # This is a simplified JWS-like approach
        header_plus_payload_dict = {
            "header": {
                "alg": "EdDSA",
                "kid": self.key_id
            },
```

```
            "payload": event.to_canonical_dict(include_crypto=False)
        }
        canonical_header_payload_str =
json.dumps(header_plus_payload_dict, sort_keys=True, separators=(",",
":"))
        message_to_sign_hash =
hashlib.sha256(canonical_header_payload_str.encode('utf-8')).hexdigest
()
        event.signature_b64 = self._sign_hash(message_to_sign_hash)

        # 4. Update ledger state
        self.events.append(event)
        self.merkle_leaves.append(event.content_hash)

        # The hash for the next event's prev_event_hash is the hash of
the full signed event record
        full_event_dict = event.to_canonical_dict(include_crypto=True)
        canonical_full_event_str = json.dumps(full_event_dict,
sort_keys=True, separators=(",", ":"))
        self.last_event_hash =
hashlib.sha256(canonical_full_event_str.encode('utf-8')).hexdigest()

        return event

    def seal_and_anchor_session(self) -> SessionSummary:
        """Finalizes the session, generates the summary, and anchors
it to the blockchain."""
        if not self.events:
            raise ValueError("Cannot seal an empty session.")

        sealed_time_utc = datetime.datetime.utcnow().isoformat() + "Z"
        merkle_root = self._build_merkle_root(self.merkle_leaves)

        summary = SessionSummary(
            session_id=self.session_id,
            event_count=len(self.events),
            start_time_utc=self.start_time_utc,
            sealed_time_utc=sealed_time_utc,
            final_event_hash=self.last_event_hash,
            merkle_root=merkle_root,
            signing_key_id=self.key_id
        )

        # Anchor the summary hash
        summary_dict = asdict(summary)
        summary_str = json.dumps(summary_dict, sort_keys=True,
separators=(",", ":"))
        summary_hash =
```

```
hashlib.sha256(summary_str.encode('utf-8')).hexdigest()

        # 1. Get Trusted Timestamp
        summary.tsa_token_rfc3161_b64 =
self.tsa_client.get_timestamp_token(summary_hash.encode('utf-8'))

        # 2. Anchor to Blockchain
        summary.blockchain_receipt =
self.blockchain_client.anchor_hash(summary_hash)

        return summary
```

Multimodal Verification Engine

This new engine contains modules for handling non-textual data, providing critical verification for modern AI systems.

```
# aintegrity/modules/multimodal_verifier.py

from typing import List, Dict, Any
from PIL import Image
import io
import imagehash # For perceptual hashing (pHash)
import torch
from transformers import CLIPProcessor, CLIPModel

from aintegrity.core.data_structures import ContentBlock, ModalityType

class VisualConsistencyVerifier:
    """
    Verifies semantic consistency between image and text content
blocks using CLIP.
    Version: 1.0
    """
    def __init__(self, model_name: str =
"openai/clip-vit-large-patch14"):
        self.device = "cuda" if torch.cuda.is_available() else "cpu"
        self.model =
CLIPModel.from_pretrained(model_name).to(self.device)
        self.processor = CLIPProcessor.from_pretrained(model_name)

    def verify(self, content_blocks: List) -> Dict[str, Any]:
        """
        Calculates the consistency score between the first image and
first text block.
        """
        image_block = next((b for b in content_blocks if b.modality ==
ModalityType.IMAGE), None)
```

```
        text_block = next((b for b in content_blocks if b.modality ==
ModalityType.TEXT), None)

        if not image_block or not text_block:
            return {"status": "SKIPPED", "reason": "Missing required
image or text modality."}

        try:
            image = Image.open(io.BytesIO(image_block.data))
            text = text_block.data

            inputs = self.processor(text=[text], images=image,
return_tensors="pt", padding=True).to(self.device)

            with torch.no_grad():
                outputs = self.model(**inputs)

            logits_per_image = outputs.logits_per_image
            probability = logits_per_image.softmax(dim=1).item()

            return {
                "status": "COMPLETED",
                "consistency_score": probability,
                "model_used": self.model.config.name_or_path
            }
        except Exception as e:
            return {"status": "ERROR", "reason": str(e)}

class MediaIntegrityAssessor:
    """
    Assesses the integrity of media files using perceptual hashing.
    Version: 1.0
    """
    def assess(self, content_block: ContentBlock) -> Dict[str, Any]:
        """Generates a perceptual hash for a given media content
block."""
        if content_block.modality not in:
            return {"status": "SKIPPED", "reason": f"Unsupported
modality for pHash: {content_block.modality.value}"}

        try:
            if content_block.modality == ModalityType.IMAGE:
                image = Image.open(io.BytesIO(content_block.data))
                # Average hashing is simple and fast
                phash = str(imagehash.average_hash(image))
                return {
                    "status": "COMPLETED",
                    "perceptual_hash": phash,
```